



清华大学计算机科学与技术系本科专业主修课

# 计算机系统结构

## 第三章 流水线技术

2026年春季学期

提升处理器性能=减少CPU时间

$$CPU\text{时间} = \left( \sum_{i=1}^n CPI_i \times IC_i \right) \times \text{时钟周期}$$



$$CPU\text{时间} = \left( \sum_{i=1}^n CPI_i \times \text{时钟周期} \right) \times IC_i$$

流水线技术



**时间重叠原理：**把一件工作按功能分割为若干相互联系的部分；把每一部分指定给专门的部件完成；然后按时间重叠原理把各部分的执行过程在时间上重叠起来，使所有部件依次分工完成一组同样的工作

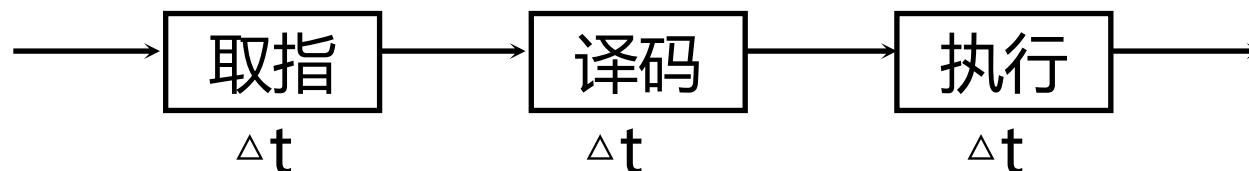
- 3. 1 [流水线的基本概念](#)
- 3. 2 [流水线的性能指标](#)
- 3. 3 [非线性流水线的调度](#)
- 3. 4 [流水线的相关与冲突](#)
- 3. 5 [流水线RISC-V的实现](#)

## 3. 1 流水线的基本概念

# 3.1.1 什么是流水线

第 5 页

**基本过程：** 一个指令执行过程可以被分解为若干子过程。



■ 顺序执行



n条指令的执行时间为：

$$T = \sum_{i=1}^n (t_{\text{取指 } i} + t_{\text{分析 } i} + t_{\text{执行 } i})$$
$$= 3 n \Delta t$$

优点：

➤ 控制简单，节省设备。

缺点：

➤ 处理器执行指令的速度慢

➤ 功能部件的利用率很低

# 3.1.1 什么是流水线

第 6 页

**工业流水线的思想**把一个重复的过程分解为若干个子过程，每个子过程由专门的功能部件来实现。

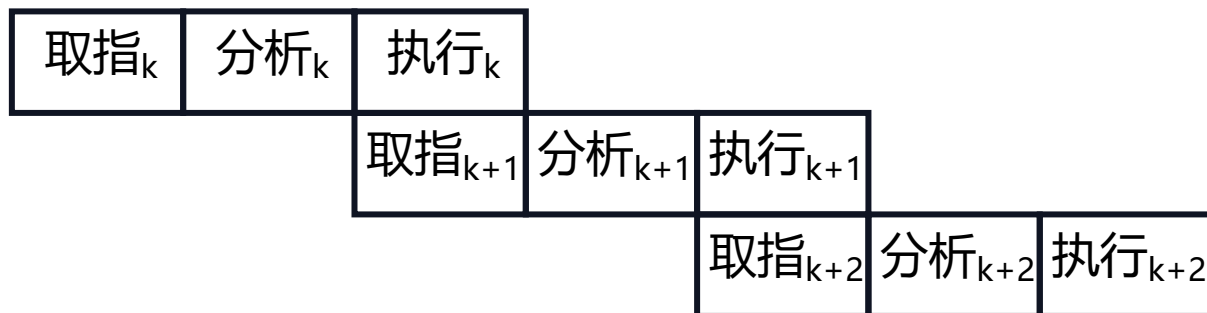


把多个处理过程在时间上错开，依次通过各功能段，这样，每个子过程就可以与其它子过程并行进行

# 3.1.1 什么是流水线

第 7 页

**基本思想：** 一个指令执行过程可以被分解为若干子过程，每个子过程可以与其它子过程同时进行。



$n$ 条指令的执行时间为： $T = (1 + 2n) \cdot \Delta t$

优点：

- 执行指令的速度更快
- 功能部件的利用率更高

$n$ 条指令的执行时间为： $T = (2 + n) \cdot \Delta t$

缺点：

- 控制更复杂，需增加设备

# 3.1.1 什么是流水线

第 8 页

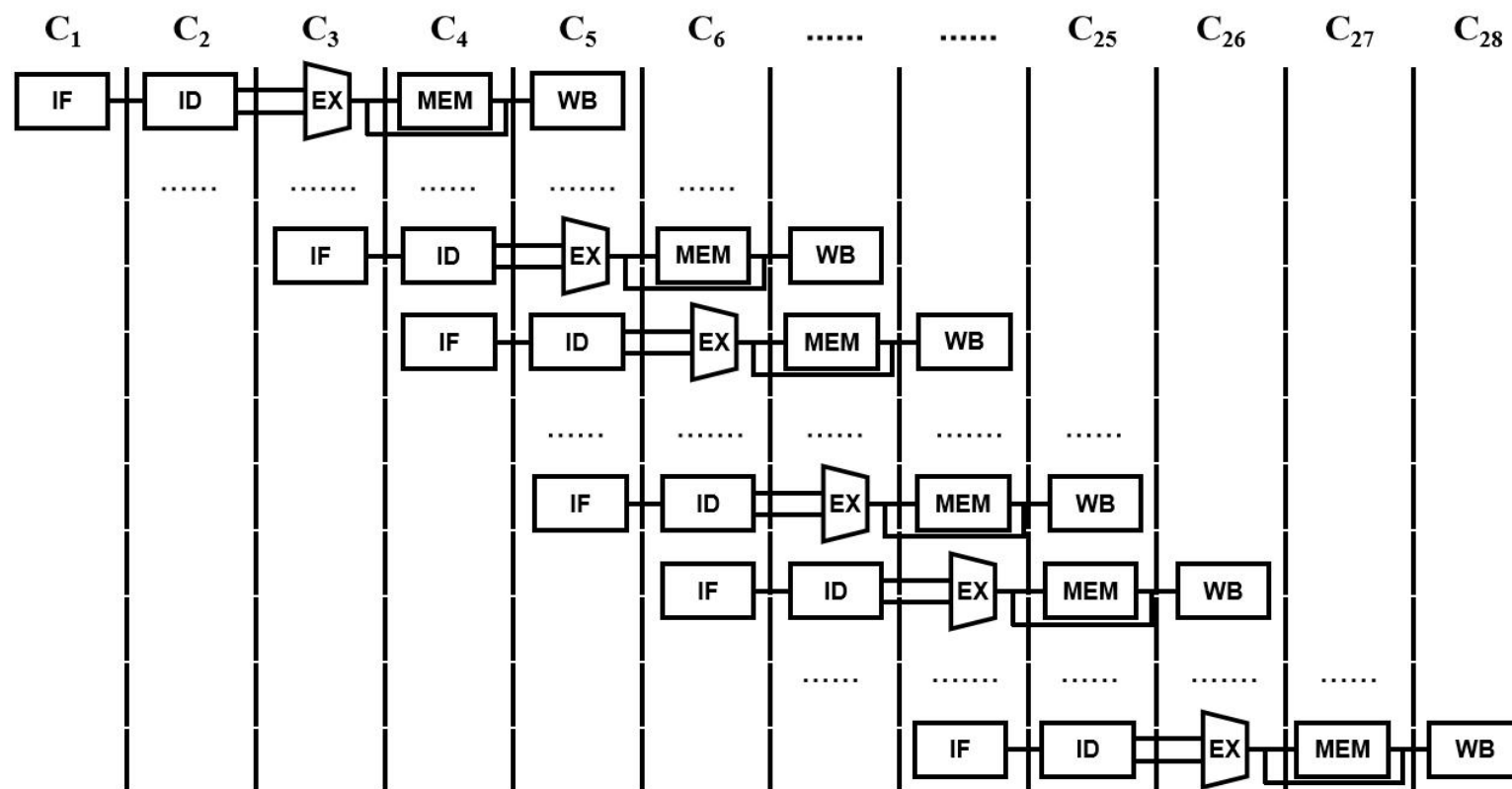
流水线的实现前提：

## ■ 空间并行性

- ◆ 指令操作可以由多个独立的操作部件共同完成

## ■ 时间并行性

- ◆ 不同指令可以分时使用同一个部件的不同部分



5段指令流水线 → **理想情况下**速度提高5倍

如何分析不理想情况下的流水线执行的实际效果？



## 3.1.1 什么是流水线

第 9 页

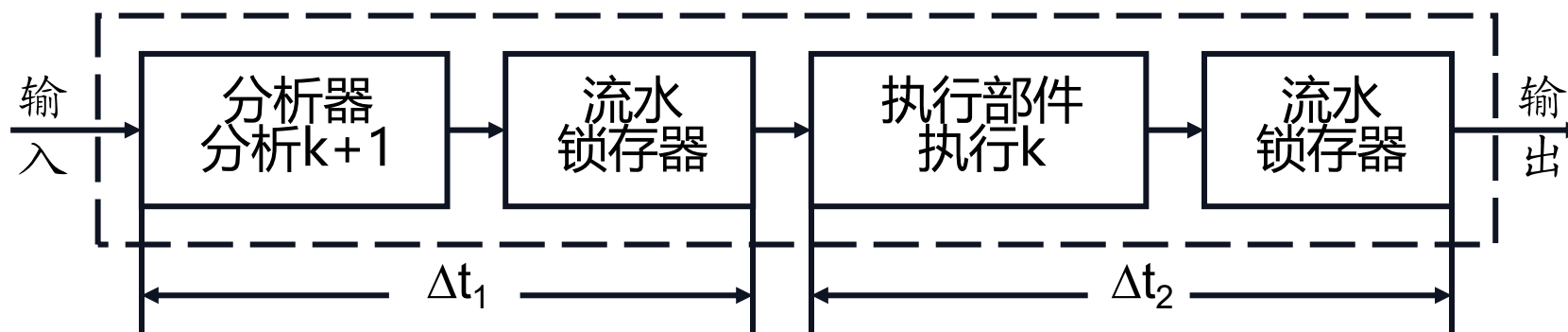
### ■ 流水线表示方法与分析手段：

- 连接图
- 时空图

# 3.1.1 什么是流水线

第 10 页

## ■ 连接图

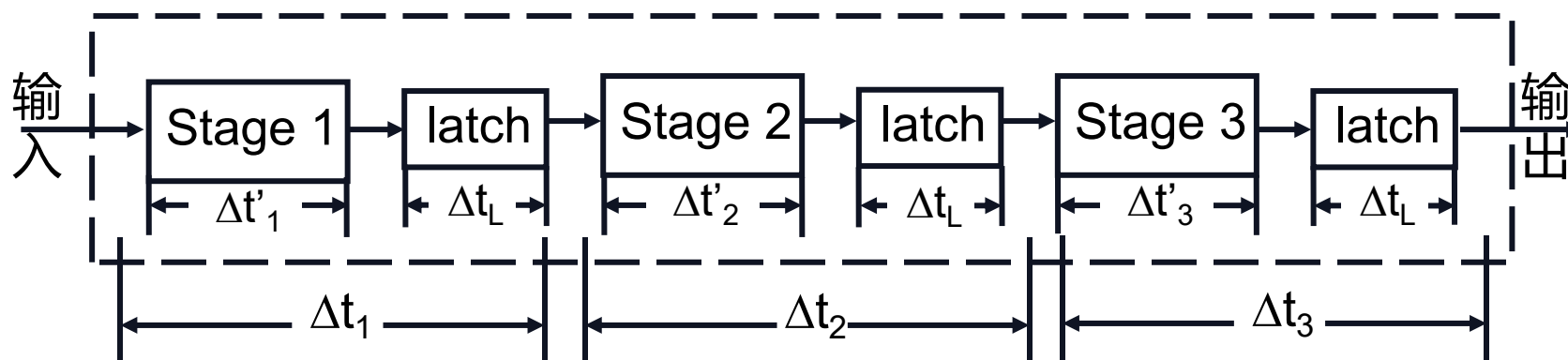


- **功能段：**流水线的每一个阶段称为流水步、流水步骤、流水段、流水线阶段、流水功能段、流水级、流水节拍等。
- **流水寄存器：**在每一个流水段的末尾或开头必须设置一个寄存器，称为流水寄存器、流水锁存器等。
  - 作用：在相邻的两段之间传送数据，以保证提供后面要用到的信息，并把各段的处理工作相互隔离。

## 3.1.1 什么是流水线

第 11 页

### ■ 连接图



非流水执行

$$T_{\text{线}} = \Delta t'_1 + \Delta t'_2 + \Delta t'_3 \quad (\text{三级操作})$$

流水执行

$$T_{\text{流}} = \max(\Delta t_1, \Delta t_2, \Delta t_3)$$

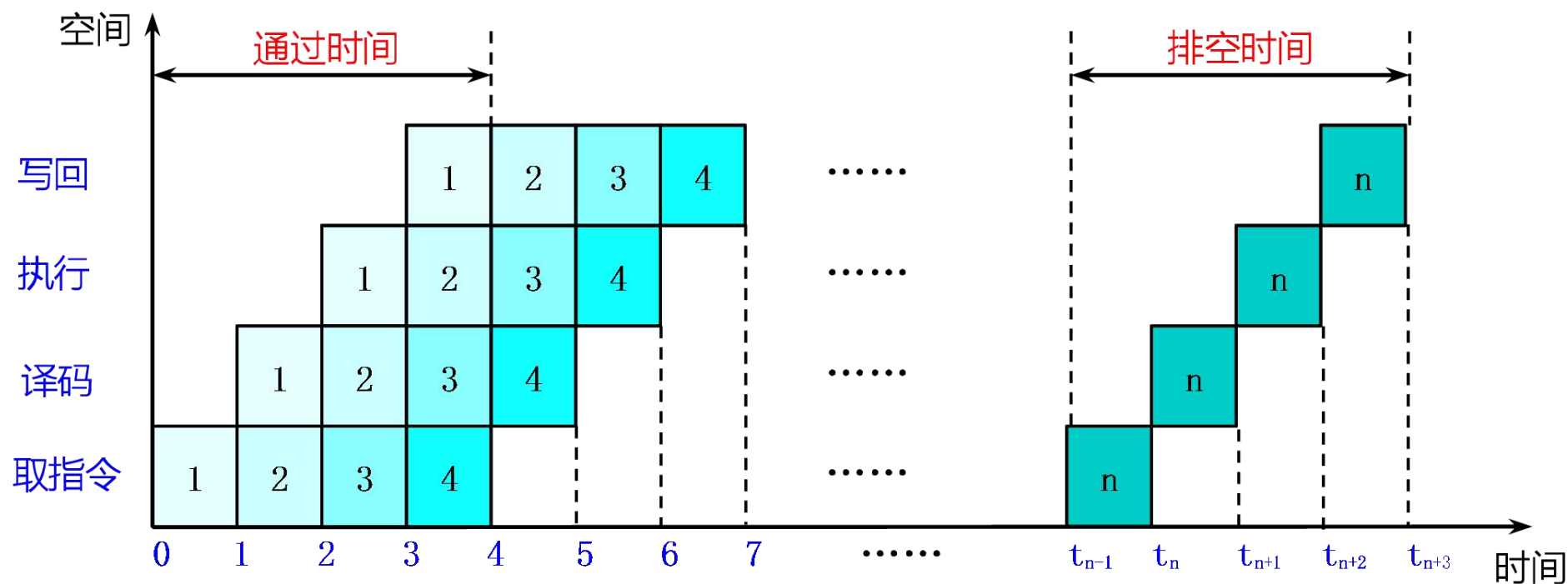
如果要使 $T_{\text{流}}$ 最小, 则要求 $\Delta t_1 \approx \Delta t_2 \approx \Delta t_3 \approx \frac{T_{\text{线}}}{3} + \Delta t_L$

■ 流水线中各段的时间应尽可能相等, 否则时间最长的段将成为流水线的瓶颈

## 3.1.1 什么是流水线

第 12 页

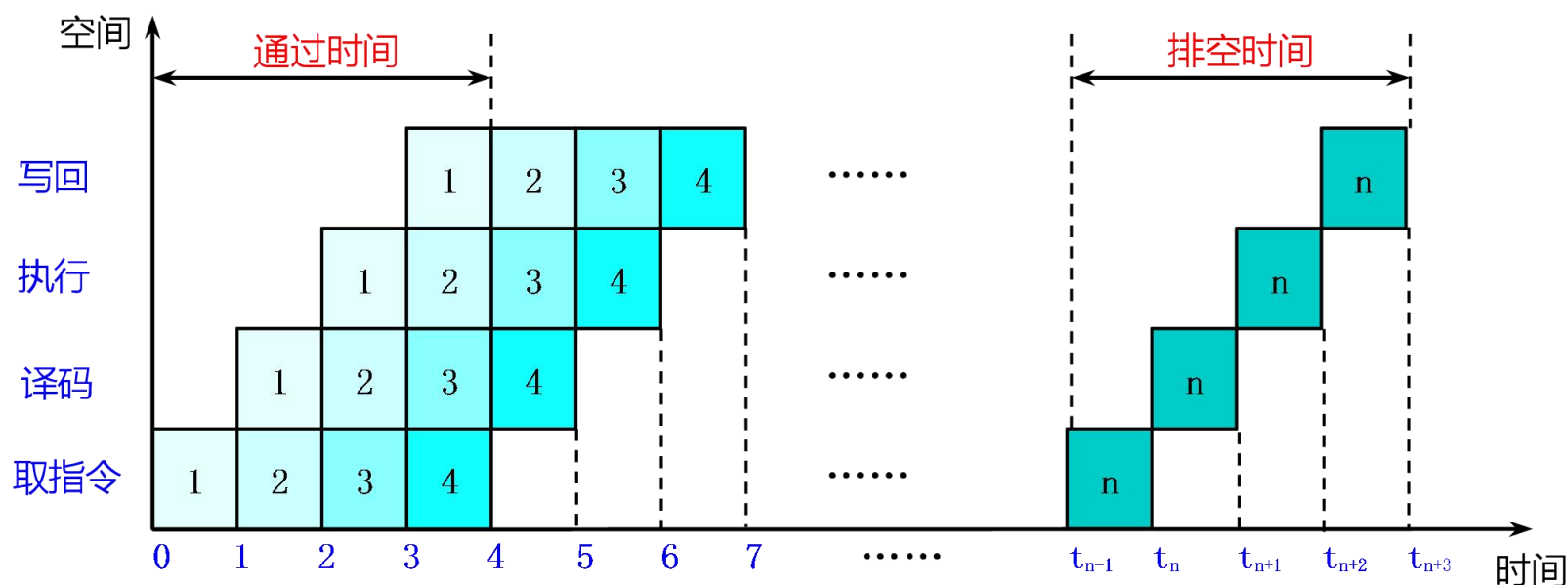
- **时—空图**从时间和空间两个方面描述了流水线的工作过程
- 时—空图中横坐标代表时间，纵坐标代表流水线的各个阶段指令执行流水线的时空图



# 3.1.1 什么是流水线

第 13 页

- **通过时间:** 第一个任务从进入流水线到流出结果所需的时间
- **排空时间:** 最后一个任务从进入流水线到流出结果所需的时间



$$T = (n + 3) \cdot \Delta t$$

$$T = (n + m - 1) \cdot \Delta t$$

$n$ : 指令数

$m$ : 流水线段数

$\Delta t$ : 表示一段流水线的执行时间

问题: 每一段流水线的执行时间是否相同, 即如何确定 $\Delta t$ 值?

## 3.1.1 什么是流水线

第 14 页

流水线的设计需求：

- 只有连续提供同类任务才能发挥流水线效率

- 尽量减少因条件分支造成的“断流”，通过编译技术提供连续的相同类型操作。

- 每个流水线段都要设置一个流水寄存器

- 用于保存本流水线段的执行结果，是流水线中需要增加的主要硬件。

- 各流水段的时间应尽量相等

- 流水线处理机的基本时钟周期等于时间最长的流水段的时间长度。

## 3.1.2 流水线分类

第 15 页

### ■ 按照流水技术的应用等级不同

- 部件级流水线：运算操作分段执行
- 处理器级流水线：指令分段执行
- 系统级流水线：程序（任务）分段执行



粒度从低到高

### ■ 按照流水线所完成的功能来分类

- 单功能流水线
- 多功能流水线
  - ✓ 静态流水线
  - ✓ 动态流水线（线性流水线与非线性流水线）

## 3.1.2 流水线分类

第 16 页

- 按照任务流入和流出的顺序是否相同进行分类
  - 顺序流水线
  - 乱序流水线
- 按照处理数据对象来进行分类
  - 标量流水线
  - 向量流水线



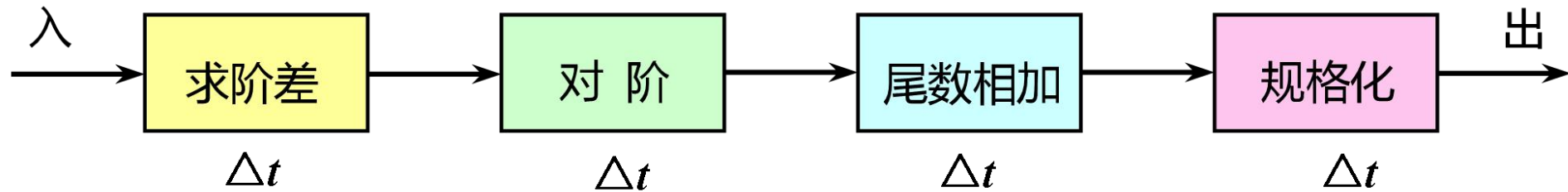
## 3.1.2 流水线分类

第 17 页

- **部件级流水线**（运算操作流水线）：把处理机中的部件分段，再把这些分段相互连接起来，使得各种类型的运算操作能够按流水方式进行。

### 浮点加法流水线：

把浮点加法的全过程分解为求阶差、对阶、尾数相加、规格化等四个子过程



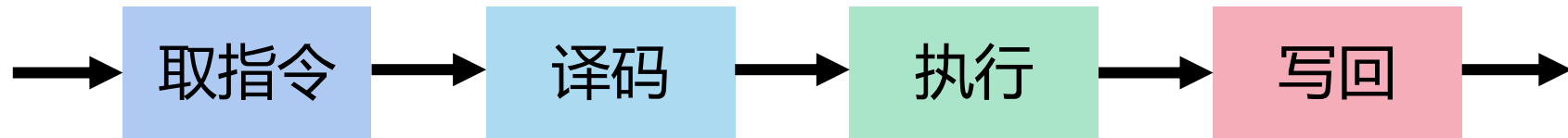
## 3.1.2 流水线分类

第 18 页

- **处理器级流水线**（指令流水线）：把指令的执行过程按照流水方式处理。把一条指令的执行过程分解为若干个子过程，每个子过程在独立的功能部件中执行。

### 指令流水线：

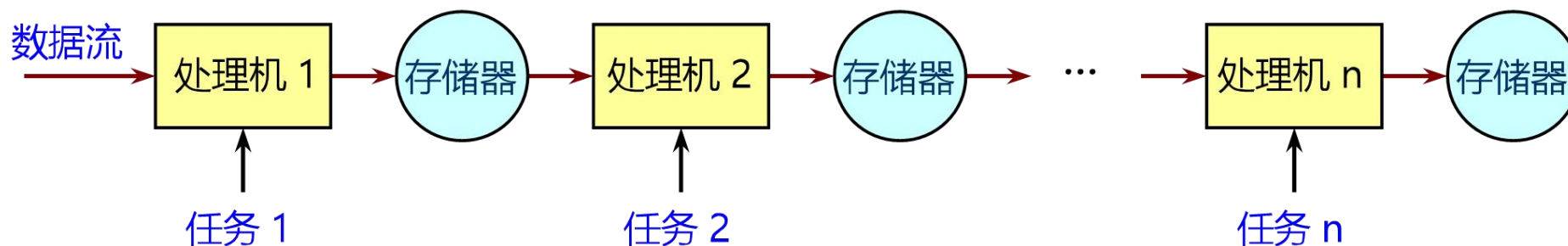
把指令执行的全过程分解为取指令、译码、执行、写回结果等四个子过程



## 3.1.2 流水线分类

第 19 页

- **系统级流水线**（宏流水线）：把多台处理机串行连接起来，对同一数据流进行处理，每个处理机完成整个任务中的一部分。

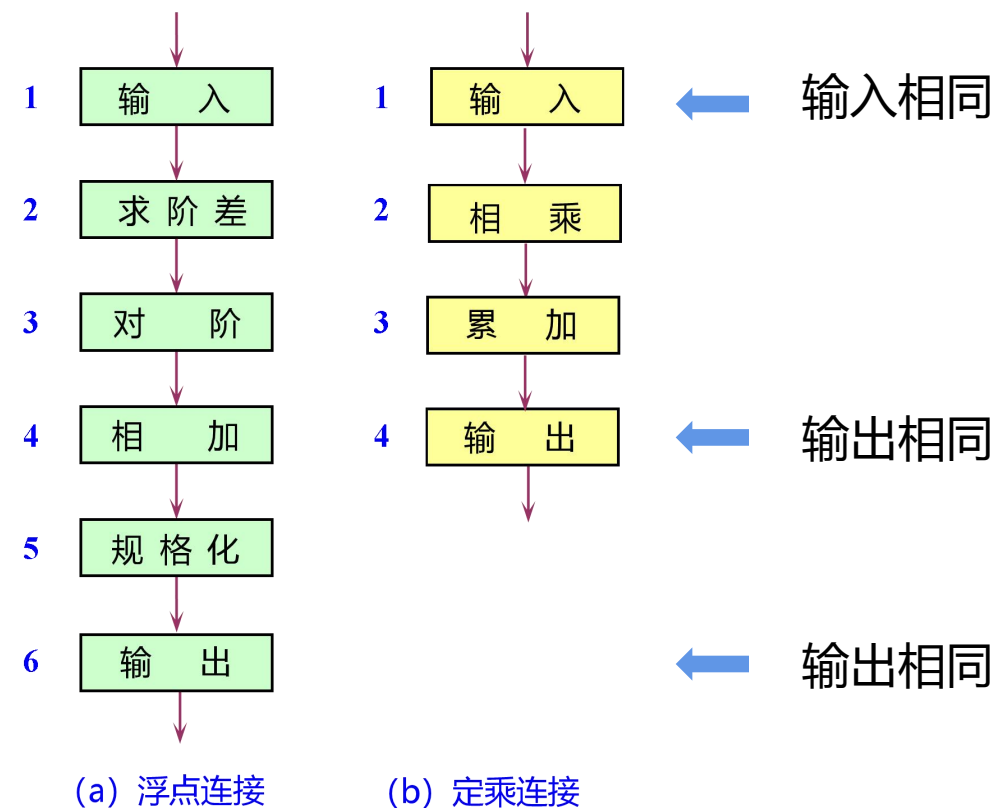


## 3.1.2 流水线分类

第 20 页

### ■单功能流水线与多功能流水线 (按照流水线所完成的功能来分类)

➤ **单功能流水线：**只能完成一种固定功能的流水线。



**一次只执行一条指令?**

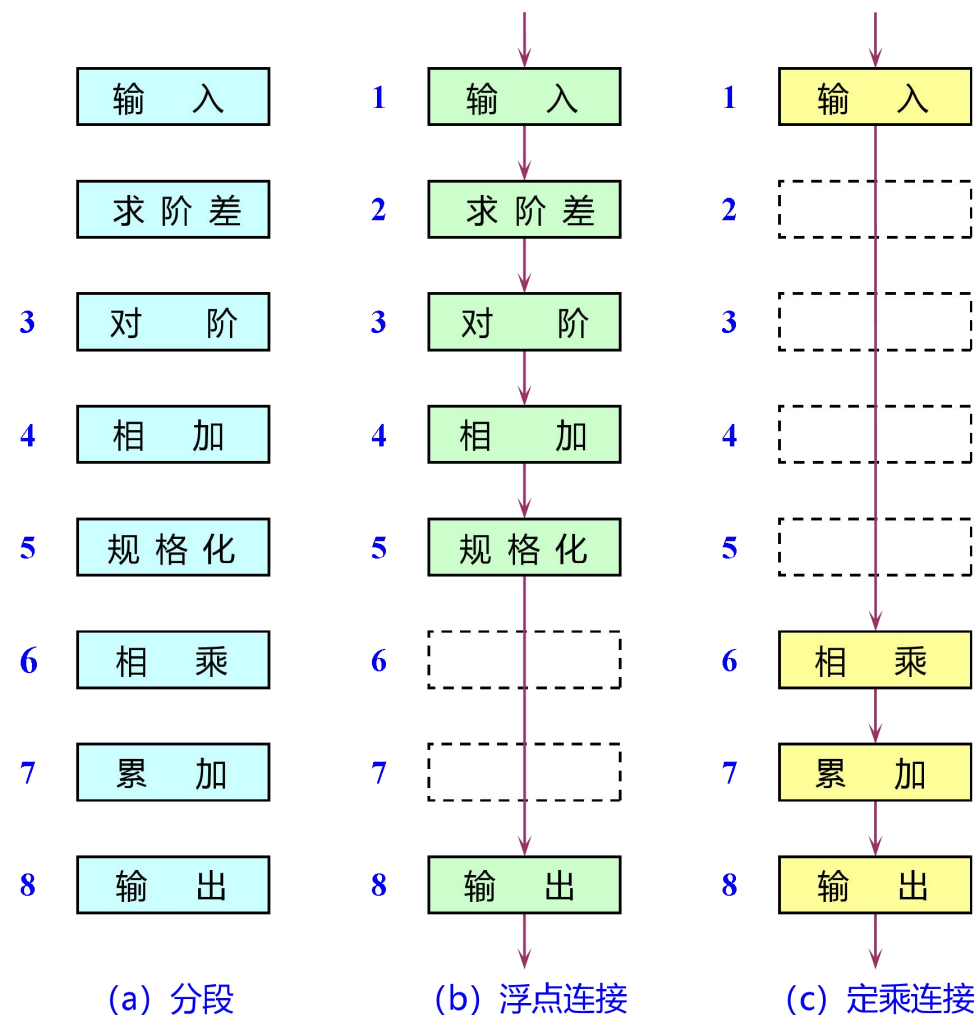
## 3.1.2 流水线分类

第 21 页

### ■单功能流水线与多功能流水线 (按照流水线所完成的功能来分类)

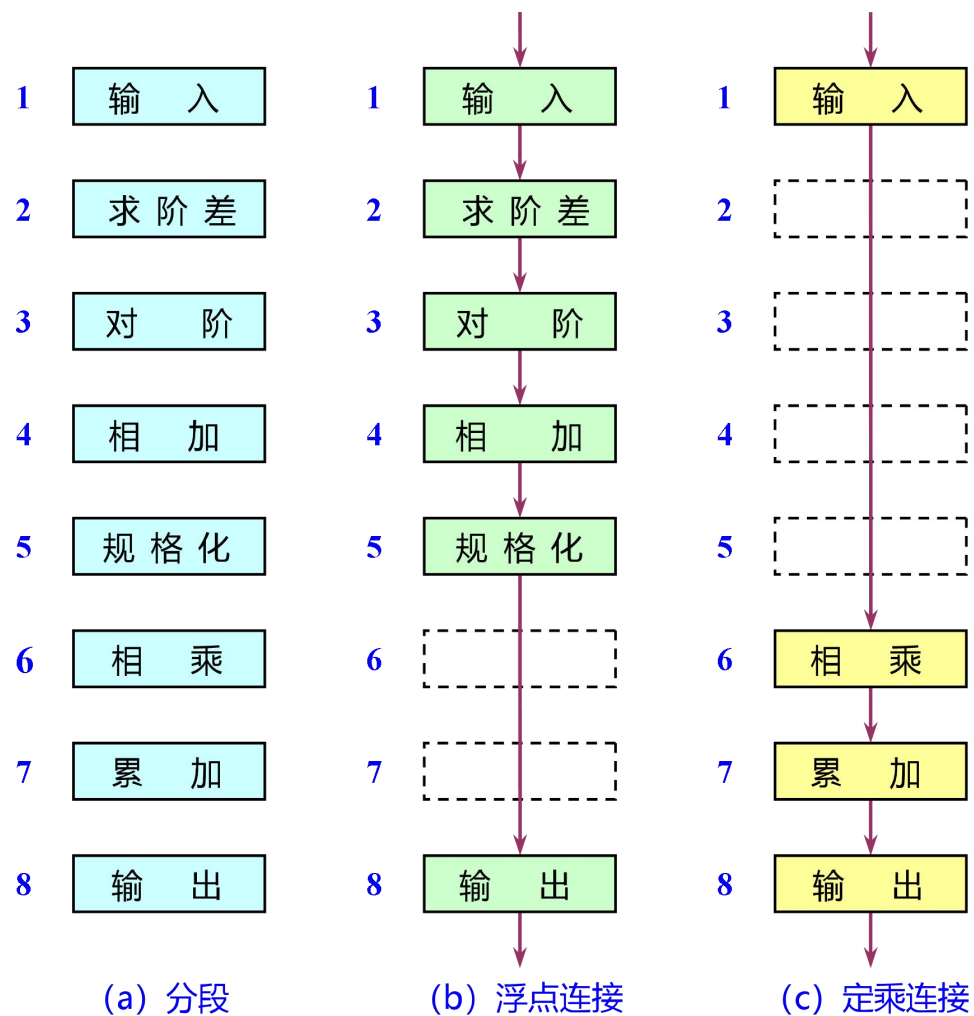
- **单功能流水线：**只能完成一种固定功能的流水线。
- **多功能流水线：**流水线的各段可以进行不同的连接，以实现不同的功能。

例：ASC的多功能流水线



## 3.1.2 流水线分类

第 22 页



■ 按照同一时间内各段之间的连接方式  
对多功能流水线作进一步的分类

- 静态流水线
- 动态流水线

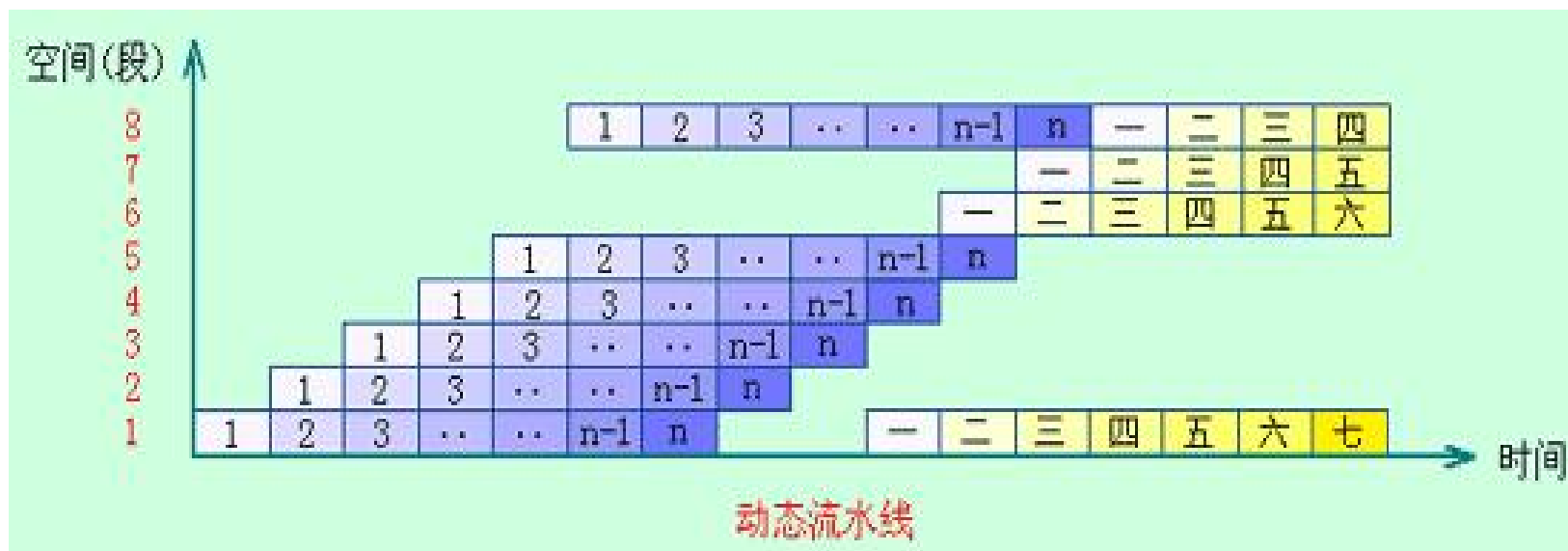


## 3.1.2 流水线分类

第 24 页

■ **动态流水线：**在同一时间内，多功能流水线中的各段可以按照不同的方式连接，同时执行多种功能

- 优点：灵活，能够提高流水线各段的使用率，从而提高处理速度
- 缺点：控制复杂



假设该流水线先执行若干浮点操作，再执行若干定点乘法操作



## 3.1.2 流水线分类

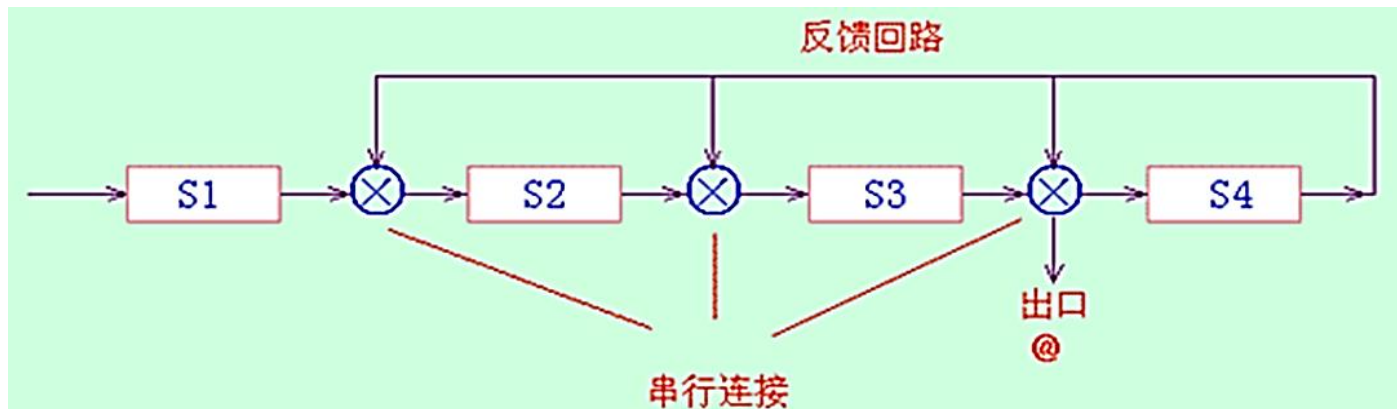
第 25 页

### ■ 按照流水线中是否有反馈回路来进行分类

- 线性流水线：流水线的各段串行连接，没有反馈回路。数据通过流水线中的各段时，每一个段最多只流过一次



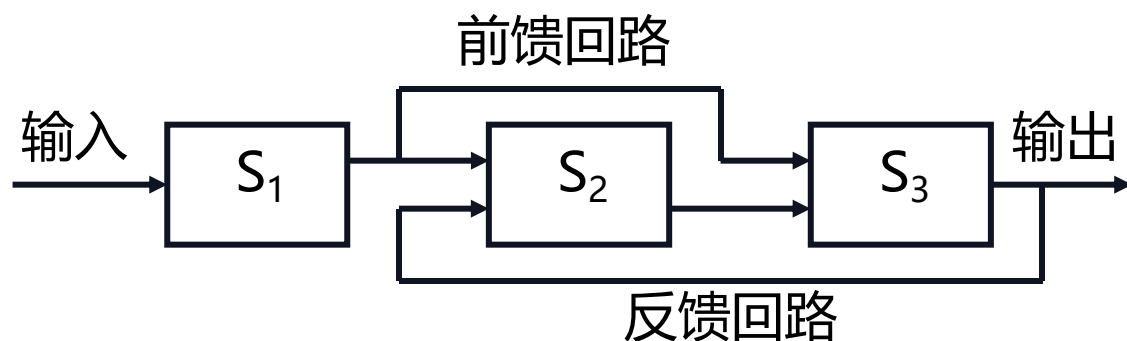
- 非线性流水线：流水线中除了有串行的连接外，还有反馈回路



## 3.1.2 流水线分类

第 26 页

- 非线性流水线必须用**连接图**和**预约表**共同表示



一种非线性流水线的连接图

预约表1:

|       | 1 | 2 | 3 | 4 |
|-------|---|---|---|---|
| $S_1$ | × |   |   |   |
| $S_2$ |   |   | × |   |
| $S_3$ |   | × |   | × |

预约表2:

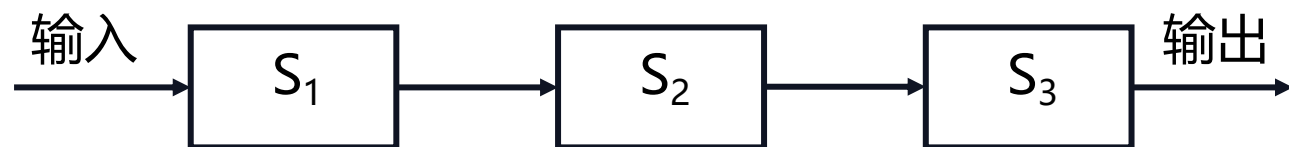
|       | 1 | 2 | 3 | 4 | 5 |
|-------|---|---|---|---|---|
| $S_1$ | × |   |   |   |   |
| $S_2$ |   | × |   | × |   |
| $S_3$ |   |   | × |   | × |

- 一条非线性流水线可以对应有很多张预约表，同样一张预约表实际上仅表示了一条非线性流水线的一种工作方式

## 3.1.2 流水线分类

第 27 页

- 线性流水线能够用**连接图**唯一表示，其实流水线实际上也有预约表，只不过它的预约表是固定的



一种线性流水线的连接图

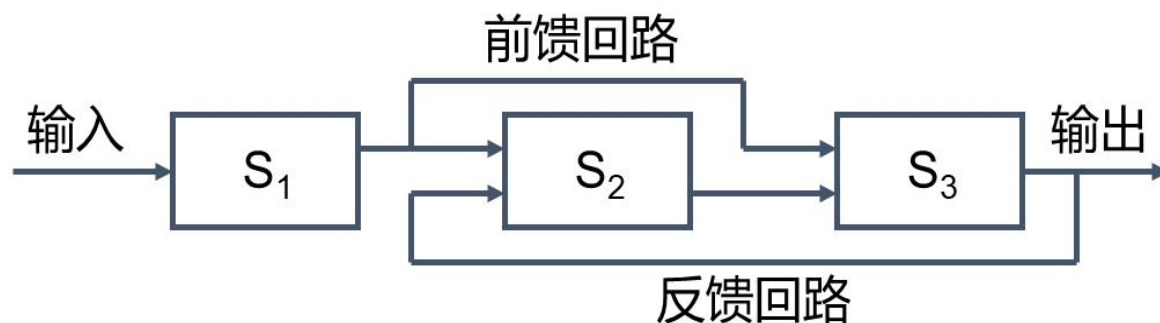


|       | 1 | 2 | 3 |
|-------|---|---|---|
| $S_1$ | × |   |   |
| $S_2$ |   | × |   |
| $S_3$ |   |   | × |

一种线性流水线的预约表

## 3.1.2 流水线分类

第 28 页

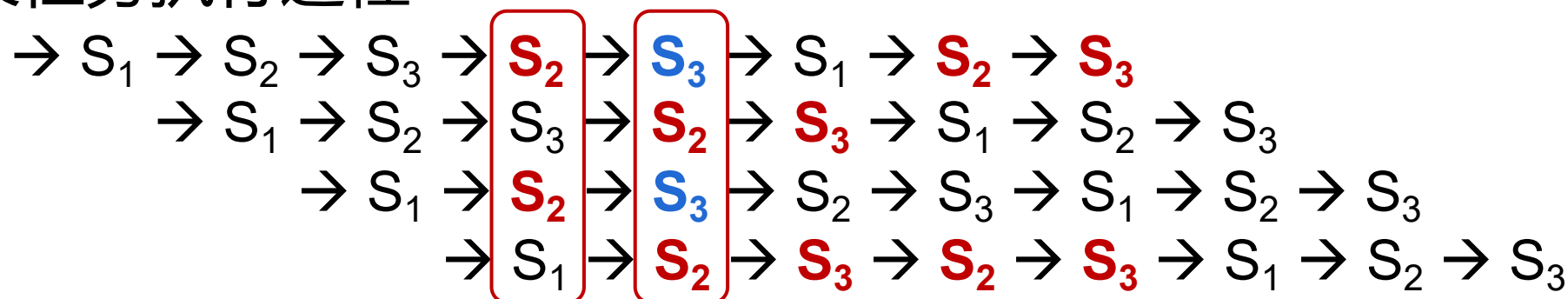


|       | 1 | 2 | 3 | 4 | 5 |
|-------|---|---|---|---|---|
| $S_1$ | × |   |   |   |   |
| $S_2$ |   | × |   | × |   |
| $S_3$ |   |   | × |   | × |

### 非线性流水线的调度问题

- 确定什么时候向流水线引进新的任务，才能使该任务不会与先前进入流水线的任务发生冲突——争用流水段

例如：某任务执行过程



## 3.1.2 流水线分类

第 29 页

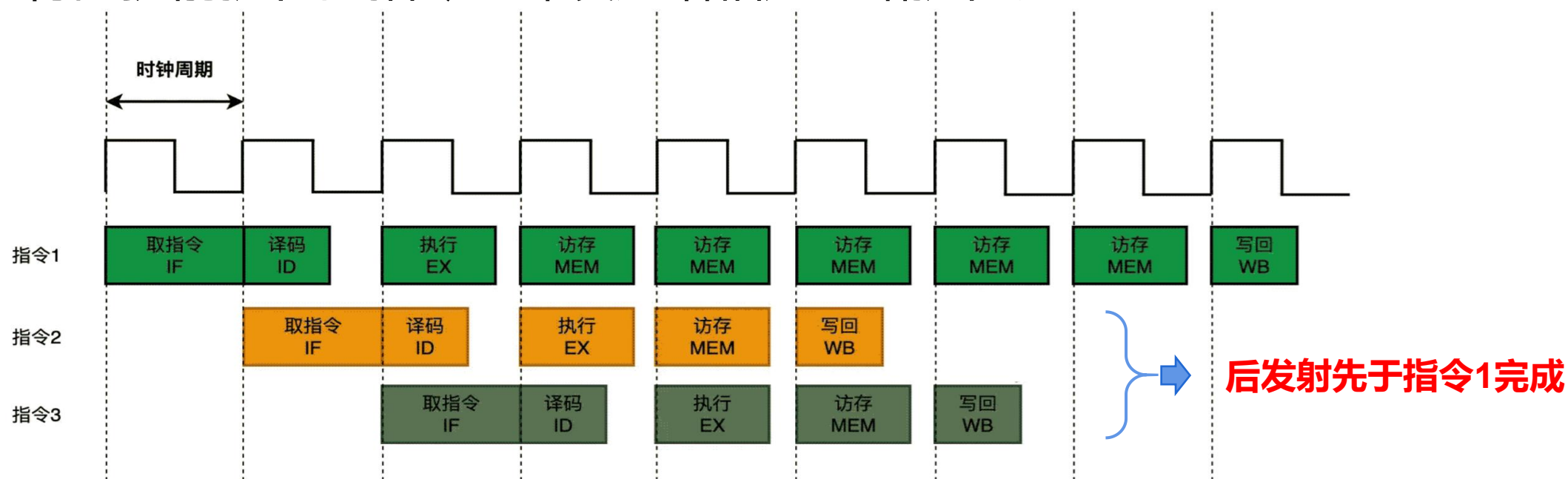
### ■ 按照任务流入和流出的顺序是否相同进行分类

- **顺序流水线：**流水线输出端任务流出的顺序与输入端任务流入的顺序完全相同。  
每一个任务在流水线的各段中是一个跟着一个顺序流动的
- **乱序流水线：**流水线输出端任务流出的顺序与输入端任务流入的顺序可以不同，  
允许后进入流水线的任务先完成（从输出端流出）  
(无序流水线、错序流水线、异步流水线)

## 3.1.2 流水线分类

第 30 页

- 乱序执行（out-of-order execution）指处理器允许将多条指令不按程序规定的顺序分开发送给各相应电路单元处理。
- 例如，处理器根据各单元电路的空闲状态和各指令能否提前执行的具体情况分析后，将能提前执行的指令立即发送给相应电路执行。



## 3.1.2 流水线分类

第 31 页

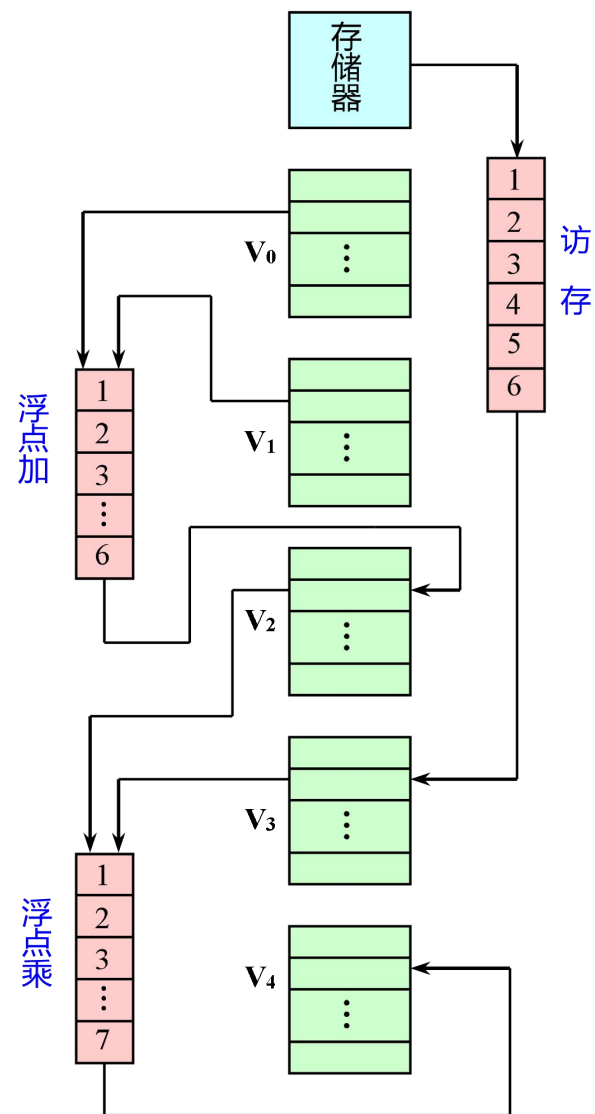
### ■ 按照处理数据对象来进行分类

- **标量流水线：**仅对标量数据进行流水处理，不具有向量数据表示和向量操作



- **向量流水线：**对向量数据和向量操作进行流水处理

(第四章 向量处理机)



## 3.2 流水线的性能指标



## 3.2 流水线性能指标

---

第 33 页

- 吞吐率
- 加速比
- 效 率

## 3.2.1 吞吐率

第 34 页

**吞吐率：**在单位时间内流水线所完成的任务数量或输出结果的数量。

$$TP = \frac{n}{T}$$

$n$ ：任务数

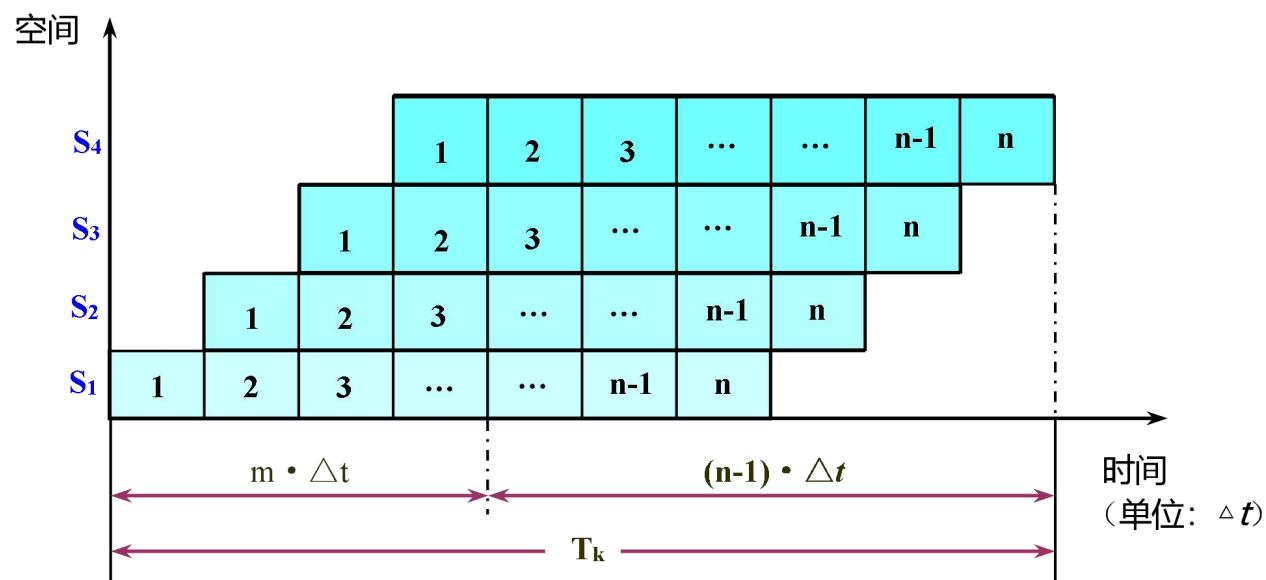
$T$ ：处理完成 $n$ 个任务所用的时间

- 各段时间均相等的流水线
- 各段时间不完全相等的流水线

## 3.2.1 吞吐率

第 35 页

### 1、各段时间均相等的流水线



- 流水线完成 $n$ 个连续任务所需要的总时间为：（假设一条 $m$ 段线性流水线）

$$T = m\Delta t + (n - 1)\Delta t = (m + n - 1)\Delta t$$

## 3.2.1 吞吐率

第 36 页

### ■ 流水线的实际吞吐率

$$TP = \frac{n}{(m + n - 1)\Delta t}$$

### ■ 最大吞吐率

$$TP = \lim_{n \rightarrow \infty} \frac{n}{(m + n - 1)\Delta t} = \frac{1}{\Delta t_{\max}}$$

### ■ 最大吞吐率与实际吞吐率的关系

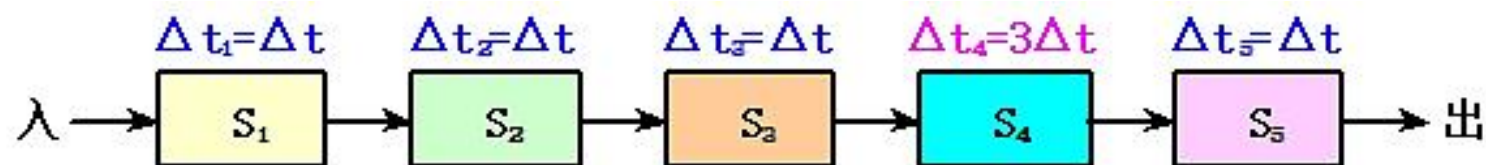
$$TP = \frac{n}{m + n - 1} TP_{\max}$$

只有当  $n \gg m$  时,  
才有  $TP \approx TP_{\max}$

## 3.2.1 吞吐量

第 37 页

### 2、各段时间不完全相等的流水线



各段时间不等的流水线时空图

#### ■例：一条5段的流水线

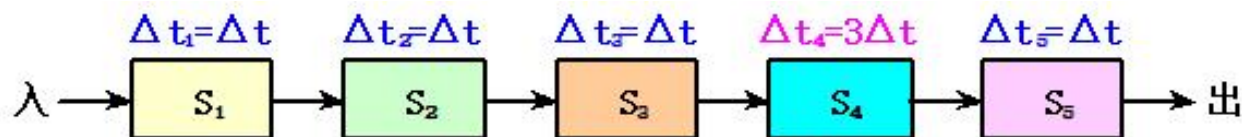
$S_1, S_2, S_3, S_5$ 各段的时间： $\Delta t$ ,  $S_4$ 的时间： $3\Delta t$ （瓶颈段）

流水线中这种时间最长的段称为流水线的**瓶颈段**

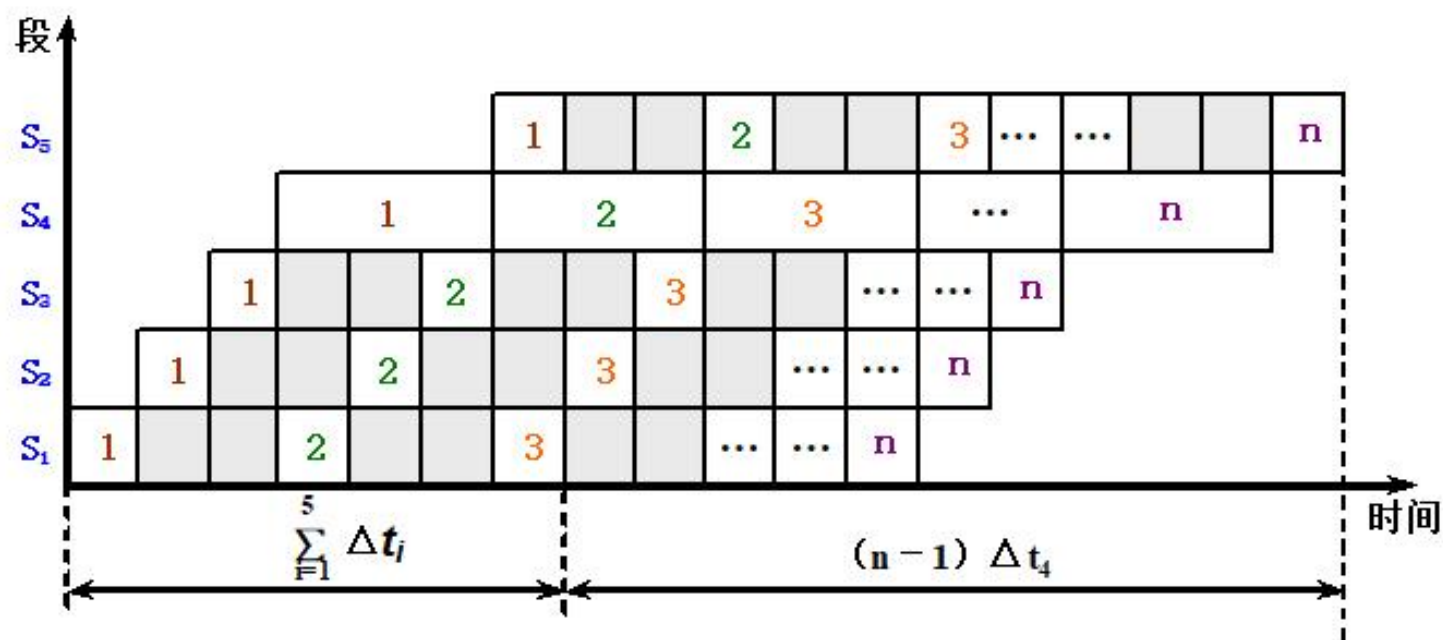
## 3.2.1 吞吐率

第 38 页

### 一条5段的流水线



(a) 流水线



(b) 时空图

## 3.2.1 吞吐率

第 39 页

- 各段时间不等的流水线的实际吞吐率为：（ $\Delta t_i$ 为第*i*段的时间，共有*m*个段）

$$TP = \frac{n}{\sum_{i=1}^m \Delta t_i + (n - 1) \max(\Delta t_1, \Delta t_2, \dots, \Delta t_m)}$$

- 流水线的最大吞吐率为：

$$TP = \frac{1}{\max(\Delta t_1, \Delta t_2, \dots, \Delta t_m)_{\max}}$$


$$TP = \frac{1}{3\Delta t_{\max}}$$

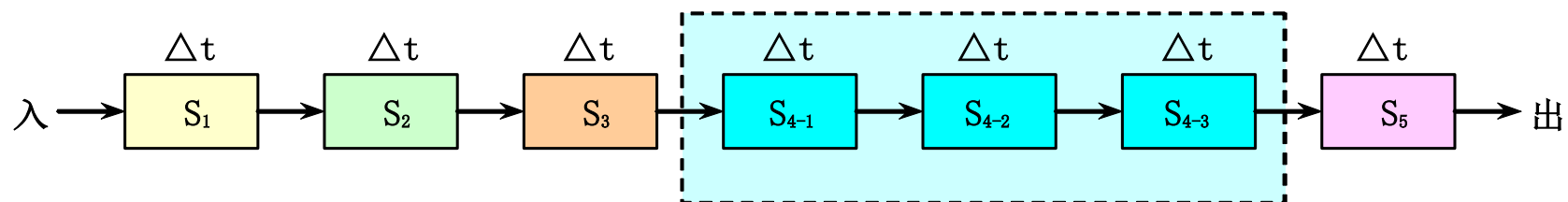
## 3.2.1 吞吐量

第 40 页

### ■解决流水线瓶颈问题的常用方法

#### ◆ 方法一：细分瓶颈段

例如：对前面的5段流水线把瓶颈段 $S_4$ 细分为3个子流水线段： $S_{4-1}$ ， $S_{4-2}$ ， $S_{4-3}$



改进后的流水线的吞吐量： $TP_{\max} = \frac{1}{\Delta t}$



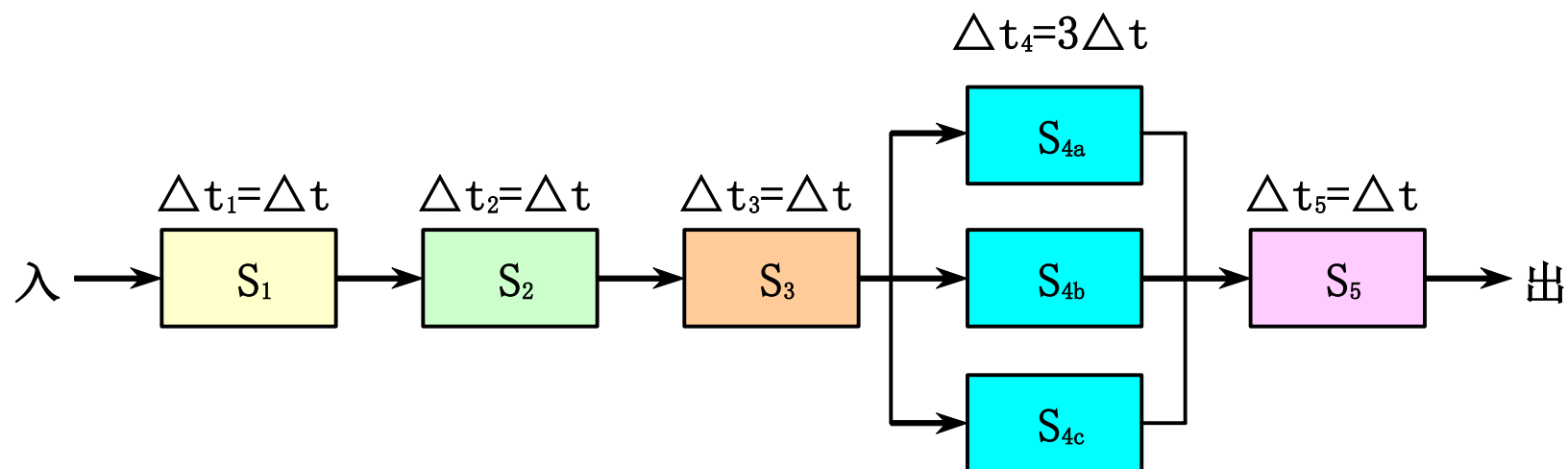
## 3.2.1 吞吐量

第 41 页

### ■解决流水线瓶颈问题的常用方法

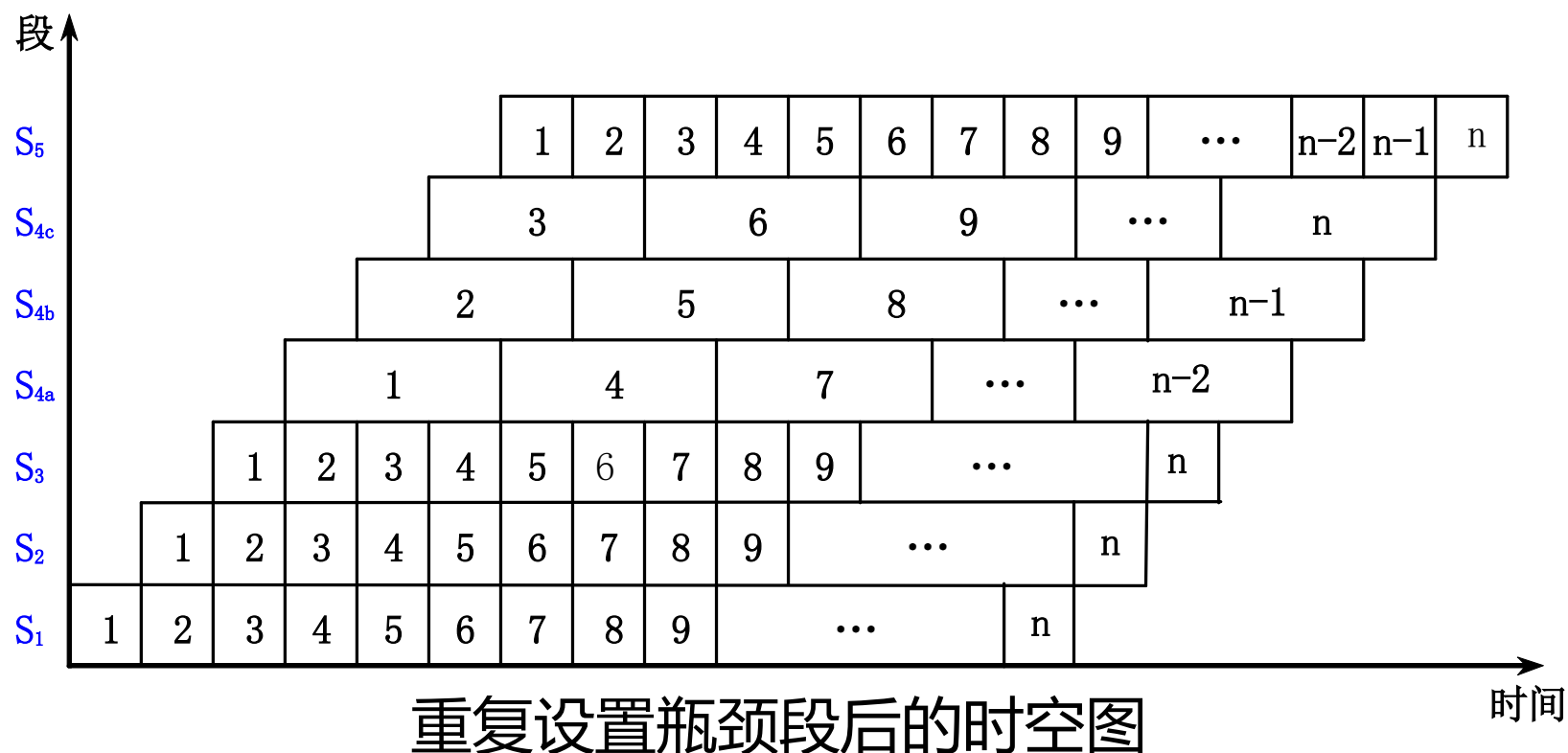
#### ◆ 方法二：重复设置瓶颈段

- 缺点：控制逻辑比较复杂，所需的硬件增加了。
- 例如：对前面的5段流水线重复设置瓶颈段 $S_4$ ： $S_{4a}$ ， $S_{4b}$ ， $S_{4c}$



## ■解决流水线瓶颈问题的常用方法

## ◆ 方法二：重复设置瓶颈段



## 3.2.2 加速比

第 43 页

■加速比：完成同样一批任务，不使用流水线所用的时间与使用流水线所用的时间之比。

◆ 假设：不使用流水线（即顺序执行）所用的时间为 $T_s$ ，使用流水线后所用的时间为 $T_m$ ，则该流水线的加速比为：

$$S = \frac{T_s}{T_m}$$

■流水线各段时间相等（都是 $\Delta t$ ）

■流水线的各段时间不完全相等

## 3.2.2 加速比

第 44 页

### ■ 流水线各段时间相等 (都是 $\Delta t$ )

◆ 一条 $m$ 段流水线完成 $n$ 个连续任务所需要的时间为:

$$T_m = (m + n - 1)\Delta t$$

◆ 顺序执行 $n$ 个任务所需要的时间:

$$T_s = nm\Delta t$$

◆ 流水线的实际加速比为:

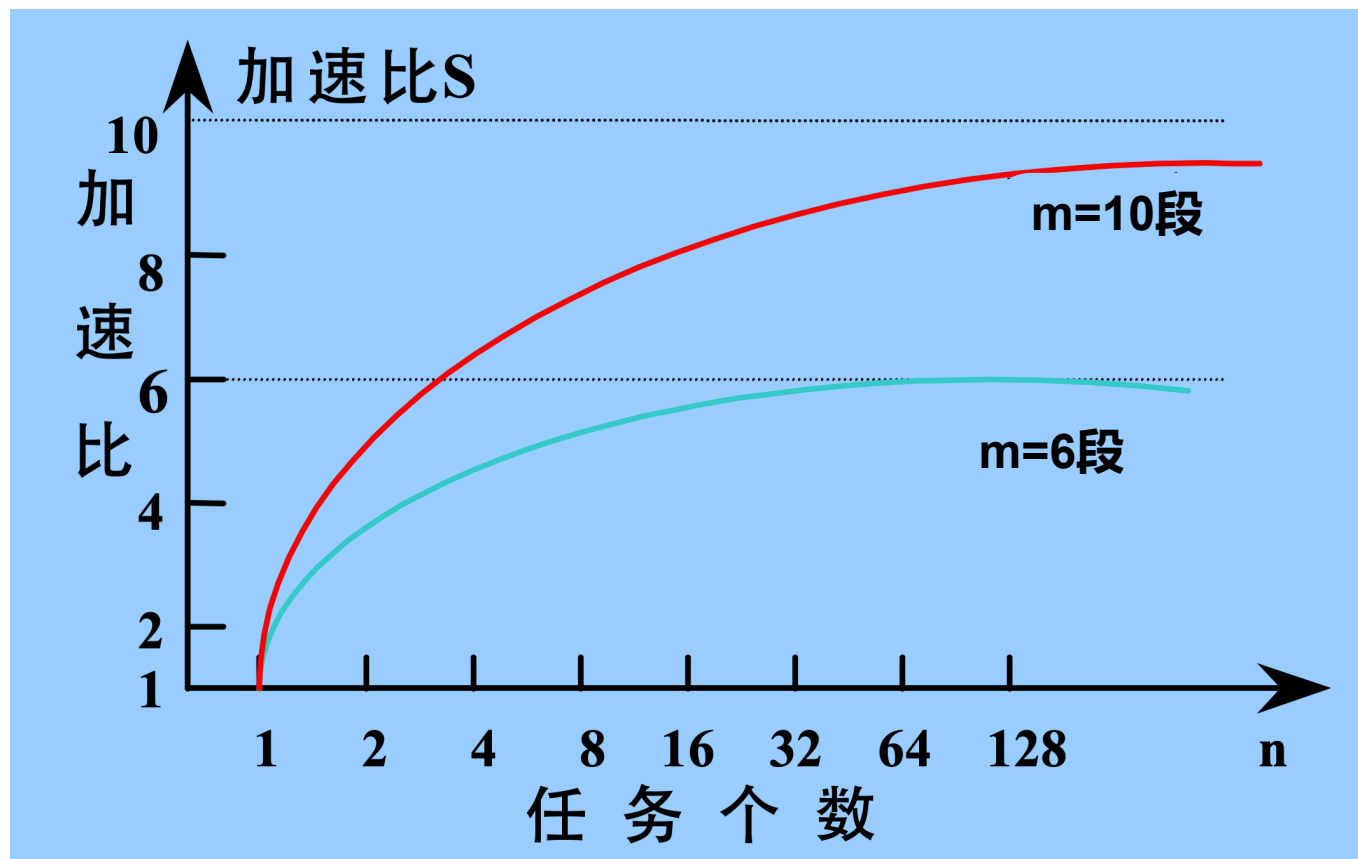
$$S = \frac{nm}{m + n - 1} \quad \longrightarrow \quad S = \lim_{n \rightarrow \infty} \frac{nm}{m + n - 1} = S_{\max}$$

当 $n \gg m$ 时,  $S \approx m$

思考: 流水线的段数愈多愈好?

## 3.2.2 加速比

第 45 页



规律：当流水线段数增加时，需要连续输入的任务数也必须增加

思考：流水线最佳段数是否越大越好？

## 3.2.2 加速比

第 46 页

### ■ 流水线各段时间不完全相等

◆ 一条m段流水线完成n个连续任务的实际加速比为：

$$S = \frac{n \sum_{i=1}^m \Delta t_i}{\sum_{i=1}^m \Delta t_i + (n - 1) \max(\Delta t_1, \Delta t_2, \dots, \Delta t_m)}$$

实现加速比的前提是流水线最大程度不断流。如何表示？

### 3.2.3 效率

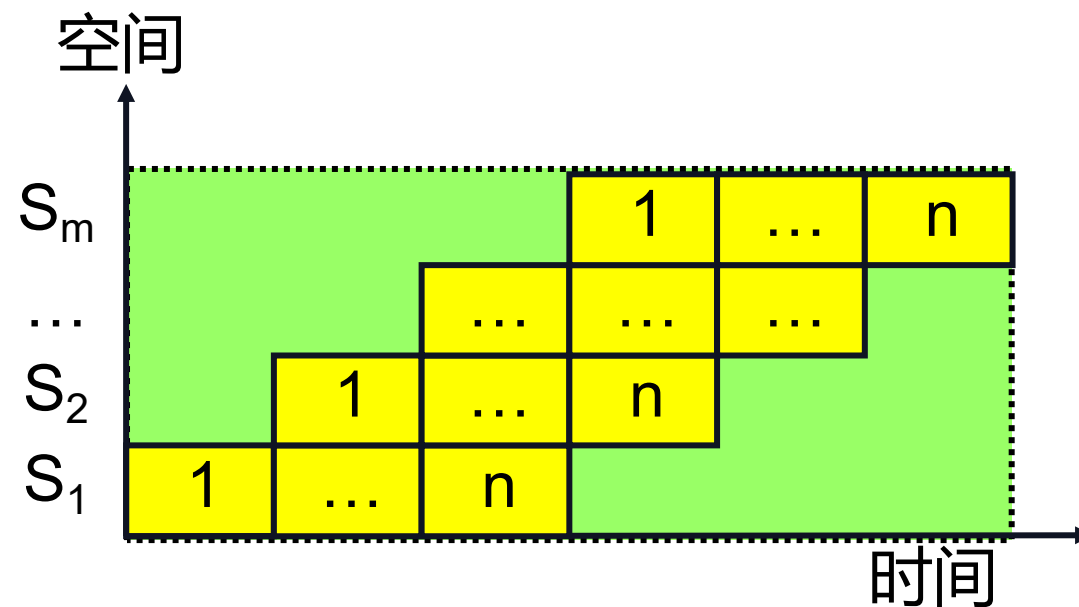
第 47 页

#### ■ 定义

是指流水线的设备利用率。在时空图上，流水线的效率定义为n个任务占用的时空区与m个功能段总的时空区之比。

#### ■ 公式

$$E = \frac{n \text{ 个任务占用的时空区}}{m \text{ 个功能段总的时空区}}$$



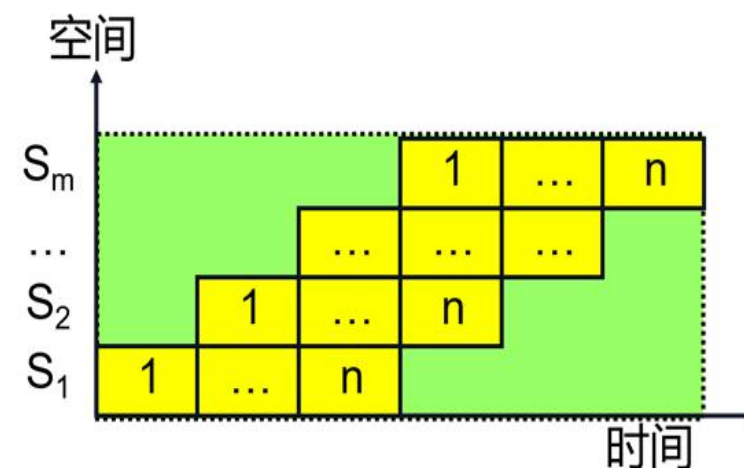
### 3.2.3 效率

第 48 页

假设在流水线各段的执行时间不相等，输入到流水线中的任务是连续的理想情况下。

#### ■ 实际效率（功能段权值相等）

$$E = \frac{n \cdot \sum_{i=1}^m \Delta t_i}{m \cdot \left[ \sum_{i=1}^m \Delta t_i + (n-1) \cdot \max(\Delta t_1, \Delta t_2, \dots, \Delta t_m) \right]}$$





### 3.2.3 效率

第 49 页

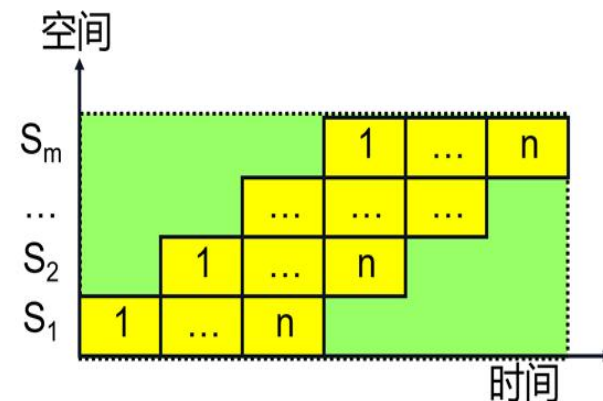
#### ■ 实际效率（功能段权值不同）

$$E = \frac{n \text{个任务占用的时空区}}{m \text{个功能段的总的时空区}}$$

$$n \cdot \left[ \sum_{i=1}^m \alpha_i \cdot \Delta t_i \right]$$

当各段时间不相等时：

$$= \frac{\sum_{i=1}^m \alpha_i \cdot \left[ \sum_{i=1}^m \Delta t_i + (n-1) \cdot \max(\Delta t_1, \Delta t_2, \dots, \Delta t_m) \right]}{n \cdot \left[ \sum_{i=1}^m \alpha_i \cdot \Delta t_i \right]}$$



其中， $\alpha_i$ 为第*i*段的权值，第*i*段的执行时间为 $\alpha_i \cdot \Delta t_i$ ， $\sum_{i=1}^m \alpha_i = M$

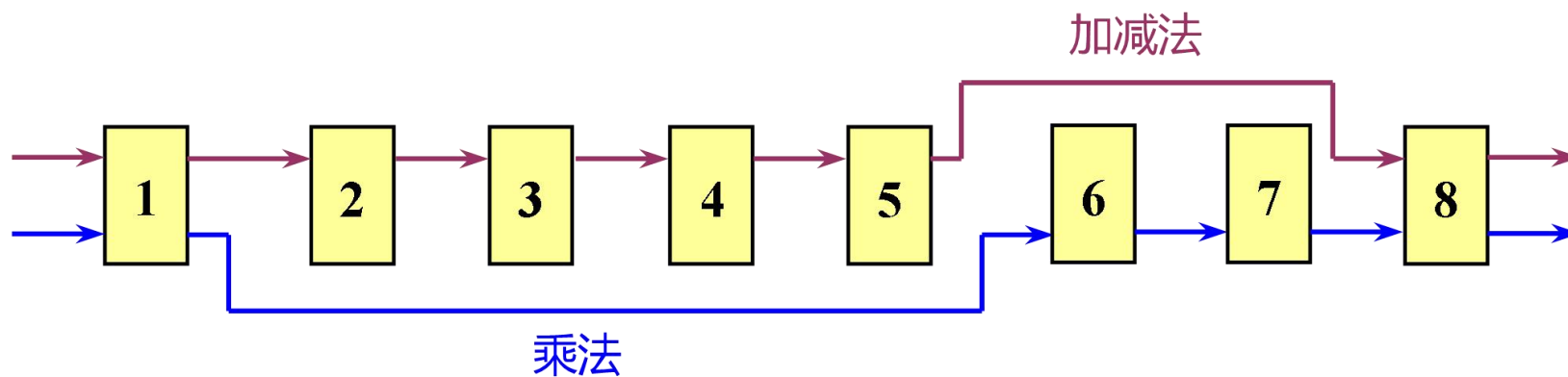
## 3.2.4 流水线的性能分析举例

第 50 页

例3.1 设在下图所示的静态流水线上计算：

$$\prod_{i=1}^4 (A_i + B_i)$$

流水线的输出可以直接返回输入端或暂存于相应的流水寄存器中，试计算其吞吐率、加速比和效率。



(每段的时间都为  $\Delta t$ )

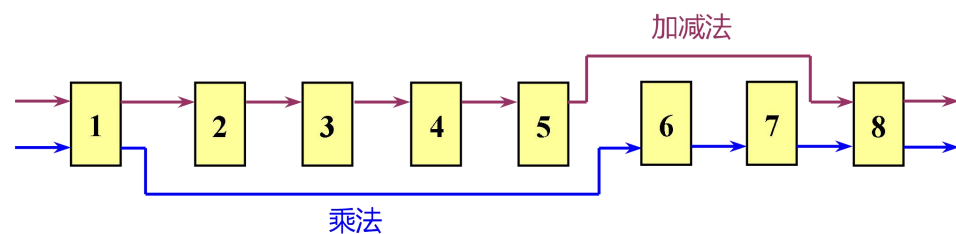
## 3.2.4 流水线的性能分析举例

第 51 页

解：（1）选择适合于流水线工作的算法

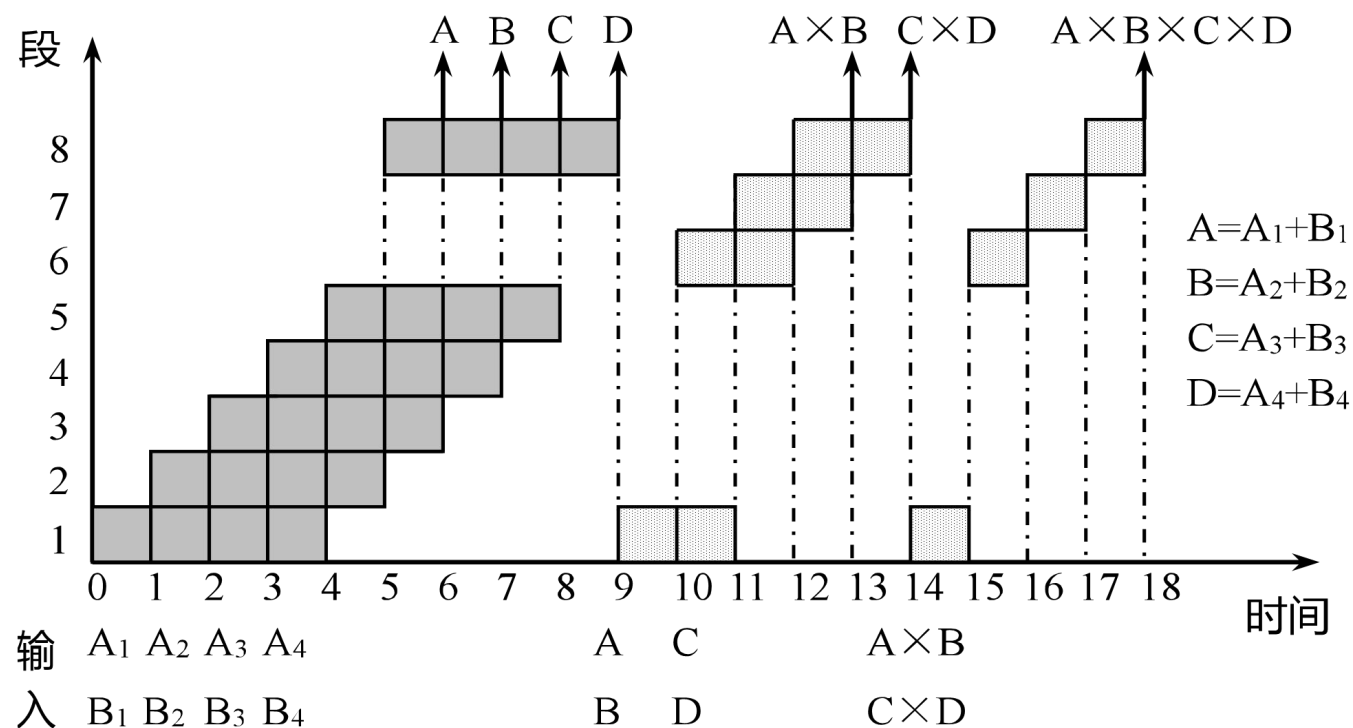
- 先计算 $A_1+B_1$ 、 $A_2+B_2$ 、 $A_3+B_3$ 和 $A_4+B_4$ ;
- 再计算 $(A_1+B_1) \times (A_2+B_2)$ 和 $(A_3+B_3) \times (A_4+B_4)$ ;
- 然后求总的乘积结果。

（2）画出时空图



（1）静态流水线在进行功能切换时，要等前一种运算全部流出流水线后才能进行后面的运算；

（2）运算之间存在关联。



## 3.2.4 流水线的性能分析举例

第 52 页

### (3) 计算性能

■ 在18个 $\Delta t$ 时间中，给出了7个结果。吞吐率为： $TP = \frac{7}{18\Delta t}$

■ 不用流水线，由于一次求和需 $6\Delta t$ ，一次求积需 $4\Delta t$ ，则产生上述7个结果共需 $(4 \times 6 + 3 \times 4) \Delta t = 36\Delta t$

加速比为：

$$S = \frac{36\Delta t}{18\Delta t} = 2$$

■ 流水线的效率

$$E = \frac{4 \times 6 + 3 \times 4}{8 \times 18} = 0.25$$

该流水线效率低的根本原因：

- (1) 多功能流水线在做某一种运算时，总有一些段是空闲的；
- (2) 流水线的工作过程有建立与排空部分。

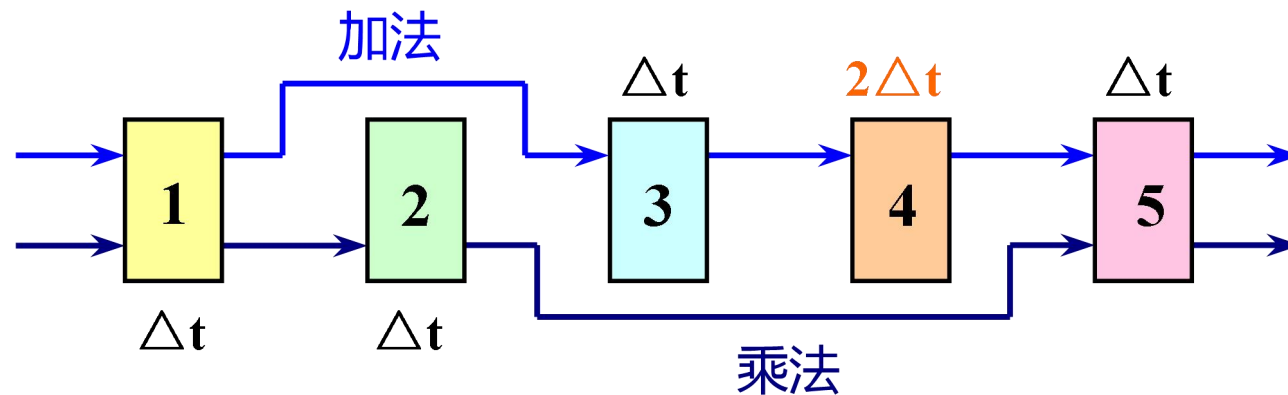
### 3.2.4 流水线的性能分析举例

第 53 页

例3.2 有一条动态多功能流水线由5段组成，加法用1、3、4、5段，乘法用1、2、5段，第4段的时间为 $2\Delta t$ ，其余各段时间均为 $\Delta t$ ，而且流水线的输出可以直接返回输入端或暂存于相应的流水寄存器中。若在该流水线上计算：

$$\sum_{i=1}^4 (A_i \times B_i)$$

试计算其吞吐率、加速比和效率。

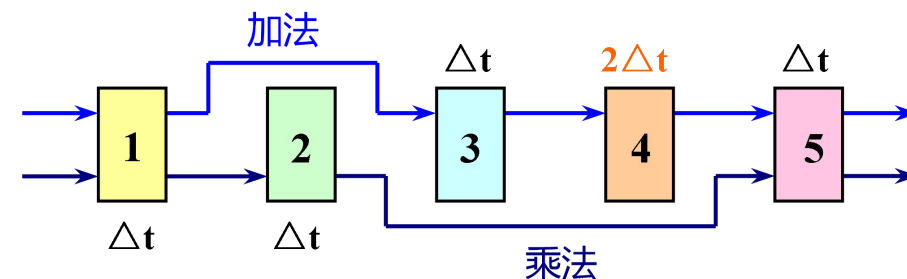


## 3.2.4 流水线的性能分析举例

第 54 页

解: (1) 选择适合于流水线工作的算法

- 应先计算  $A_1 \times B_1$ 、 $A_2 \times B_2$ 、 $A_3 \times B_3$  和  $A_4 \times B_4$ ;
- 再计算  $(A_1 \times B_1) + (A_2 \times B_2)$ 、 $(A_3 \times B_3) + (A_4 \times B_4)$ ;
- 然后求总的累加结果。



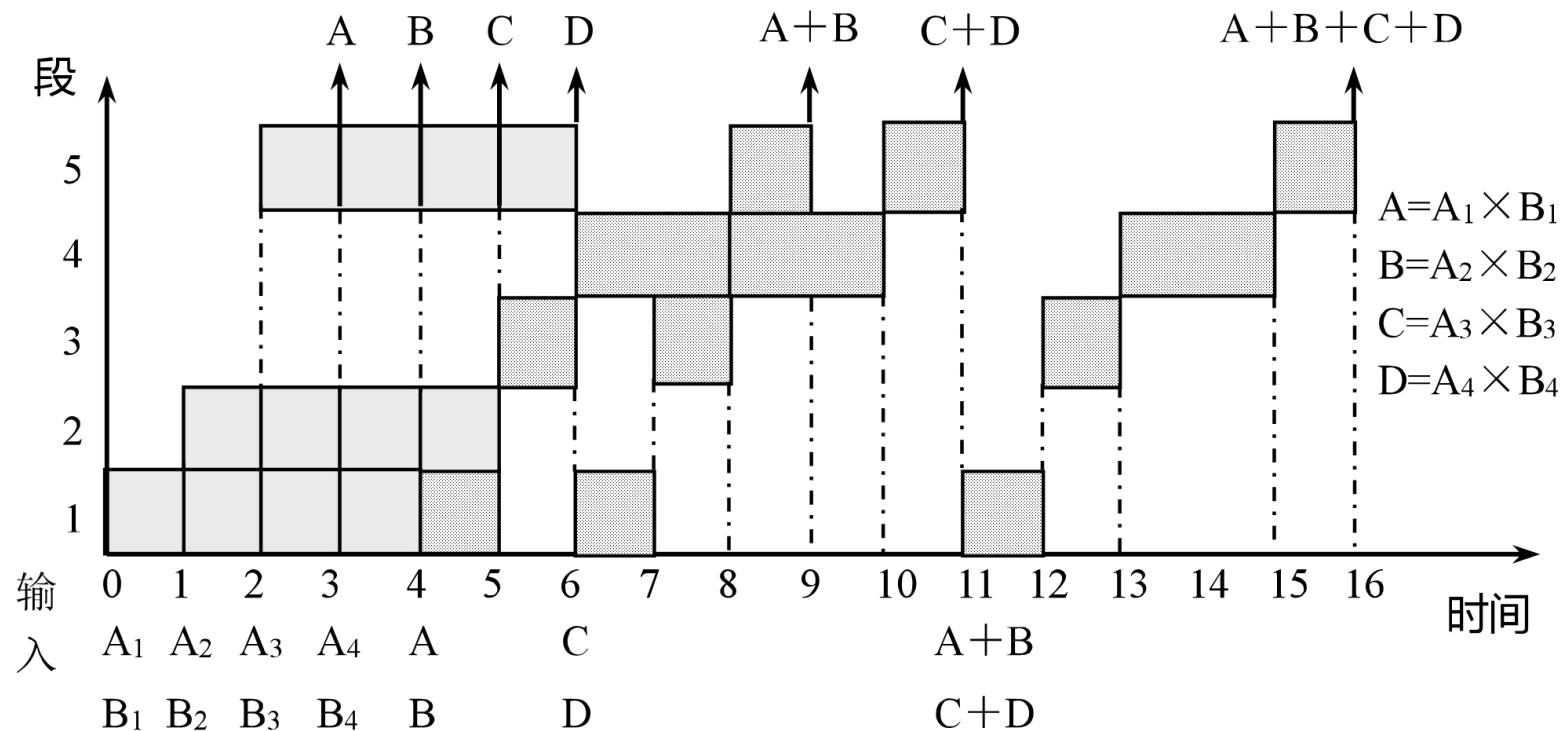
(2) 画出时空图

(3) 计算性能

$$TP = \frac{7}{16\Delta t}$$

$$S = \frac{27\Delta t}{16\Delta t} \approx 1.69$$

$$E = \frac{4 \times 3 + 3 \times 5}{5 \times 16} \approx 0.338$$

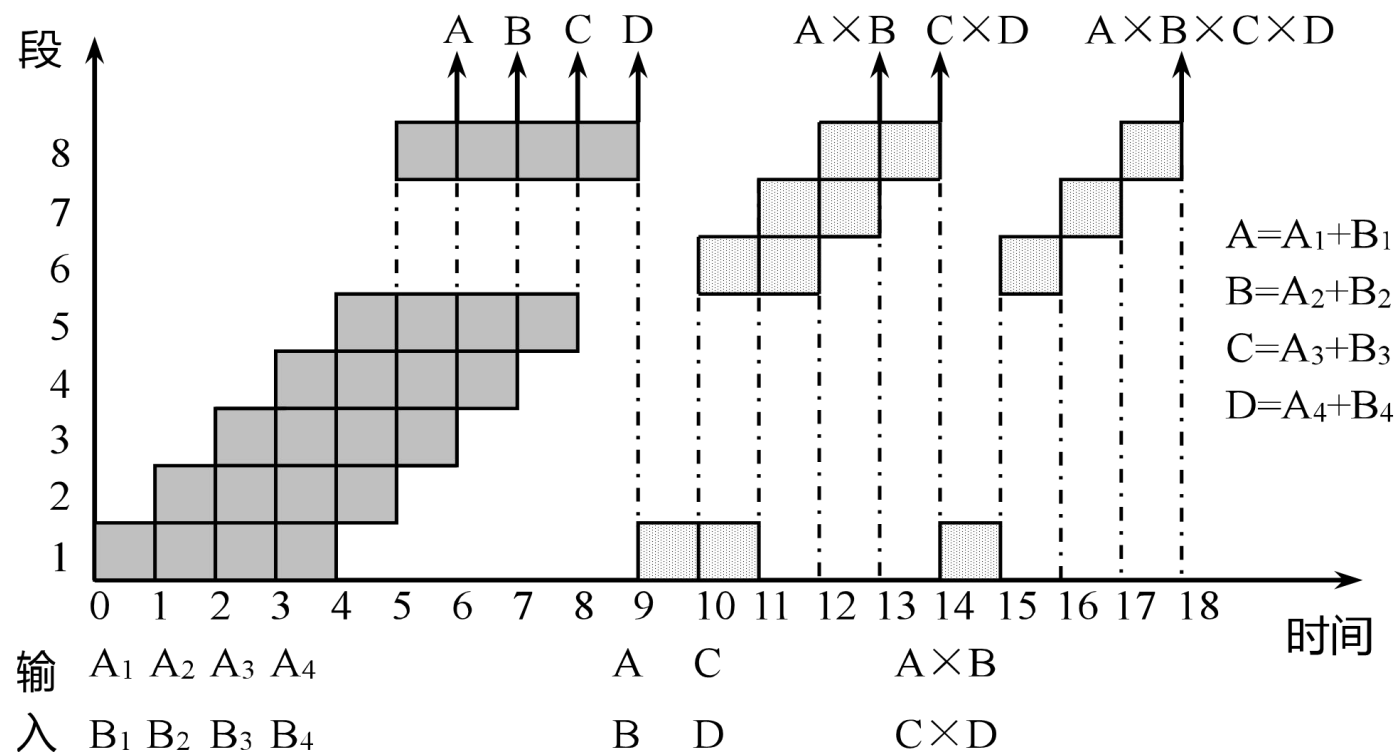


## 3.2.4 流水线的性能分析举例

第 55 页

### 存在问题：流水线效率低

- 多功能流水线在做某一种运算时，总有一些段是空闲的；
- 静态流水线在进行功能切换时，要等前一种运算全部流出流水线后才能进行后面的运算；
- 运算之间存在关联，后面有些运算要用到前面运算的结果；
- 流水线的工作过程有建立与排空部分。



如何提升非线性流水线效率？ ← 调度

## 3.3 非线性流水线的调度

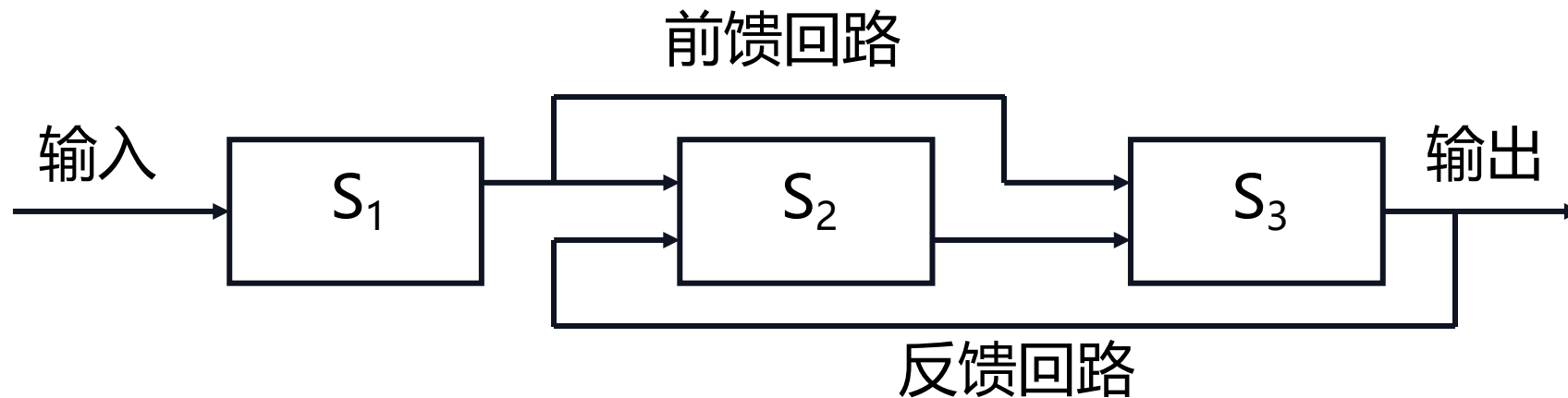


### 3.3 非线性流水线调度

第 57 页

- 因资源冲突而导致流水线断流。

例: 指重叠指令或流水线中的指令同时要用同一个功能部件, 实际上是一种冲突。非线性流水线的调度就是为了尽量避免资源使用上的冲突。



- 流水线调度要解决的问题: 应按什么样的时间间隔向流水线输入新任务, 既不发生功能段使用冲突, 又能使流水线有较高的吞吐率和效率?

## 3.3 非线性流水线调度

第 58 页

启动距离为 3 循环时的流水线预约表

| 时间<br>功能段      | 1              | 2              | 3              | 4                             | 5                             | 6              | 7                                            | 8                             | 9              | 10                                           | 11                            | ... |
|----------------|----------------|----------------|----------------|-------------------------------|-------------------------------|----------------|----------------------------------------------|-------------------------------|----------------|----------------------------------------------|-------------------------------|-----|
| S <sub>1</sub> | X <sub>1</sub> | 禁止启动距离<br>=3   |                | X <sub>1</sub> X <sub>2</sub> |                               |                | X <sub>1</sub> X <sub>2</sub> X <sub>3</sub> |                               |                | X <sub>2</sub> X <sub>3</sub> X <sub>4</sub> |                               | ... |
| S <sub>2</sub> |                | X <sub>1</sub> |                |                               | X <sub>1</sub> X <sub>2</sub> |                |                                              | X <sub>2</sub> X <sub>3</sub> |                |                                              | X <sub>3</sub> X <sub>4</sub> | ... |
| S <sub>3</sub> |                | X <sub>1</sub> |                |                               | X <sub>2</sub>                | X <sub>1</sub> |                                              | X <sub>3</sub>                | X <sub>2</sub> |                                              | X <sub>4</sub>                | ... |
| S <sub>4</sub> |                |                | X <sub>1</sub> |                               |                               | X <sub>2</sub> |                                              |                               | X <sub>3</sub> |                                              |                               | ... |

- **预约表**：横向（向右）是时间（一般用时钟周期表示），纵向（向下）是流水线的段。如果在第 $n$ 个时钟周期使用第 $k$ 段，则在第 $k$ 行和第 $n$ 列的交叉处的格子里有一个 $X_i$  ( $i \in [1, n]$ , 第 $i$ 条指令)。
- **禁止启动距离**：引起非线性流水线冲突的启动距离称为禁止启动距离。

## 3.3 非线性流水线调度

第 59 页

启动距离为(1,7)循环时的流水线预约表

| 时间<br>功能段 | 1     | 2     | 3     | 4     | 5     | 6     | 7     | 8     | 9     | 10    | 11    | 12    | 13    | 14    | 15    | 16    | ... |
|-----------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-----|
| $S_1$     | $X_1$ | $X_2$ |       | $X_1$ | $X_2$ |       | $X_1$ | $X_2$ | $X_3$ | $X_4$ |       | $X_3$ | $X_4$ |       | $X_3$ | $X_4$ | ... |
| $S_2$     |       | $X_1$ | $X_2$ |       | $X_1$ | $X_2$ |       |       |       | $X_3$ | $X_4$ |       | $X_3$ | $X_4$ |       |       | ... |
| $S_3$     |       | $X_1$ | $X_2$ |       |       | $X_1$ | $X_2$ |       |       | $X_3$ | $X_4$ |       |       | $X_3$ | $X_4$ |       | ... |
| $S_4$     |       |       | $X_1$ | $X_2$ |       |       |       |       |       |       | $X_3$ | $X_4$ |       |       |       |       | ... |

← 启动周期 →      ← 重复启动周期 →

■ **启动距离**：向一条非线性流水线的输入端顺序输入两个任务之间的时间间隔称为启动距离/等待时间      例如 (1) <- 是否能够循环执行

■ **启动循环**：不发生冲突的启动距离一般是一个循环数列，称为非线性流水线的启动循环，记作 (1, 7)。

### 3.3 非线性流水线调度

第 60 页

启动距离为 5 时的流水线不冲突

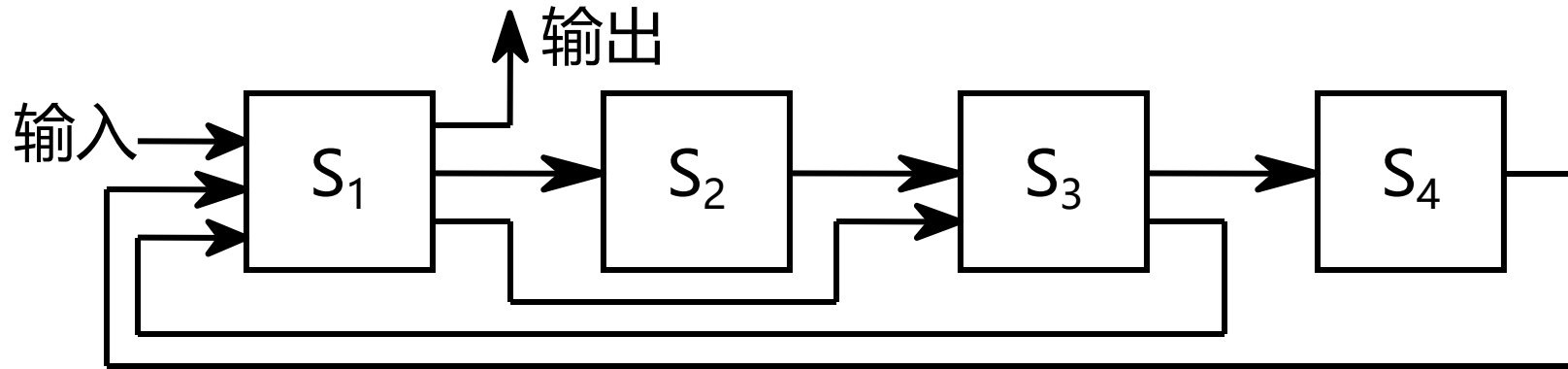
| 时间<br>功能段 | 1     | 2     | 3     | 4     | 5     | 6     | 7     | 8     | 9     | 10    | 11    | ... |
|-----------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-----|
| $S_1$     | $X_1$ |       |       | $X_1$ |       | $X_2$ | $X_1$ |       | $X_2$ |       | $X_3$ | ... |
| $S_2$     |       | $X_1$ |       |       | $X_1$ |       | $X_2$ |       |       | $X_2$ |       | ... |
| $S_3$     |       | $X_1$ |       |       |       | $X_1$ | $X_2$ |       |       |       | $X_2$ | ... |
| $S_4$     |       |       | $X_1$ |       |       |       |       | $X_2$ |       |       |       | ... |

← 启动周期 →      ← 重复启动周期 →

- **恒定循环：**启动距离5也可以认为是一个循环数列，称为非线性流水线的恒定循环，记作 (5) 。

### 3.3 非线性流水线调度

第 61 页

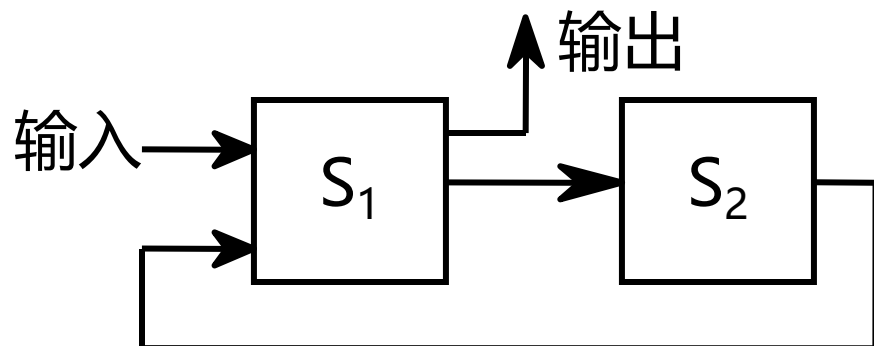


| 时间<br>流水段 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|-----------|---|---|---|---|---|---|---|
| $S_1$     | × |   |   | × |   |   | × |
| $S_2$     |   | × |   |   | × |   |   |
| $S_3$     |   | × |   |   |   | × |   |
| $S_4$     |   |   | × |   |   |   |   |

- 流水线调度目的：找出一个最小启动循环，按照这周期向流水线输入新任务，流水线的各个功能段都不会发生冲突，而且流水线的吞吐率和效率最高。

## 3.3 非线性流水线调度

第 62 页



预约表

| 流水段 \ 时间 | 1 | 2 | 3 |
|----------|---|---|---|
| $S_1$    | × |   | × |
| $S_2$    |   | × |   |

产生冲突

功能 $X_1$ 的执行情况:

| 流水段 \ 时间 | 1     | 2     | 3     | 4 | 5 | 6 | 7 |
|----------|-------|-------|-------|---|---|---|---|
| $S_1$    | $X_1$ |       | $X_1$ |   |   |   |   |
| $S_2$    |       | $X_1$ |       |   |   |   |   |

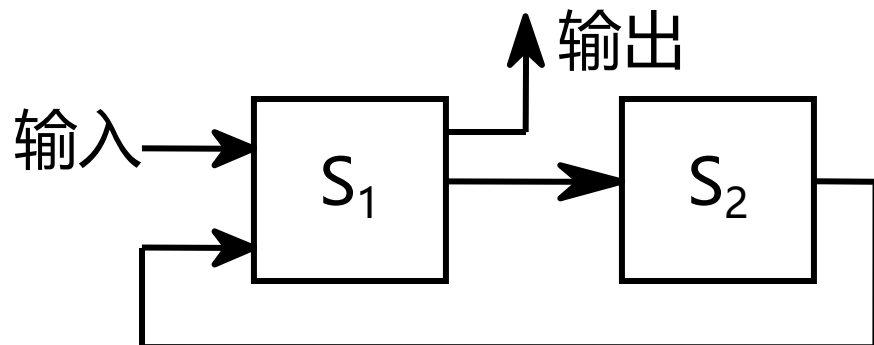
特征:  $(010)_2$  代表  $3-1=2$

禁止向量

记录产生冲突的距离

# 3.3 非线性流水线调度

第 63 页



预约表

| 流水段 \ 时间 | 1 | 2 | 3 |
|----------|---|---|---|
| $S_1$    | × |   | × |
| $S_2$    |   | × |   |

功能 $X_2$ 的执行情况（启动距离1）：

| 流水段 \ 时间 | 1     | 2     | 3     | 4     | 5 | 6 | 7 |
|----------|-------|-------|-------|-------|---|---|---|
| $S_1$    | $X_1$ | $X_2$ | $X_1$ | $X_2$ |   |   |   |
| $S_2$    |       | $X_1$ | $X_2$ |       |   |   |   |

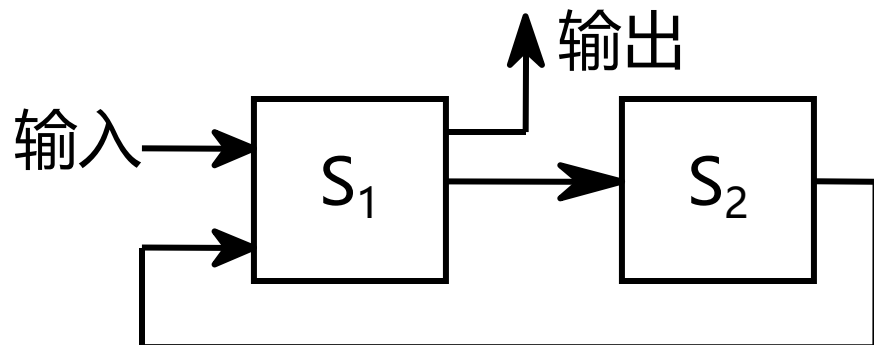
特征：(011) 代表第1、2位

禁止向量

(010)V(001)=(011)

## 3.3 非线性流水线调度

第 64 页



预约表

| 流水段 \ 时间 | 1 | 2 | 3 |
|----------|---|---|---|
| $S_1$    | × |   | × |
| $S_2$    |   | × |   |

功能X3的执行情况（启动距离3）：

| 流水段 \ 时间 | 1     | 2     | 3     | 4     | 5     | 6     | 7     |
|----------|-------|-------|-------|-------|-------|-------|-------|
| $S_1$    | $X_1$ | $X_2$ | $X_1$ | $X_2$ | $X_3$ |       | $X_3$ |
| $S_2$    |       | $X_1$ | $X_2$ |       |       | $X_3$ |       |

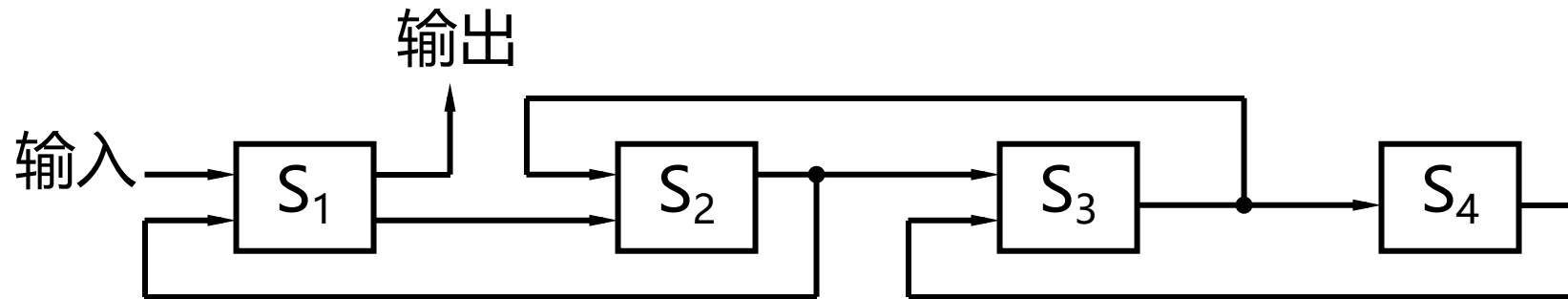
$(010)V(000)=(010)$  特征：(010)  
禁止向量



## 3.3 非线性流水线调度

第 65 页

例3.3 一个4功能段非线性流水线预约表



| 时间<br>流水段 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|-----------|---|---|---|---|---|---|---|
| $S_1$     | × |   |   |   |   |   | × |
| $S_2$     |   | × |   |   |   | × |   |
| $S_3$     |   |   | × |   | × |   |   |
| $S_4$     |   |   |   | × |   |   |   |

求最小启动循环。

## 3.3 非线性流水线调度

第 66 页

- 步骤一：由预约表得到禁止表F
- 步骤二：由禁止表F得到冲突向量C
- 步骤三：由冲突向量构造调度流水线的状态转换图
- 步骤四：在状态图中找出可用启动距离，并计算平均启动距离
- 步骤五：找出平均启动距离最小的启动循环或恒定循环

## 3.3 非线性流水线调度

第 67 页

步骤一：由预约表得到禁止表F  禁止集合

| 时间<br>流水段      | 1 | 2 | 3 | 4 | 5 | 6 | 7 |                                                                                       |
|----------------|---|---|---|---|---|---|---|---------------------------------------------------------------------------------------|
| S <sub>1</sub> | × |   |   |   |   |   | × |  6 |
| S <sub>2</sub> |   | × |   |   |   | × |   |  4 |
| S <sub>3</sub> |   |   | × |   | × |   |   |  2 |
| S <sub>4</sub> |   |   |   | × |   |   |   |                                                                                       |

**定义：**将一条非线性流水线的各功能段禁止启动距离组合在一起形成的数列。

**计算：**将预约表中的每一行中任意两个“×”之间的距离都计算出来，去掉重复的，由这些数形成禁止表。

**例子：**非线性流水线的禁止表F= (2, 4, 6) ← 一个由禁止启动距离构成的集合

## 3.3 非线性流水线调度

第 68 页

步骤二：由禁止表F得到冲突向量C

| 时间<br>流水段 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |     |
|-----------|---|---|---|---|---|---|---|-----|
| $S_1$     | × |   |   |   |   |   | × | ← 6 |
| $S_2$     |   | × |   |   |   | × |   | ← 4 |
| $S_3$     |   |   | × |   | × |   |   | ← 2 |
| $S_4$     |   |   |   | × |   |   |   |     |

**概念：**冲突向量用一个m位的二进制数表示（其中m是禁止向量中的最大值），一般格式为 $C = (C_m C_{m-1} \dots C_i \dots C_2 C_1)$ 。若i在冲突向量中，则 $C_i = 1$ ；否则 $C_i = 0$ ，其中 $C_m$ 一定为1。

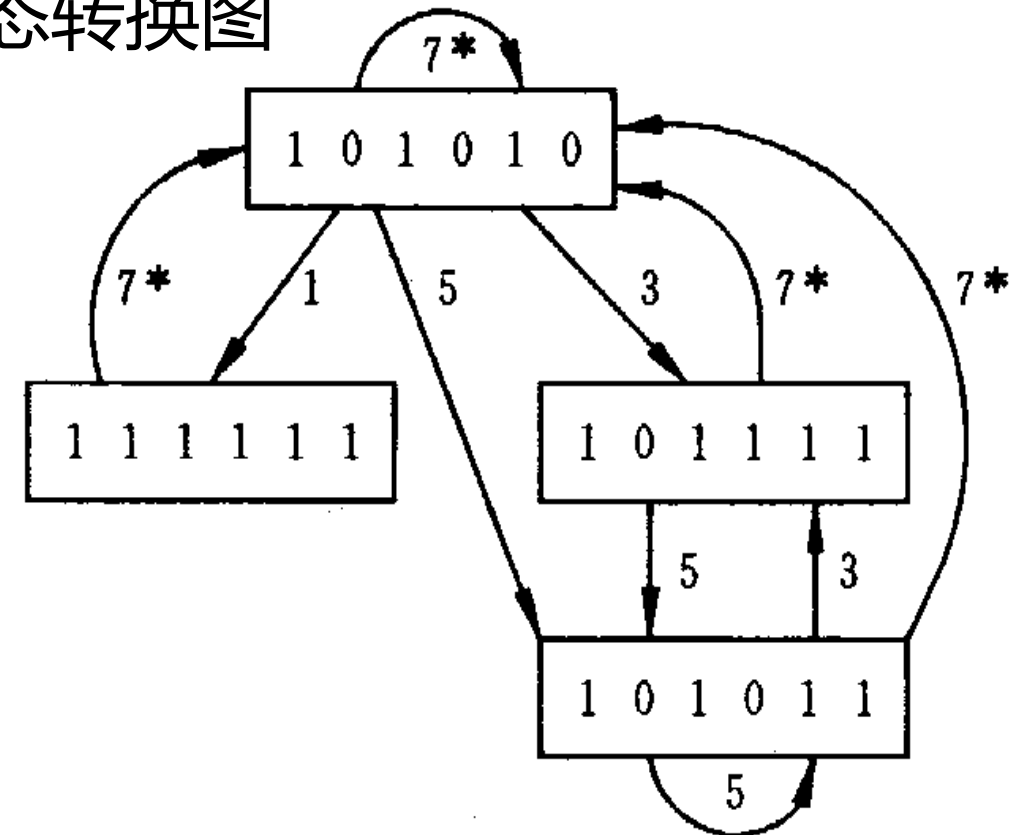
**例子：**非线性流水线的冲突向量为 $C = (101010)$ 。

## 3.3 非线性流水线调度

第 69 页

步骤三：由冲突向量构造调度流水线的状态转换图

1. 将冲突向量C作为初始冲突向量 $C_0$ 送入一个m位逻辑右移移位器，移位m次；
  - 若移出的是“0”，用移位器中的值与初始冲突向量作“按位或”运算，得到一个新的冲突向量；
  - 若移出的是“1”，不作任何处理。
2. 将中间形成的每一个新的冲突向量同样处理；
3. 画出状态图。

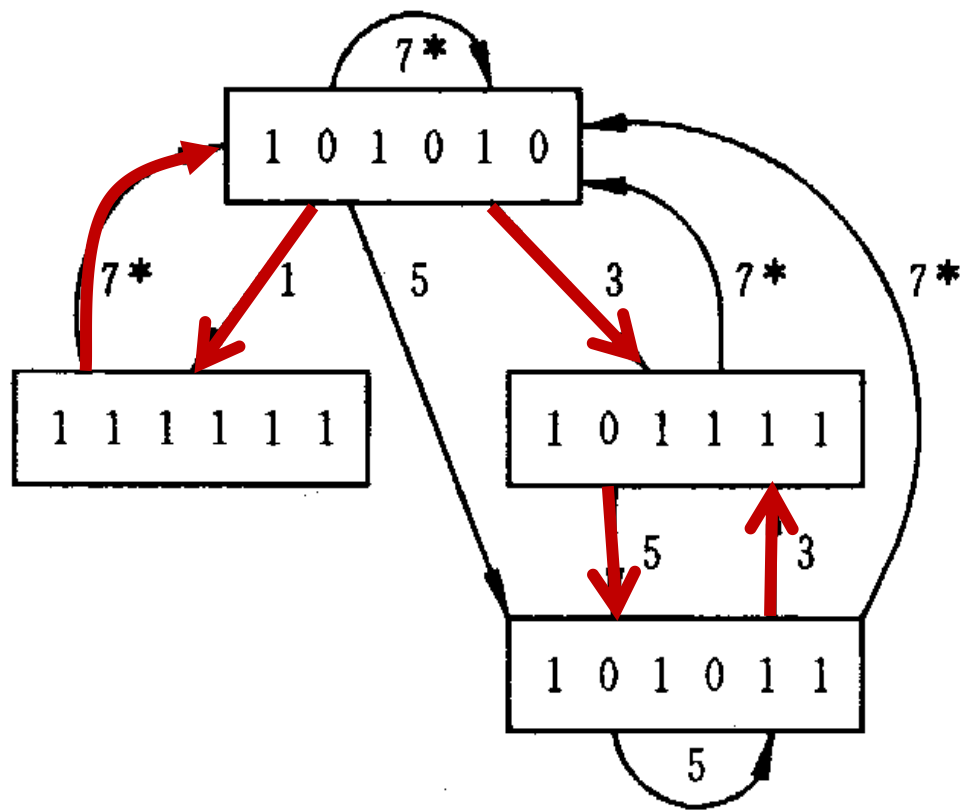


$\text{SHR}^j(C_0) \vee C_0$   
其中： $\text{SHR}^j$ 表示逻辑右移j位

### 3.3 非线性流水线调度

第 70 页

步骤三：由冲突向量构造调度流水线的状态转换图



$\text{SHR}^{(j)}(C_0) \vee C_0$   
其中： $\text{SHR}^{(j)}$ 表示逻辑右移 $j$ 位

$$\begin{aligned}\text{SHR}^{(1)}(101010) &= 010101 \\ (010101) \vee (101010) &= 111111 \\ \text{SHR}^{(7)}(111111) &= 000000 \\ (000000) \vee (101010) &= 101010\end{aligned}$$

第一组

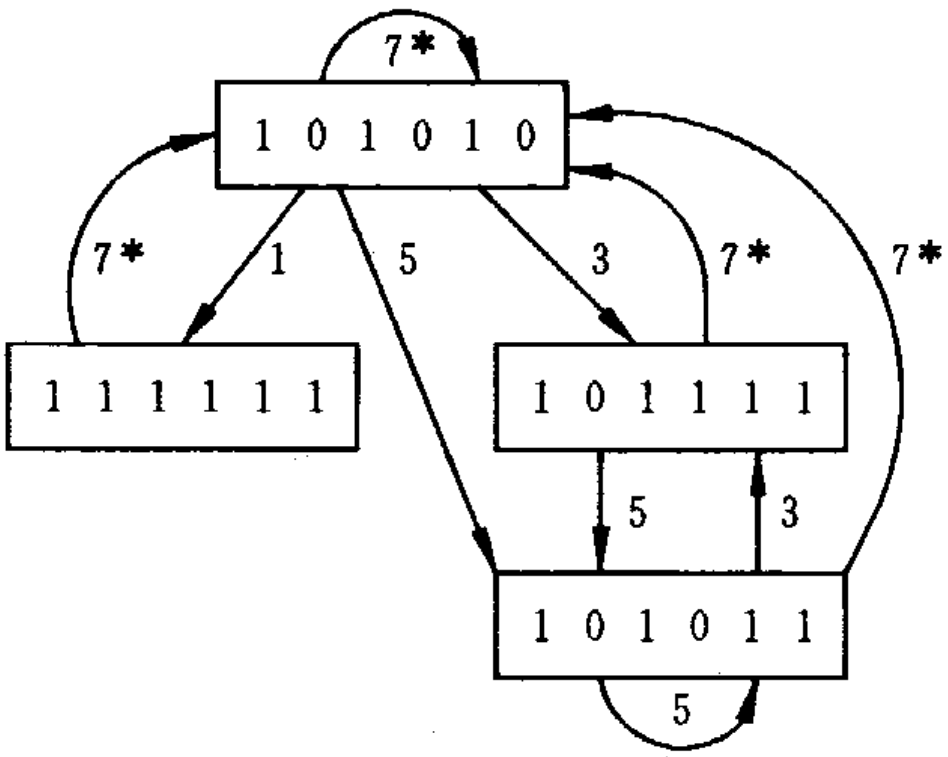
$$\begin{aligned}\text{SHR}^{(3)}(101010) &= 000101 \\ (000101) \vee (101010) &= 101111 \\ \text{SHR}^{(5)}(101111) &= 000001 \\ (000001) \vee (101010) &= 101011 \\ \text{SHR}^{(3)}(101011) &= 000101 \\ (000101) \vee (101010) &= 101111\end{aligned}$$

第二组

# 3.3 非线性流水线调度

步骤四：在状态图中找出可用启动距离，并计算平均启动距离

在状态图中从初始状态出发，能构成一种间隔拍数呈周期性重复的方案就是可用启动距离。即找出所有的**简单循环**（指在状态图中各种冲突向量只经过一次的启动循环）。



| 简单循环    | 平均启动距离 |
|---------|--------|
| 1, 7    | 4      |
| 3, 7    | 5      |
| 5, 7    | 6      |
| 3, 5, 7 | 5      |
| 5, 3, 7 | 5      |
| 3, 5    | 4      |
| 5       | 5      |
| 7       | 7      |

## 3.3 非线性流水线调度

第 72 页

步骤五：找出平均启动距离最小的启动循环或恒定循环

**启动循环：** (1, 7) 和 (3, 5)

最小启动循环 (1, 7) 的流水线工作状态

|                | 1              | 2              | 3              | 4              | 5              | 6              | 7              | 8              | 9              | 10             | 11             | 12             | 13             | 14             | 15             | 16             | 17             | 18             | 19             | 20             | ... |
|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|-----|
| S <sub>1</sub> | X <sub>1</sub> | X <sub>2</sub> |                |                |                |                | X <sub>1</sub> | X <sub>2</sub> | X <sub>3</sub> | X <sub>4</sub> |                |                |                |                | X <sub>3</sub> | X <sub>4</sub> | X <sub>5</sub> | X <sub>6</sub> |                |                |     |
| S <sub>2</sub> |                | X <sub>1</sub> | X <sub>2</sub> |                |                | X <sub>1</sub> | X <sub>2</sub> |                |                | X <sub>3</sub> | X <sub>4</sub> |                |                | X <sub>3</sub> | X <sub>4</sub> |                |                | X <sub>5</sub> | X <sub>6</sub> |                |     |
| S <sub>3</sub> |                |                | X <sub>1</sub> | X <sub>2</sub> | X <sub>1</sub> | X <sub>2</sub> |                |                |                |                | X <sub>3</sub> | X <sub>4</sub> | X <sub>3</sub> | X <sub>4</sub> |                |                |                |                | X <sub>5</sub> | X <sub>6</sub> |     |
| S <sub>4</sub> |                |                |                | X <sub>1</sub> | X <sub>2</sub> |                |                |                |                |                |                | X <sub>3</sub> | X <sub>4</sub> |                |                |                |                |                |                | X <sub>5</sub> |     |
| 启动周期           |                |                |                |                |                |                |                | 重复启动周期         |                |                |                |                |                |                |                |                |                |                |                |                |     |

思考：是否可以进一步发挥流水线效率？



### 3.3 非线性流水线调度

第 73 页

- 优化调度方法：L.E.Shar于1972年提出流水线最小平均启动距离的限制范围
  1. **下限是预约表中任意一行里 “×” 的最多个数。**
  2. 小于或等于状态图中任意一个简单循环的平均启动距离。
  3. 最小平均启动距离的上限是冲突向量中1的个数再加上1。
- 1992年，L.E.Shar又证明了上述限制范围。

最有用的是第1条，即预约表中 “×” 最多的行一定是瓶颈流水段

# 3.3 非线性流水线调度

第 74 页

一个4功能段非线性流水线预约表

| 时间<br>流水段      | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|----------------|---|---|---|---|---|---|---|
| S <sub>1</sub> | × |   |   |   |   |   | × |
| S <sub>2</sub> |   | × |   |   |   | × |   |
| S <sub>3</sub> |   |   | × |   | × |   |   |
| S <sub>4</sub> |   |   |   | × |   |   |   |



| 简单循环    | 平均启动距离 |
|---------|--------|
| 1, 7    | 4      |
| 3, 7    | 5      |
| 5, 7    | 6      |
| 3, 5, 7 | 5      |
| 5, 3, 7 | 5      |
| 3, 5    | 4      |
| 5       | 5      |
| 7       | 7      |

- 1、理想最小平均启动距离 (MAL) 等于预约表任一行中 “×” 的最多个数
- 2、理想最小启动循环：一般恒定循环作为最小启动循环。

对于上面的例子：理想最小平均启动距离为2，理想最小启动循环为恒定循环（2）。

## 3.3 非线性流水线调度

第 75 页

采用**预留算法**来调度非线性流水线，可以达到最优调度：

■ 核心思想：通过插入非计算延迟段——修改预约表实现最小启动循环。

采用预留调度算法的预约表

| 时 间         |                | 1 | 2 | 3 | 4 | 5     | 6     | 7     | 8 |
|-------------|----------------|---|---|---|---|-------|-------|-------|---|
| 功<br>能<br>段 | S <sub>1</sub> | × |   |   |   |       |       | ① → × |   |
|             | S <sub>2</sub> |   | × |   |   |       | ① → × |       |   |
|             | S <sub>3</sub> |   |   | × |   | ① → × |       |       |   |
|             | S <sub>4</sub> |   |   |   | × |       |       |       |   |
| 延迟          | D <sub>1</sub> |   |   |   |   | ×     |       |       |   |

注：①表示由D<sub>1</sub>延迟一个时钟周期

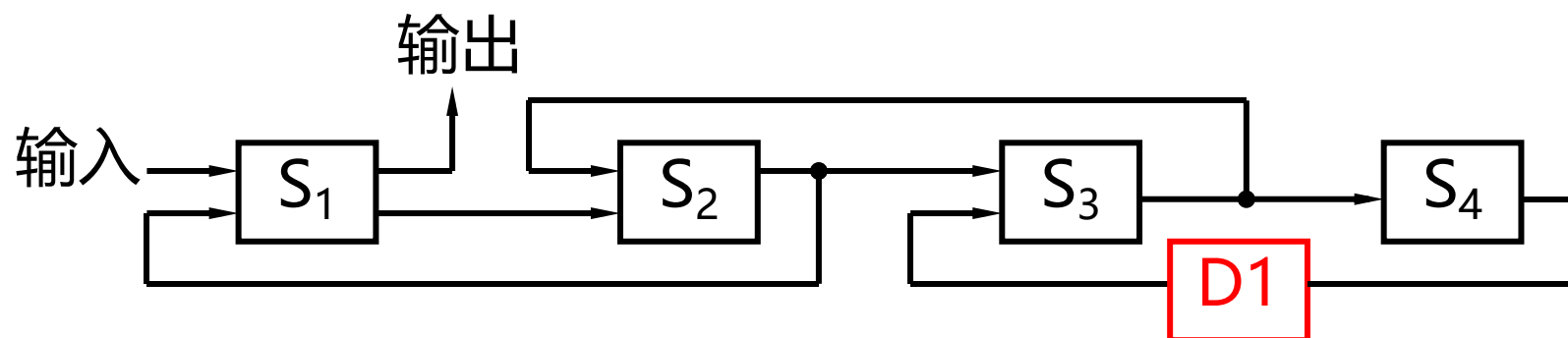
实现方案：只要在流水段S<sub>4</sub>的输出端到流水段S<sub>3</sub>的输入端中间插入一个非计算延迟D<sub>1</sub>

■ 每一行中与第1个“×”的距离为2的倍数的位置都要预留出来。

- S<sub>3</sub>行的第2个“×”从周期5延迟到周期6
- S<sub>2</sub>行的第2个“×”要向后延迟一个周期，从周期6延迟到周期7；
- S<sub>1</sub>行的第2个“×”要向后延迟一个周期，从周期7延迟到周期8。

### 3.3 非线性流水线调度

第 76 页



| 时间  |       | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|-----|-------|---|---|---|---|---|---|---|---|
| 功能段 | $S_1$ | × |   |   |   |   |   |   | × |
|     | $S_2$ |   | × |   |   |   |   | × |   |
|     | $S_3$ |   |   | × |   |   | × |   |   |
|     | $S_4$ |   |   |   | × |   |   |   |   |
| 延迟  | $D_1$ |   |   |   |   | × |   |   |   |

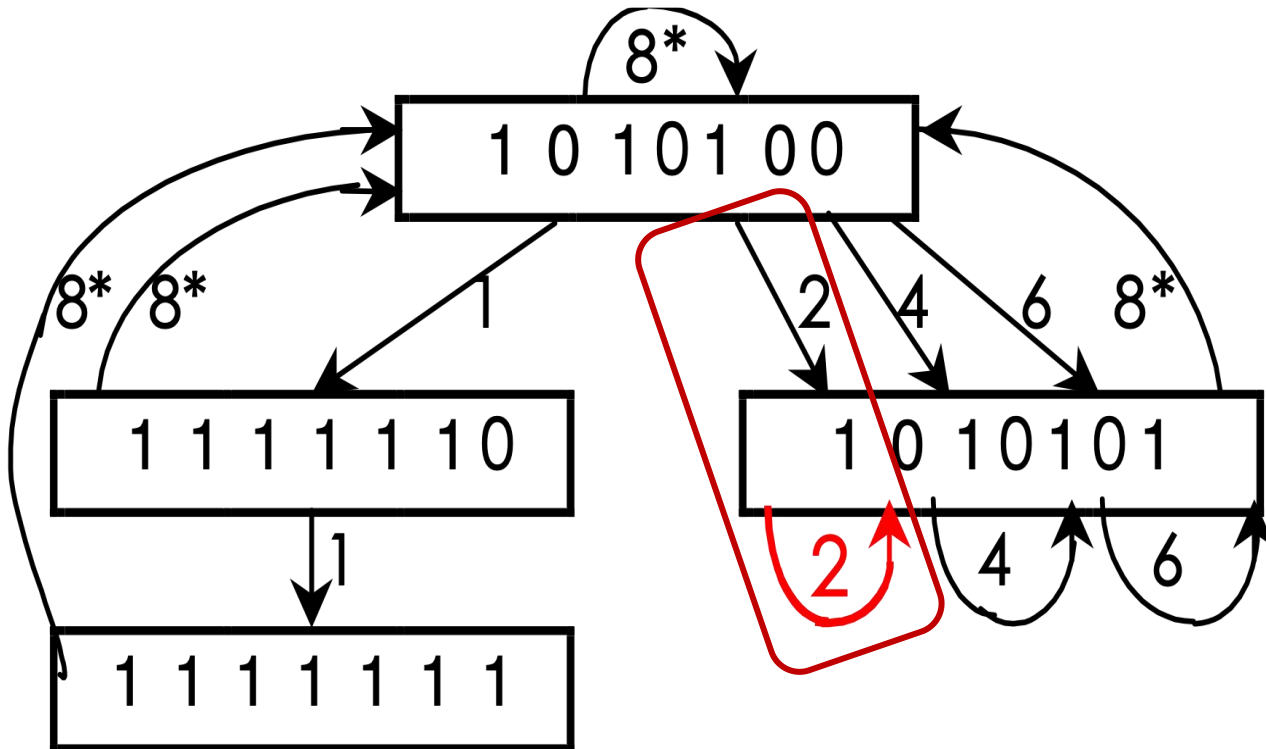
Annotations on the right side of the table:

- Blue arrow pointing left: 7
- Blue arrow pointing left: 5
- Blue arrow pointing left: 3
- Green dotted arrows indicating dependencies:  $S_4 \rightarrow S_3$ ,  $S_3 \rightarrow S_2$ ,  $S_2 \rightarrow S_1$ .

## 3.3 非线性流水线调度

第 77 页

有非计算延迟的流水线状态图



$$\text{SHR}^{(1)}(1010100) = 0101010$$
$$(0101010) \vee (1010100) = 1111110$$

$$\text{SHR}^{(2)}(1010100) = 0010101$$
$$(0010101) \vee (1010100) = 1010101$$
$$\text{SHR}^{(2)}(1010101) = 0010101$$
$$(0010101) \vee (1010100) = 1010101$$

| 简单循环 | 平均启动距离 |
|------|--------|
| 2    | 2      |

## 3.3 非线性流水线调度

第 78 页

按照最小启动循环 (2) 工作的流水线预约表

| 时间          |                | 1              | 2              | 3              | 4              | 5              | 6              | 7              | 8              | 9              | 10             | 11             | 12             | ... |
|-------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|-----|
| 功<br>能<br>段 | S <sub>1</sub> | X <sub>1</sub> |                | X <sub>2</sub> |                | X <sub>3</sub> |                | X <sub>4</sub> | X <sub>1</sub> | X <sub>5</sub> | X <sub>2</sub> | X <sub>6</sub> | X <sub>3</sub> | ... |
|             | S <sub>2</sub> |                | X <sub>1</sub> |                | X <sub>2</sub> |                | X <sub>3</sub> | X <sub>1</sub> | X <sub>4</sub> | X <sub>2</sub> | X <sub>5</sub> | X <sub>3</sub> | X <sub>6</sub> | ... |
|             | S <sub>3</sub> |                |                | X <sub>1</sub> |                | X <sub>2</sub> | X <sub>1</sub> | X <sub>3</sub> | X <sub>2</sub> | X <sub>4</sub> | X <sub>3</sub> | X <sub>5</sub> | X <sub>4</sub> | ... |
|             | S <sub>4</sub> |                |                |                | X <sub>1</sub> |                | X <sub>2</sub> |                | X <sub>3</sub> |                | X <sub>4</sub> |                | X <sub>5</sub> | ... |
| 延           | D <sub>1</sub> |                |                |                |                | X <sub>1</sub> |                | X <sub>2</sub> |                | X <sub>3</sub> |                | X <sub>4</sub> |                | ... |

- 在非线性流水线中，“×”最多的流水段一定是“瓶颈”流水段。
- 实现最优调度的目标是使“瓶颈”流水段处于忙碌状态，没有空闲周期。
- 最优调度方法能够使非线性流水线的吞吐率、加速比和效率达到最优。

## 3.3 非线性流水线调度

第 79 页

例3.4：一个5功能段非线性流水线预约表

| <div>时间</div> <div>功能段</div> | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|------------------------------|---|---|---|---|---|---|---|---|---|
| $S_1$                        | ✓ |   |   |   |   |   |   |   | ✓ |
| $S_2$                        |   | ✓ | ✓ |   |   |   |   | ✓ |   |
| $S_3$                        |   |   |   | ✓ |   |   |   |   |   |
| $S_4$                        |   |   |   |   | ✓ | ✓ |   |   |   |
| $S_5$                        |   |   |   |   |   |   | ✓ | ✓ |   |

求最小启动循环。

## 3.3.1 单功能非线性流水线的调度

第 80 页

1. 根据预约表写出禁止表F

| 时间<br>功能段 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |           |
|-----------|---|---|---|---|---|---|---|---|---|-----------|
| $S_1$     | √ |   |   |   |   |   |   |   | √ | ← 8       |
| $S_2$     |   | √ | √ |   |   |   |   | √ |   | ← 1, 5, 6 |
| $S_3$     |   |   |   | √ |   |   |   |   |   |           |
| $S_4$     |   |   |   |   | √ | √ |   |   |   | ← 1       |
| $S_5$     |   |   |   |   |   |   | √ | √ |   | ← 1       |

其禁止表是：  $F = \{ 1, 5, 6, 8 \}$



## 3.3 非线性流水线调度

第 81 页

### 2. 根据初始冲突向量 $C_0$ 画出状态转换图

(1)  $C_0 = (10110001)$

引入后续任务可用的时间间隔为：2、3、4、7个时钟周期

如果采用2，则新的冲突向量为：

$$(00101100) \vee (10110001) = (10111101)$$

如果采用3，则新的冲突向量为：

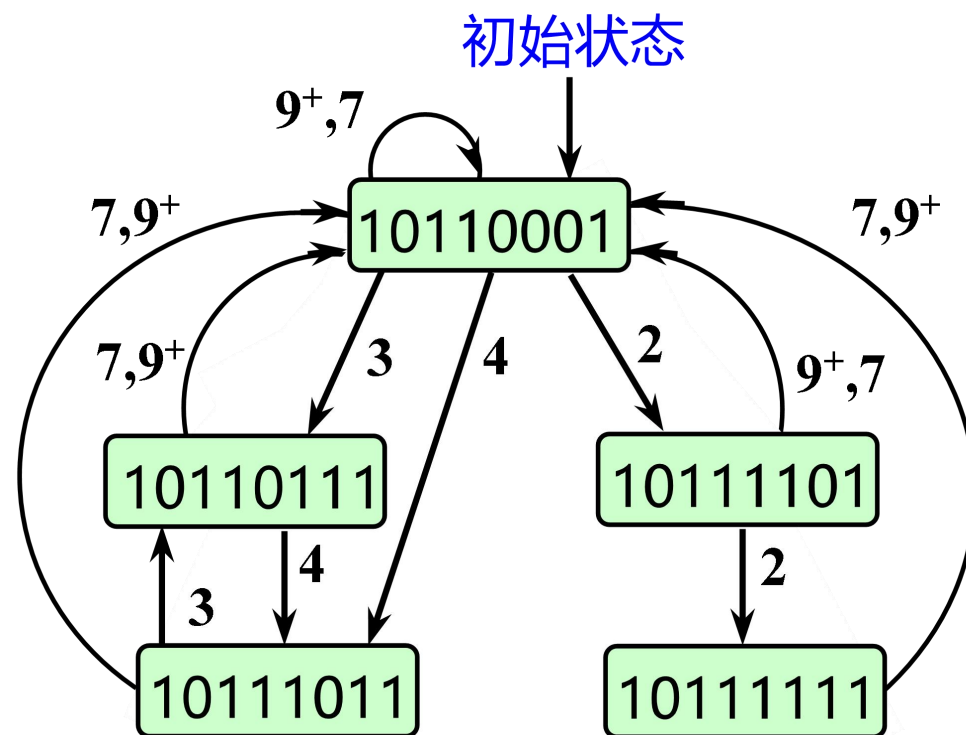
$$(00010110) \vee (10110001) = (10110111)$$

如果采用4，则新的冲突向量为：

$$(00001011) \vee (10110001) = (10111011)$$

如果采用7，则新的冲突向量为：

$$(00000001) \vee (10110001) = (10110001)$$



## 3.3 非线性流水线调度

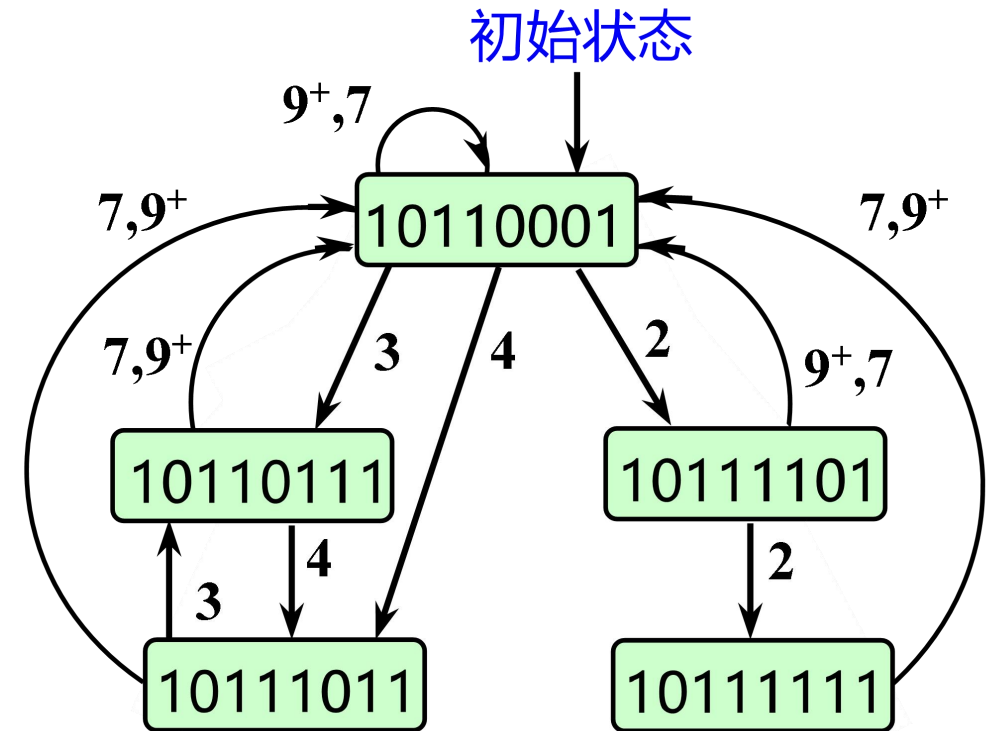
第 82 页

2. 根据初始冲突向量 $C_0$ 画出状态转换图

(2) 对于新向量 (10111101)，其可用的时间间隔为2个和7个时钟周期。用类似上面的方法，可以求出其后续的冲突向量分别为 (10111101) 和 (10110001)。

(3) 对于其他新向量，也照此处理。

(4) 在此基础上，画出状态转移示意图。



## 3.3 非线性流水线调度

第 83 页

### 3. 根据状态转换图写出最优调度方案

各种调度策略及平均延迟拍数

| 调度策略      | 平均延迟拍数 |
|-----------|--------|
| (2,7)     | 4.5    |
| (2,2,7)   | 3.67   |
| (3,7)     | 5      |
| (3,4)     | 3.5    |
| (3,4,3,7) | 4.25   |
| (3,4,7)   | 4.67   |
| (4,3,7)   | 4.67   |
| (4,7)     | 5.5    |
| (7)       | 7      |

➤ 最佳方案： (3, 4)

平均间隔时间：3.5个时钟周期（吞吐率最高）

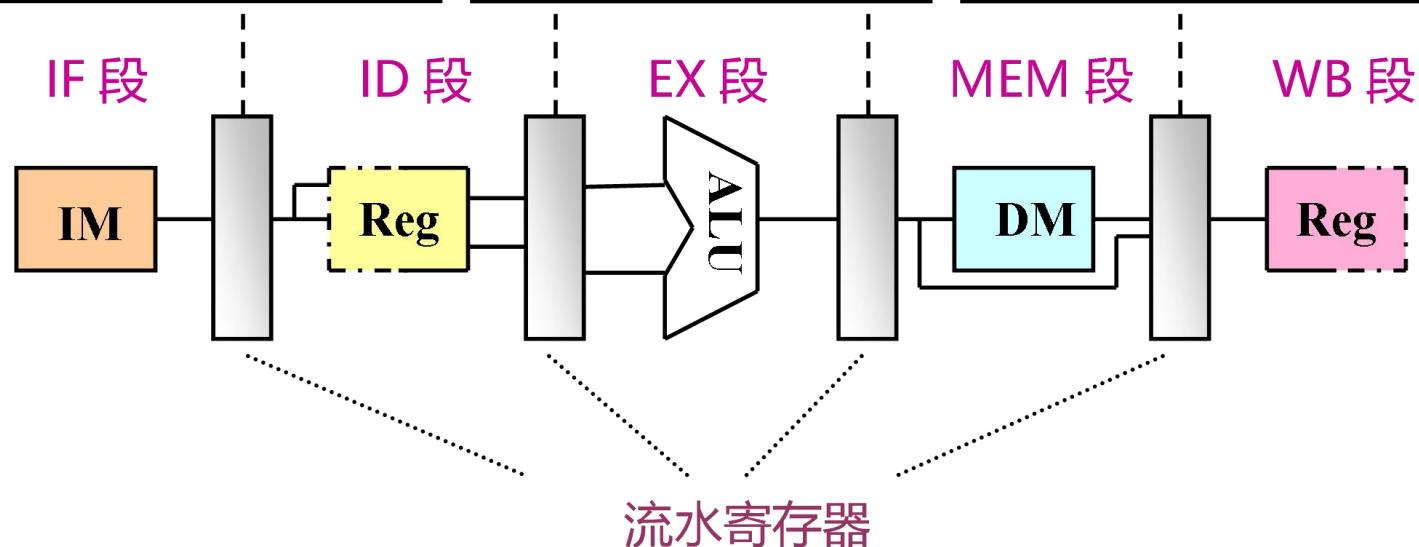
- 方案 (3, 4) 是一种不等时间间隔的调度方案，与等间隔的调度方案相比，在控制上要复杂得多。为了简化控制，也可以采用等间隔时间的调度方案，但吞吐率和效率往往会下降不少。
- 等时间间隔的方案只有一个： (7) ，其吞吐率下降了一半。

## 3.4 流水线的相关与冲突

## 3.4.1 一个经典的5段流水线

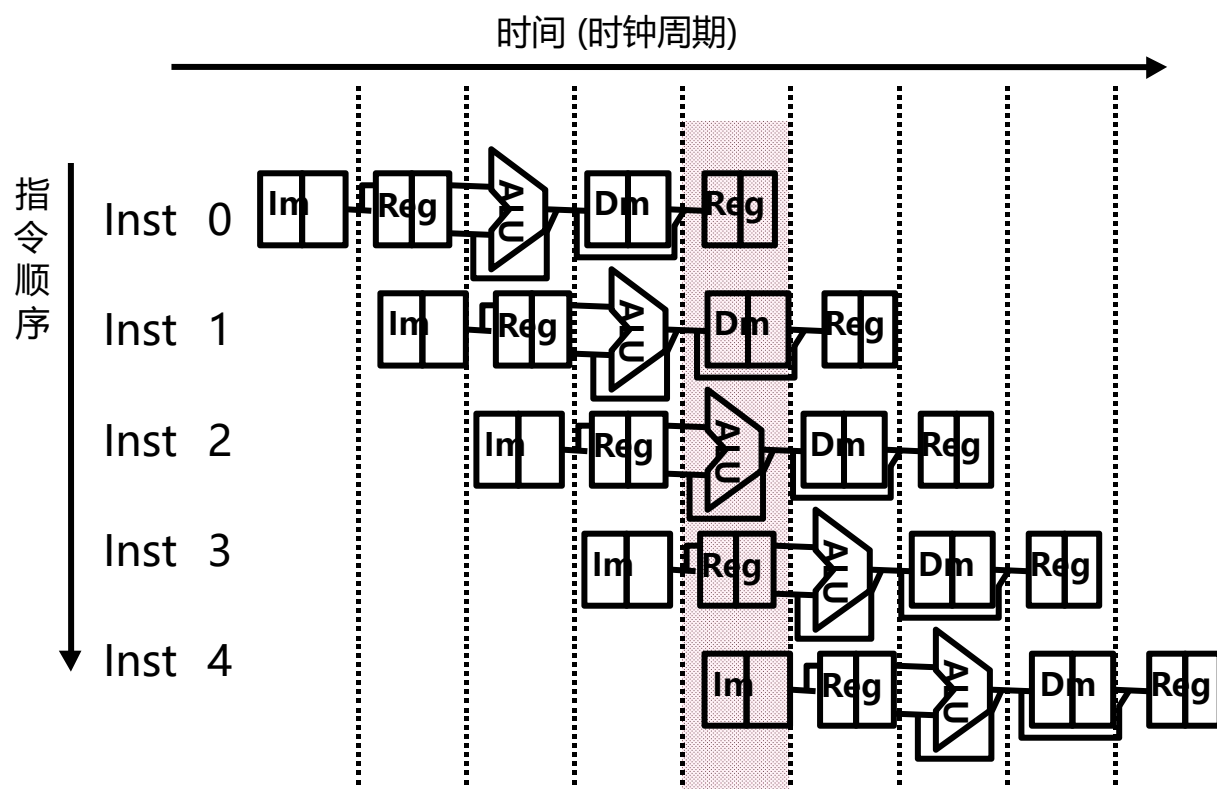
第 85 页

| 取指IF段                                                                                     | 译码ID段                                                                                            | 执行EX段                                                                                                           | 访存MEM段                                                                                                     | 写回WB段                                                                      |
|-------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>■ 取IR (PC)</li><li>■ <math>PC=PC+4</math></li></ul> | <ul style="list-style-type: none"><li>■ 指令译码,</li><li>■ 根据IR中的寄存器地址去访问通用寄存器组, 读出所需的操作数</li></ul> | <ul style="list-style-type: none"><li>■ ALU指令</li><li>■ load和store指令 (数据地址计算)</li><li>■ 分支指令 (指令地址计算)</li></ul> | <ul style="list-style-type: none"><li>■ store指令 (存数据)</li><li>■ load指令 (取数据)</li><li>■ 分支指令 (执行)</li></ul> | <ul style="list-style-type: none"><li>■ ALU运算指令</li><li>■ load指令</li></ul> |



## 3.4.1 一个经典的5段流水线

第 86 页



■ 采用流水线方式实现时，应解决好如下冲突问题：

- 不会在同一时钟周期要求同一个功能段做两件不同的工作。

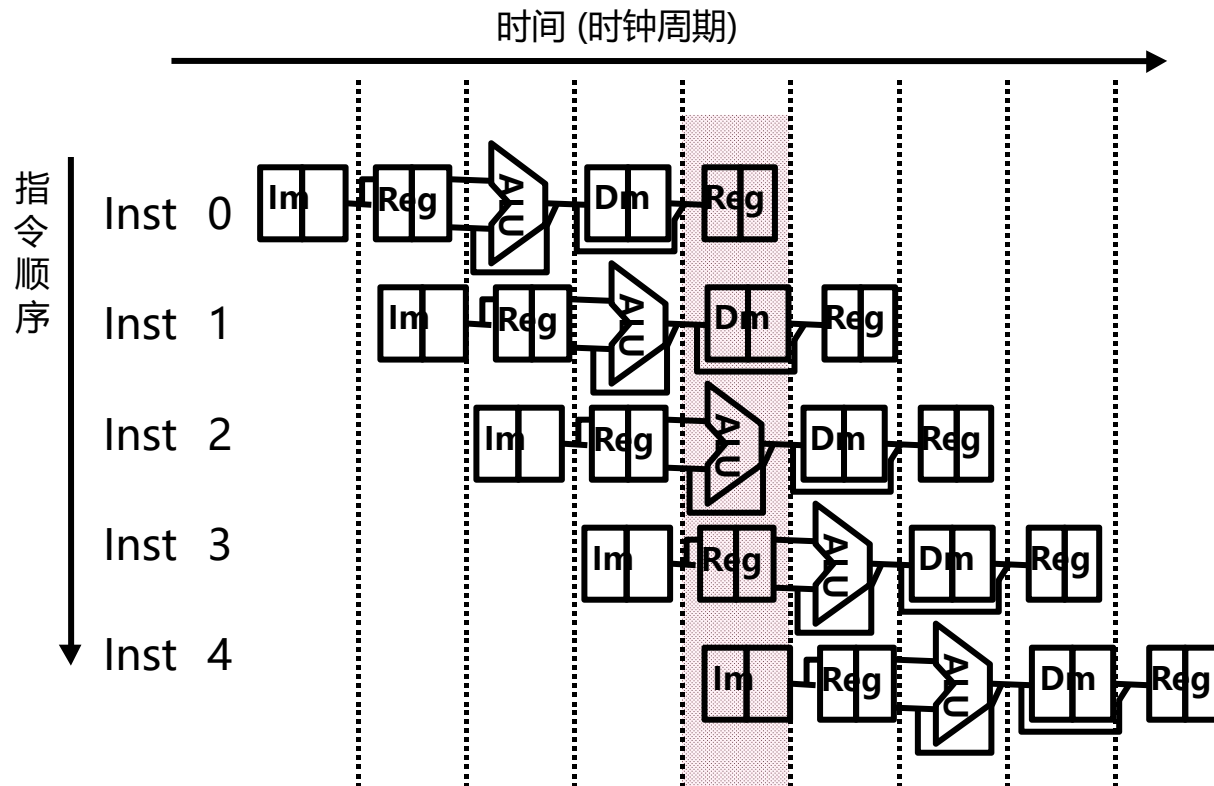
例如：不能要求ALU同时做有效地址计算和算术运算。

- 避免IF段的访存（取指令）与MEM段的访存（读/写数据）发生冲突。

可以采用分离的指令存储器和数据存储器，一般采用分离的指令Cache和数据Cache。

## 3.4.1 一个经典的5段流水线

第 87 页



■ 采用流水线方式实现时，应解决好如下冲突问题：

➤ ID段和WB段都要访问同一寄存器文件。

ID段：读      WB段：写

如何解决？先写后读？

➤ 分支指令也可能改变PC的值，而且是在MEM段进行，这会导致冲突。

如何处理分支指令？。

## 3.4.2 流水线相关

第 88 页

- 相关：两条指令之间存在某种依赖关系。

如果两条指令相关，则它们就有可能不能在流水线中重叠执行或者只能部分重叠执行。

- 相关有3种类型

- 数据相关（也称真数据相关）
- 名相关
- 控制相关



## 3.4.2 流水线相关

第 89 页

### 1. 数据相关

- 对于两条指令*i*（在前，下同）和*j*（在后，下同），如果下述条件之一成立，则称指令*j*与指令*i*数据相关。

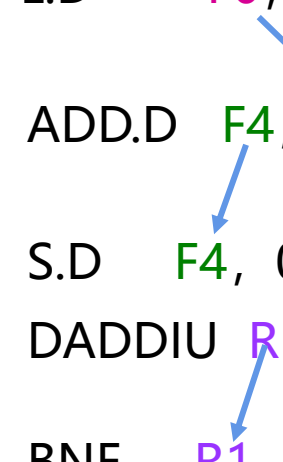
- 指令*j*使用指令*i*产生的结果；
- 指令*j*与指令*k*数据相关，而指令*k*又与指令*i*数据相关。

- 数据相关具有传递性。

数据相关反映了数据的流动关系，即如何从其产生者流动到其消费者。

例如：下面这一段代码存在数据相关。

```
Loop: L.D    F0, 0 (R1)    // F0为数组元素
      ADD.D  F4, F0, F2    // 加上F2中的值
      S.D    F4, 0 (R1)    // 保存结果
      DADDIU R1, R1, -8    // 数组指针递减8个字节
      BNE    R1, R2, Loop  // 如果R1≠R2, 则分支
```



The diagram illustrates the data flow between instructions. A blue arrow points from the register **F0** in the first instruction (L.D) to the register **F4** in the second instruction (ADD.D). Another blue arrow points from the register **F4** in the second instruction to the register **R1** in the fourth instruction (DADDIU). This shows how data produced by one instruction is used by subsequent instructions, creating data dependencies.

- 当数据的流动是经过寄存器时，相关的检测比较直观和容易。 **(寄存器直接寻址)**
- 当数据的流动是经过存储器时，检测比较复杂。
  - 相同形式的地址其有效地址未必相同；
  - 形式不同的地址其有效地址却可能相同。

寻址方式不同

### 2. 名相关

- 名：指令所访问的寄存器或存储器单元的名称。
- 如果两条指令使用相同的名，但是它们之间并没有数据流动，则称这两条指令存在名相关。
- 指令j与指令i之间的名相关有两种：
  - 反相关：如果指令j写的名与指令i读的名相同，则称指令i和j发生了反相关。  
指令j写的名 = 指令i读的名
  - 输出相关：如果指令j和指令i写相同的名，则称指令i和j发生了输出相关。  
指令j写的名 = 指令i写的名
- 如果名相关的两条指令之间并没有数据的传送，如果一条指令中的名改变了，并不影响另外一条指令的执行。

## 3.4.2 流水线相关

第 91 页

### ■ 换名技术

- 换名技术：通过改变指令中操作数的名来消除名相关。
- 对于寄存器操作数进行换名称为寄存器换名。

**既可以用编译器静态实现，也可以用硬件动态完成。**

例如：考虑下述代码：

|       |              |
|-------|--------------|
| DIV.D | F2, F8, F4   |
| ADD.D | F8, F0, F12  |
| SUB.D | F10, F8, F14 |

DIV.D和ADD.D存在反相关。

进行寄存器换名（F8换成S）后，变成：

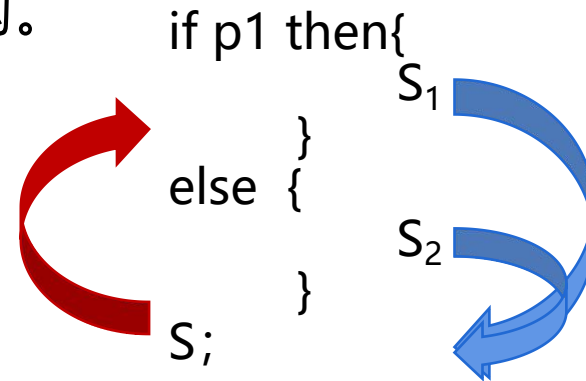
|       |             |
|-------|-------------|
| DIV.D | F2, F8, F4  |
| ADD.D | S, F0, F12  |
| SUB.D | F10, S, F14 |

## 3.4.2 流水线相关

第 92 页

### 3. 控制相关

- 控制相关是指由分支指令引起的相关。
  - 为了保证程序应有的执行顺序，必须严格按控制相关确定的顺序执行。
- 典型的程序结构是 “if-then-else” 结构。
- 请看一个示例：



- 控制相关带来了以下两个限制：
  - 与一条分支指令控制相关的指令不能被移到该分支之前。否则这些指令就不受该分支控制了。对于上述的例子，then 部分中的指令不能移到if语句之前。
  - 如果一条指令与某分支指令不存在控制相关，就不能把该指令移到该分支之后。

对于上述的例子，不能把S移到if语句的then或else部分中。

### 3.4.3 流水线冲突

第 93 页

■流水线冲突是指对于具体的流水线来说，由于相关的存在，使得指令流中的下一条指令不能在指定的时钟周期执行。 **（相关是前提，冲突是后果）**

■流水线冲突有3种类型：

- 结构冲突：因硬件资源满足不了指令重叠执行的要求而发生的冲突。
- 数据冲突：当指令在流水线中重叠执行时因需要用到前面指令的执行结果而发生的冲突。
- 控制冲突：流水线遇到分支指令和其它会改变PC值的指令所引起的冲突。

■带来的几个问题：

- 可能性1：流水线不停顿，导致错误的执行结果。
- 可能性2：流水线出现停顿，降低流水线的效率和实际的加速比。

## 3.4.3 流水线冲突

第 94 页

**指令执行原则：**当一条指令被暂停时，在该暂停指令之后流出的所有指令都要被暂停，而在该暂停指令之前流出的指令则继续进行（否则就永远无法消除冲突）。

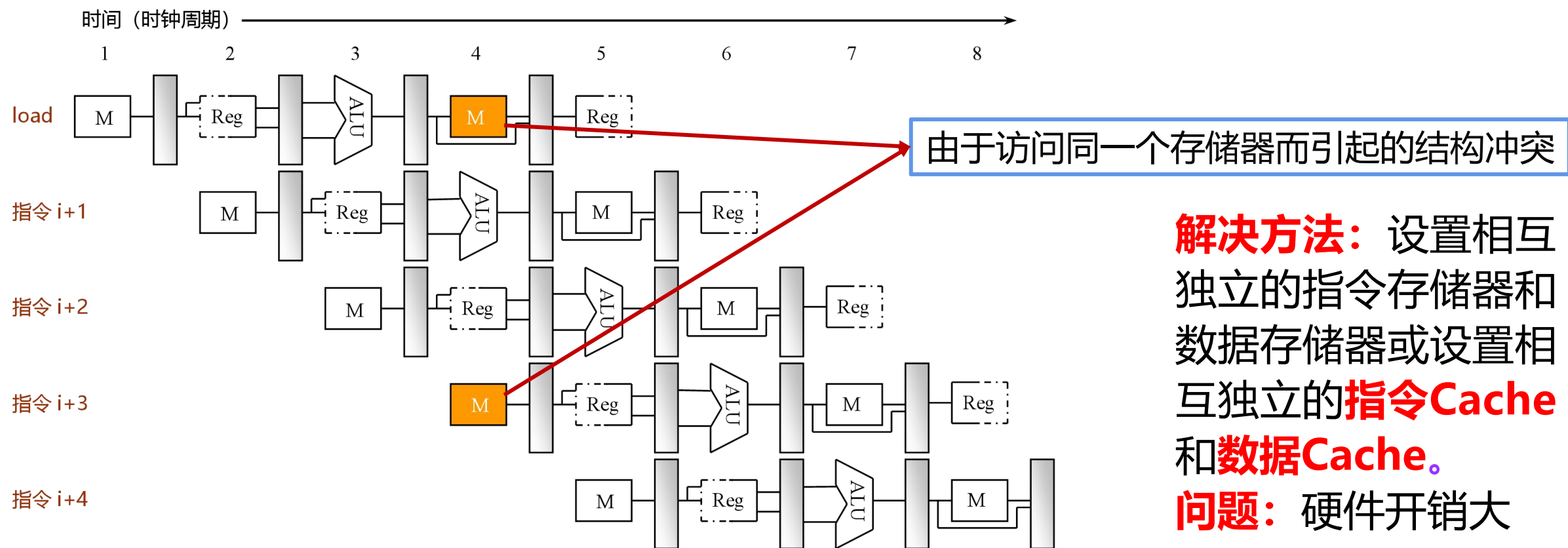
### 1. 结构冲突

- 在流水线处理机中，为了能够使各种组合的指令都能顺利地重叠执行，需要对功能部件进行流水或重复设置资源。
- 如果某种指令组合因为**资源冲突**而不能正常执行，则称该处理机有**结构冲突**。
- 常见的导致结构冲突的原因：
  - 硬件资源份数不够
  - 部件同时服务功能不够

### 3.4.3 流水线冲突

第 95 页

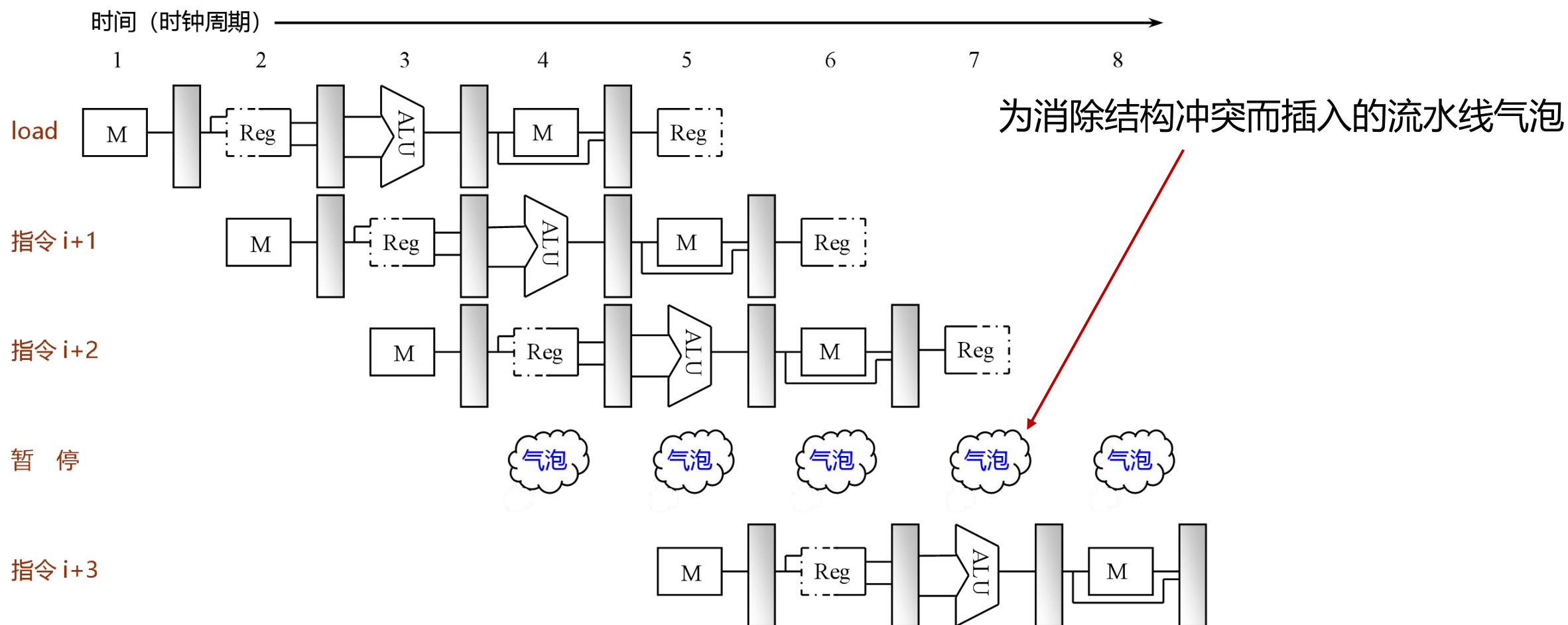
- 结构冲突举例：访存冲突，有些流水线处理机只有一个存储器，将数据和指令放在一起，访存指令会导致访存冲突 —— **增加硬件资源或同时服务取指令和取数据**



### 3.4.3 流水线冲突

第 96 页

- 解决办法：插入暂停周期（“流水线气泡”或“气泡”），引入暂停后的时空图





# 3.4.3 流水线冲突

■ 解决办法：插入暂停周期（“流水线气泡”或“气泡”），引入暂停后的时空图

| 指令编号  | 时钟周期 |    |    |       |     |     |    |     |     |     |
|-------|------|----|----|-------|-----|-----|----|-----|-----|-----|
|       | 1    | 2  | 3  | 4     | 5   | 6   | 7  | 8   | 9   | 10  |
| 指令i   | IF   | ID | EX | MEM   | WB  |     |    |     |     |     |
| 指令i+1 |      | IF | ID | EX    | MEM | WB  |    |     |     |     |
| 指令i+2 |      |    | IF | ID    | EX  | MEM | WB | WB  |     |     |
| 指令i+3 |      |    |    | stall | IF  | ID  | EX | MEM | WB  |     |
| 指令i+4 |      |    |    |       |     | IF  | ID | EX  | MEM | WB  |
| 指令i+5 |      |    |    |       |     |     | IF | ID  | EX  | MEM |

引入暂停后的时空图

## 3.4.3 流水线冲突

第 98 页

### 2. 数据冲突

当相关的指令靠得足够近时，它们在流水线中的重叠执行或者重新排序会改变指令读/写操作数的顺序，使之不同于它们串行执行时的顺序，则发生了数据冲突。

举例：

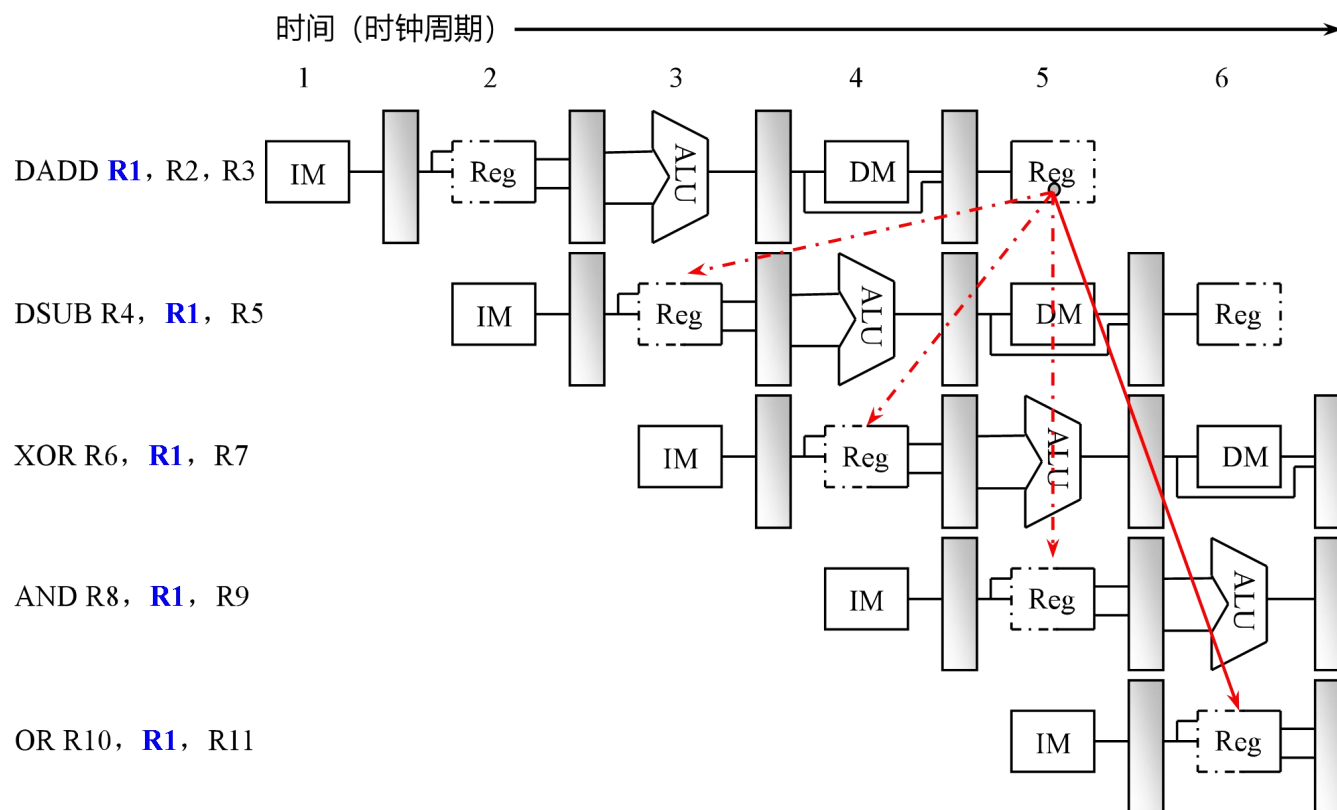
DADD R1, R2, R3

DSUB R4, R1, R5

XOR R6, R1, R7

AND R8, R1, R9

OR R10, R1, R11



流水线的数  
据冲突举例

### 3.4.3 流水线冲突

第 99 页

■根据指令读访问和写访问的顺序，可以将数据冲突分为3种类型。考虑两条指令i和j，且i在j之前进入流水线，可能发生的数据冲突有：

- 写后读冲突（RAW）：在i写入之前，j先去读。j读出的内容是错误的。这是最常见的一种数据冲突，它对应于**真数据相关**。

例子：



|                  |                   |
|------------------|-------------------|
| i: DADD R1,R2,R3 | ;(R2) + (R3)→(R1) |
| j: DSUB R4,R1,R5 | ;(R1) - (R5)→(R4) |

### 3.4.3 流水线冲突

第 100 页

■根据指令读访问和写访问的顺序，可以将数据冲突分为3种类型。考虑两条指令i和j，且i在j之前进入流水线，可能发生的数据冲突有：

➤写后写冲突（WAW）：在i写入之前，j先写。最后写入的结果是i的（错误）！这种冲突对应于输出相关。

✓ 写后写冲突仅发生在这样的流水线中：流水线中不只一个段可以进行写操作；指令被重新排序了

例子

 i: DSUB R1,R4,R5  
j: DADD R1,R2,R3

;(R4) - (R5)→(R1)

;(R2) + (R3)→(R1)

### 3.4.3 流水线冲突

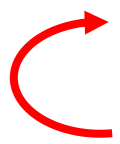
第 101 页

■根据指令读访问和写访问的顺序，可以将数据冲突分为3种类型。考虑两条指令i和j，且i在j之前进入流水线，可能发生的数据冲突有：

➤读后写冲突（WAR）：在i读之前，j先写。i读出的内容是错误的！由反相关引起。

✓ 这种冲突仅发生在这样的情况下：有些指令的写结果操作提前了，而且有些指令的读操作滞后了；指令被重新排序了。

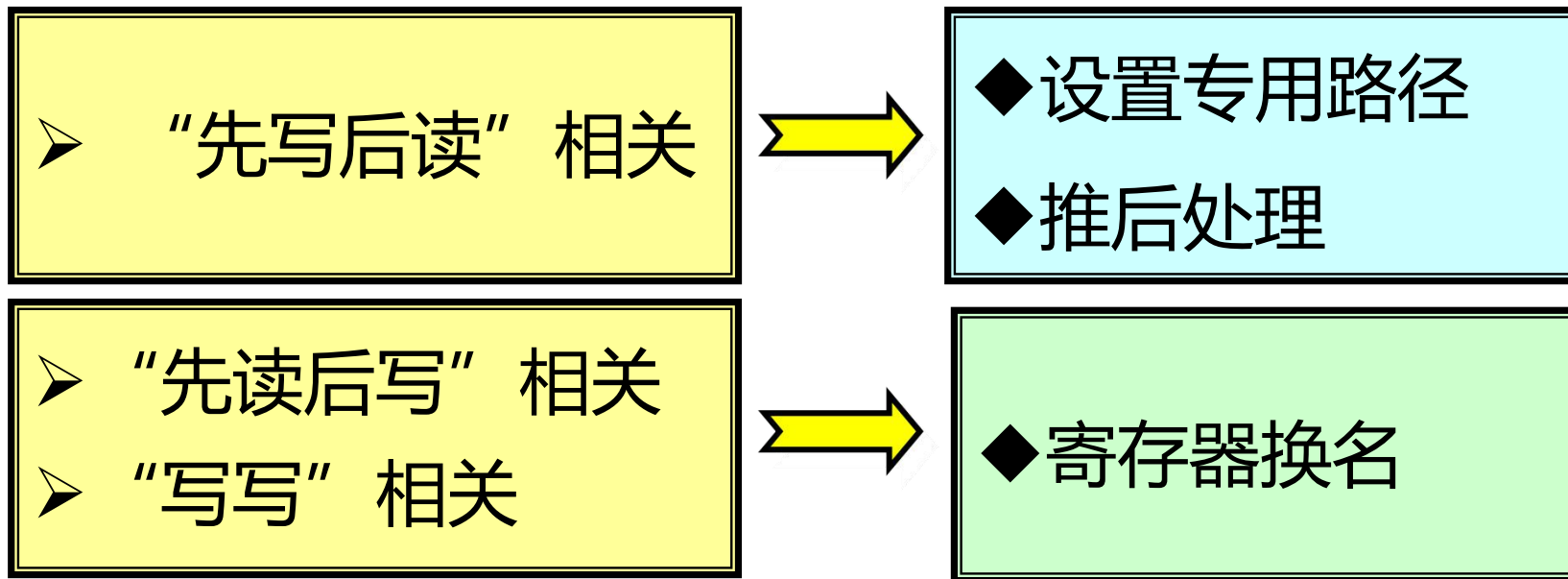
例子

 i: DSUB R4, R1, R5 ; (R1) - (R5) → (R4)  
j: DADD R1, R2, R3 ; (R2) + (R3) → (R1)

### 3.4.3 流水线冲突

第 102 页

- “先写后读” 相关在流水线顺序执行和乱序执行时都可能发生
- “先读后写” 相关和 “写写” 相关只有在流水线乱序执行时才可能发生，而 “读读” 相关无需处理。



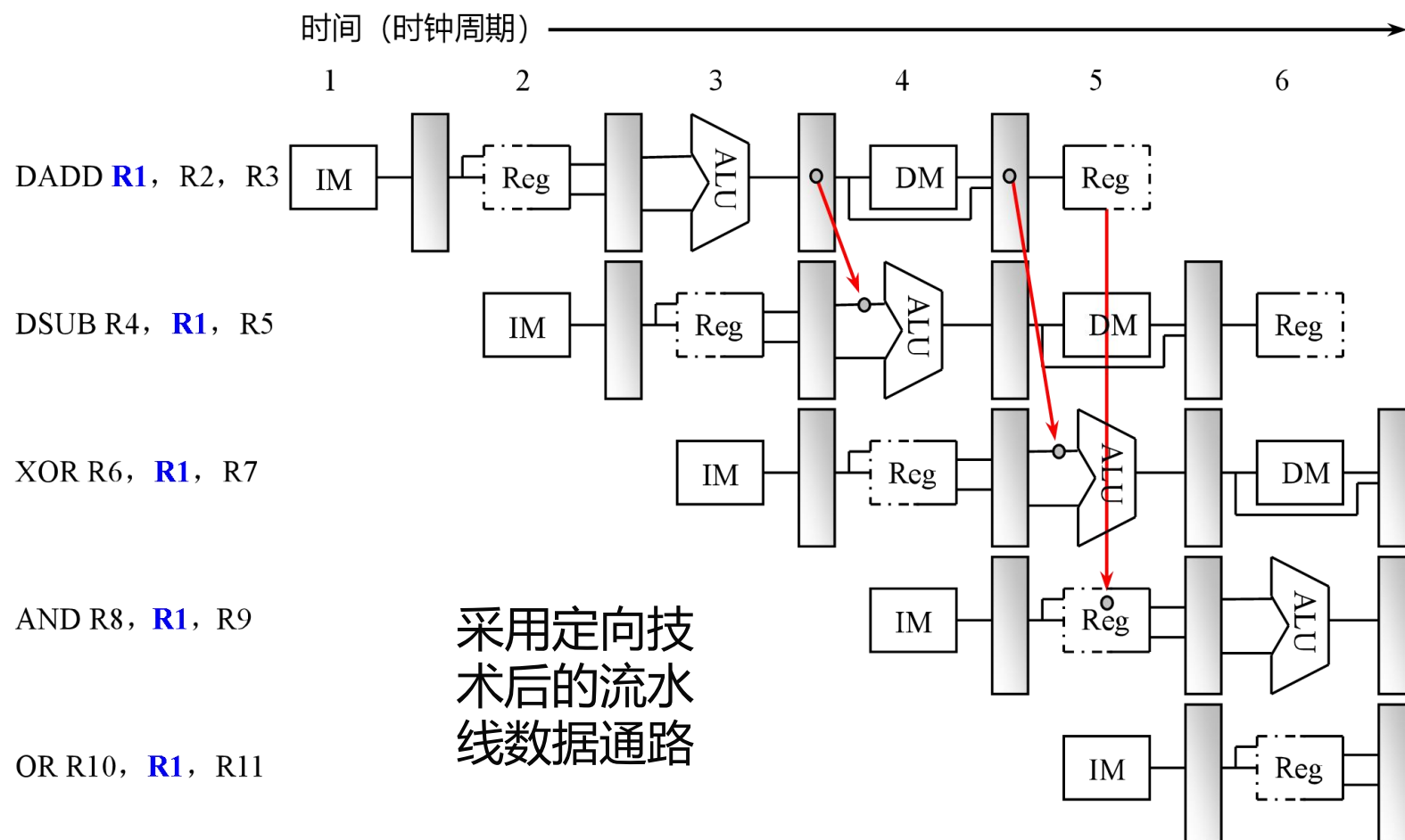
## 3.4.3 流水线冲突

第 103 页

■ 通过定向技术减少数据冲突引起的停顿（定向技术也称为旁路或短路）

- 关键思想：在计算结果尚未出来之前，后面等待使用该结果的指令并不真正立即需要该计算结果，如果能够将该计算结果从其产生的地方直接送到其它指令需要它的地方，那么就可以避免停顿。

- 采用定向技术消除上例中的相关。



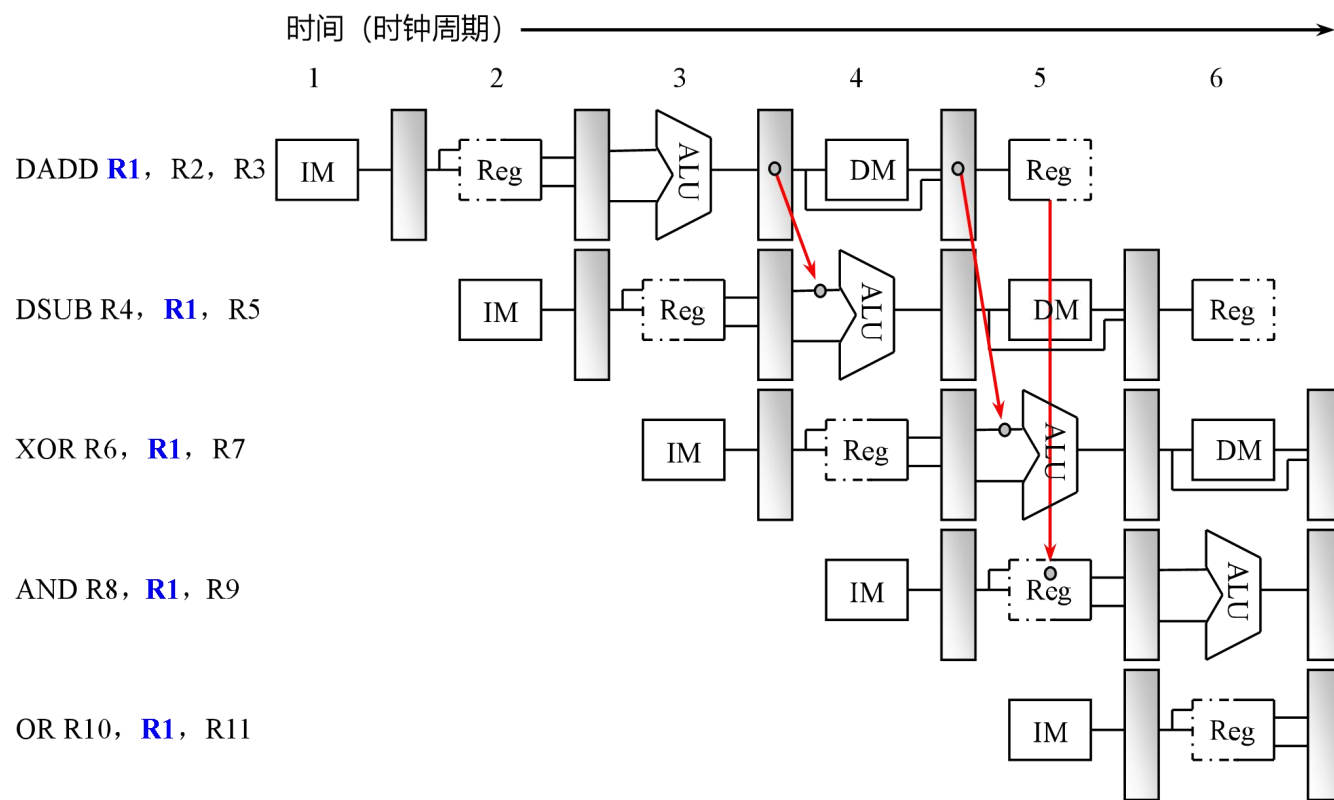
## 3.4.3 流水线冲突

第 104 页

### ■ 定向的实现

- EX段和MEM段之间的流水寄存器中保存的ALU运算结果总是回送到ALU的入口。
- 当定向硬件检测到前一个ALU运算结果写入的寄存器就是当前ALU操作的源寄存器时，那么控制逻辑就选择定向的数据作为ALU的输入，而不采用从通用寄存器组读出的数据。

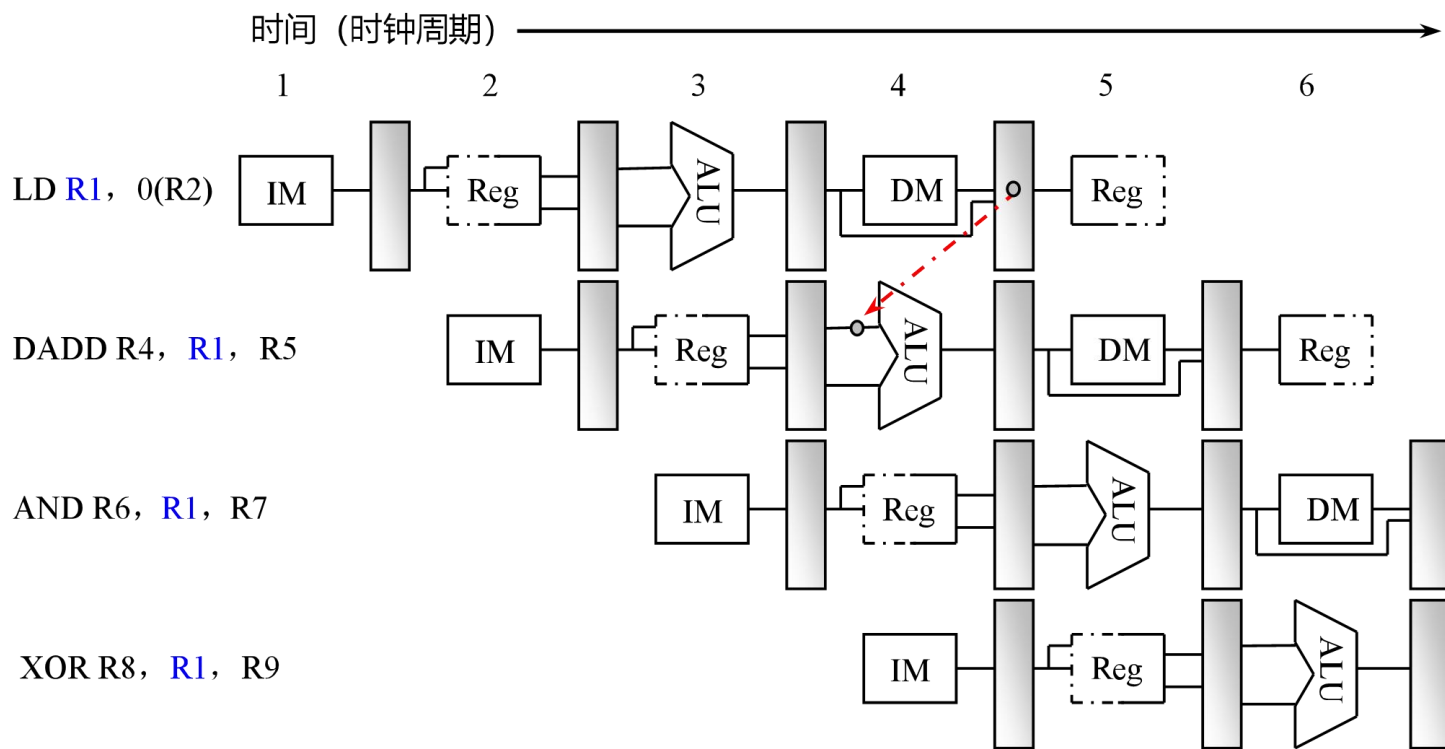
- 结果数据不仅可以从某一功能部件的输出定向到其自身的输入，而且还可以定向到其它功能部件的输入。





## 3.4.3 流水线冲突

第 105 页



无法将LD指令的结果定向到DADD指令

■需要停顿的数据冲突，并不是所有的数据冲突都可以用定向技术来解决。

举例: LD R1, 0 (R2)  
DADD R4, R1, R5  
AND R6, R1, R7  
XOR R8, R1, R9

■增加流水线互锁机制，插入“暂停”。

■作用：检测发现数据冲突，并使流水线停顿，直至冲突消失。

# 3.4.3 流水线冲突

## ■依靠编译器解决数据冲突

让编译器重新组织指令顺序来消除冲突，  
这种技术称为指令调度或流水线调度。

举例：

请为下列表达式生成没有暂停的指令序列：

$A = B + C ;$

$D = E - F ;$

假设载入延迟为1个时钟周期。

| 调度前的代码            | 调度后的代码            |
|-------------------|-------------------|
| LD      Rb, B     | LD      Rb, B     |
| LD      Rc, C     | LD      Rc, C     |
| DADD   Ra, Rb, Rc | LD      Re, E     |
| SD      Ra, A     | DADD   Ra, Rb, Rc |
| LD      Re, E     | LD      Rf, F     |
| LD      Rf, F     | SD      Ra, A     |
| DSUB   Rd, Re, Rf | DSUB   Rd, Re, Rf |
| SD      Rd, D     | SD      Rd, D     |

## 3. 控制冲突

- 执行分支指令的结果有两种：
  - 分支成功：PC值改变为分支转移的目标地址。在条件判定和转移地址计算都完成后，才改变PC值。
  - 不成功或者失败：PC的值保持正常递增，指向顺序的下一条指令。
- 处理分支指令最简单的方法：“冻结”或者“排空”流水线
  - 优点：简单
  - 前述5段流水线中，改变PC值是在MEM段进行的。给流水线带来了3个时钟周期的延迟

|          |    |    |        |       |    |    |    |     |     |     |
|----------|----|----|--------|-------|----|----|----|-----|-----|-----|
| 分支指令     | IF | ID | EX     | MEM   | WB |    |    |     |     |     |
| 分支目标指令   |    | IF | stall  | stall | IF | ID | EX | MEM | WB  |     |
| 分支目标指令+1 |    |    |        |       |    | IF | ID | EX  | MEM | WB  |
| 分支目标指令+2 |    |    | 分支成功情况 |       |    |    | IF | ID  | EX  | MEM |
| 分支目标指令+3 |    |    |        |       |    |    |    | IF  | ID  | EX  |

## 3.4.3 流水线冲突

第 108 页

■把由分支指令引起的延迟称为分支延迟。

■分支指令在目标代码中出现的频度

- 每3 ~ 4条指令就有一条是分支指令。假设：分支指令出现的频度是30%，流水线理想  $CPI = 1$ ，那么流水线的实际  $CPI = 1.9$

■可采取两种措施来减少分支延迟。

- 在流水线中尽早判断出分支转移是否成功；

- **尽早计算出分支目标地址。**

下面的讨论中，我们假设：

|          |    |    |       |     |    |    |     |     |     |     |
|----------|----|----|-------|-----|----|----|-----|-----|-----|-----|
| 分支指令     | IF | ID | EX    | MEM | WB |    |     |     |     |     |
| 分支目标指令   |    | IF | stall | IF  | ID | EX | MEM | WB  |     |     |
| 分支目标指令+1 |    |    |       |     | IF | ID | EX  | MEM | WB  |     |
| 分支目标指令+2 |    |    |       |     |    | IF | ID  | EX  | MEM | WB  |
| 分支目标指令+3 |    |    |       |     |    |    | IF  | ID  | EX  | MEM |

这两步工作被提前到ID段完成，即分支指令是在ID段的末尾执行完成，所带来的分支延迟为一个时钟周期。

### ■ 3种通过软件（编译器）来减少分支延迟的方法

- 预测分支失败
- 预测分支成功
- 延迟分支

# 3.4.3 流水线冲突

➤预测分支失败

- ✓允许分支指令后的指令继续在流水线中流动，就好象什么都没发生似的；
- ✓若确定分支失败，将分支指令看作是一条普通指令，流水线正常流动；
- ✓若确定分支成功，流水线就把在分支指令之后取出的所有指令转化为空操作，并按分支目地重新取指令执行。

DLX流水线对分支的处理过程

|            |    |    |    |     |     |     |     |     |    |
|------------|----|----|----|-----|-----|-----|-----|-----|----|
| 分支指令 i(失败) | IF | ID | EX | MEM | WB  |     |     |     |    |
| 指令 i+1     |    | IF | ID | EX  | MEM | WB  |     |     |    |
| 指令 i+2     |    |    | IF | ID  | EX  | MEM | WB  |     |    |
| 指令 i+3     |    |    |    | IF  | ID  | EX  | MEM | WB  |    |
| 指令 i+4     |    |    |    |     | IF  | ID  | EX  | MEM | WB |

# 3.4.3 流水线冲突

➤预测分支成功

- ✓假设分支转移成功，并从分支目标地址处取指令执行。
- ✓**起作用的前题：**先知道分支目标地址，后知道分支是否成功；
- ✓前述5段流水线中，这种方法没有任何好处。

DLX流水线对分支的处理过程

|            |    |    |    |      |      |      |     |     |    |
|------------|----|----|----|------|------|------|-----|-----|----|
| 分支指令 i(成功) | IF | ID | EX | MEM  | WB   |      |     |     |    |
| 指令 i+1     |    | IF | ID | idle | idle | idle |     |     |    |
| 分支目标 j     |    |    | IF | ID   | EX   | MEM  | WB  |     |    |
| 分支目标 j+1   |    |    |    | IF   | ID   | EX   | MEM | WB  |    |
| 分支目标 j+2   |    |    |    |      | IF   | ID   | EX  | MEM | WB |

# 3.4.3 流水线冲突

## ■ 延迟分支

- 主要思想：从逻辑上“延长”分支指令的执行时间。把延迟分支看成是由原来的分支指令和若干个延迟槽构成，不管分支是否成功，都要按顺序执行延迟槽中的指令。

|      |           |    |    |    |     |     |     |     |     |    |
|------|-----------|----|----|----|-----|-----|-----|-----|-----|----|
| 分支失败 | 分支指令i     | IF | ID | EX | MEM | WB  |     |     |     |    |
|      | 延迟槽指令 i+1 |    | IF | ID | EX  | MEM | WB  |     |     |    |
|      | 指令 i+2    |    |    | IF | ID  | EX  | MEM | WB  |     |    |
|      | 指令 i+3    |    |    |    | IF  | ID  | EX  | MEM | WB  |    |
|      | 指令 i+4    |    |    |    |     | IF  | ID  | EX  | MEM | WB |

具有一个分支延迟槽的流水线的执行过程

|      |           |    |    |    |     |     |     |     |     |    |
|------|-----------|----|----|----|-----|-----|-----|-----|-----|----|
| 分支成功 | 分支指令i     | IF | ID | EX | MEM | WB  |     |     |     |    |
|      | 延迟槽指令 i+1 |    | IF | ID | EX  | MEM | WB  |     |     |    |
|      | 分支目标指令j   |    |    | IF | ID  | EX  | MEM | WB  |     |    |
|      | 分支目标指令j+1 |    |    |    | IF  | ID  | EX  | MEM | WB  |    |
|      | 分支目标指令j+2 |    |    |    |     | IF  | ID  | EX  | MEM | WB |

分支延迟槽中的指令“掩盖”了流水线原来必需插入的暂停周期。



### ■ 3种通过软件（编译器）来减少分支延迟的方法

#### ➤ 共同点：

- ✓ 对分支的处理方法在程序的执行过程中始终是不变的，是静态的。
- ✓ 要么总是预测分支成功，要么总是预测分支失败。

#### ➤ 要保证：分支结果出来之前不能改变处理机的状态，以便一旦猜错时，处理机能够回退到原先的状态。

## 3.4.3 流水线冲突

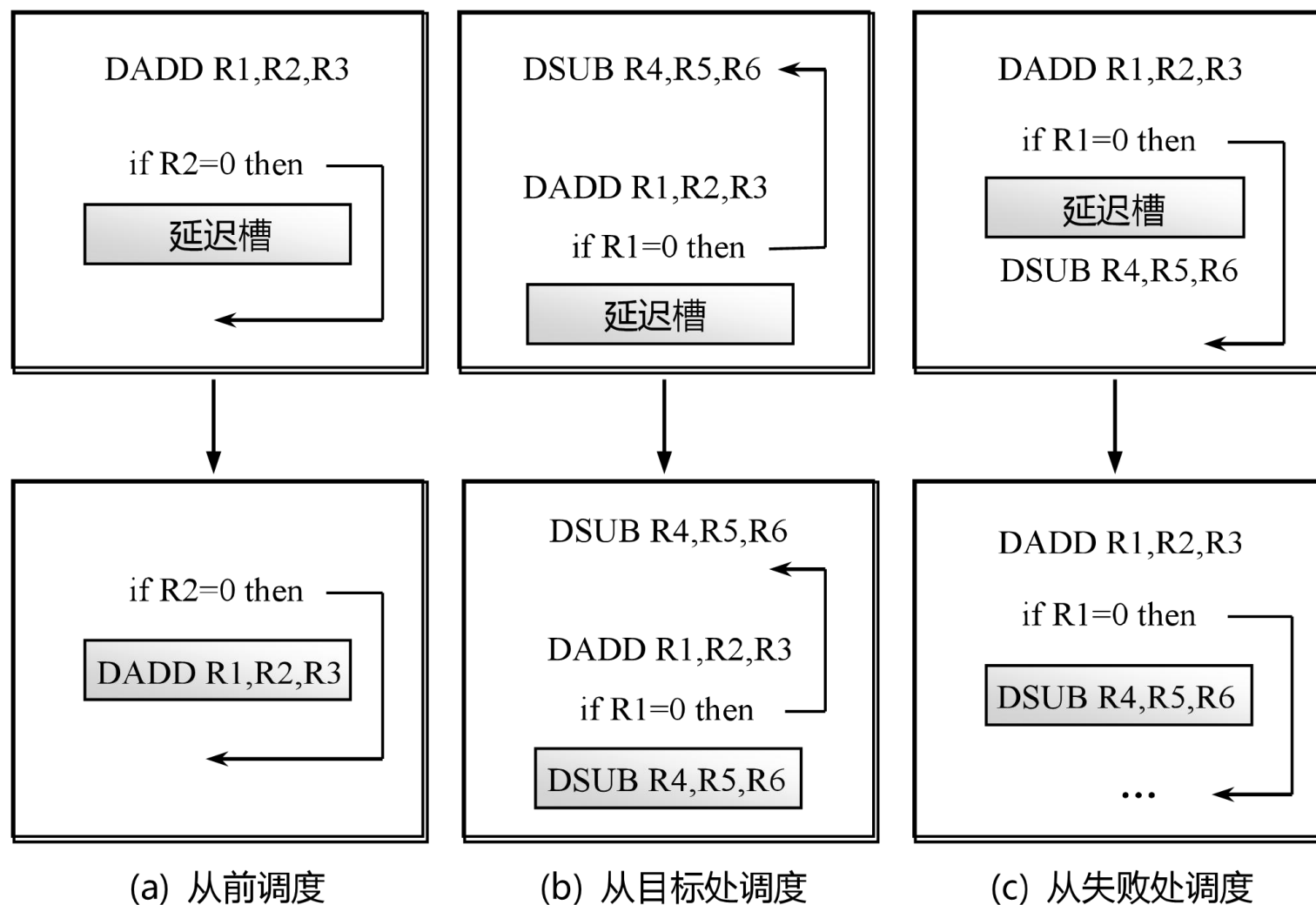
第 114 页

### ■分支延迟指令的调度

➤任务：在延迟槽中放入有用的指令，由编译器完成。能否带来好处取决于编译器能否把有用的指令调度到延迟槽中。

➤三种调度方法：

- ✓从前调度
- ✓从目标处调度
- ✓从失败处调度



## 3.4.3 流水线冲突

第 115 页

### 三种方法的要求及效果

| 调度策略   | 对调度的要求                              | 什么情况下起作用?                  |
|--------|-------------------------------------|----------------------------|
| 从前调度   | 被调度的指令必须与分支无关                       | 任何情况                       |
| 从目标处调度 | 必须保证在分支失败时执行被调度的指令不会导致错误。有可能需要复制指令。 | 分支成功时(但由于复制指令, 有可能会增大程序空间) |
| 从失败处调度 | 必须保证在分支成功时执行被调度的指令不会导致错误。           | 分支失败时                      |

#### ■ 分支延迟受到两个方面的限制:

- 可以被放入延迟槽中的指令要满足一定的条件;
- 编译器预测分支转移方向的能力。

# 3.4.3 流水线冲突

■ 进一步改进：分支取消机制

- 当分支的实际执行方向和事先所预测的一样时，执行分支延迟槽中的指令，否则就将分支延迟槽中的指令转化成一个空操作。“预测成功-取消”分支的执行过程

|      |           |    |    |      |      |      |      |     |     |    |
|------|-----------|----|----|------|------|------|------|-----|-----|----|
| 分支失败 | 分支指令i     | IF | ID | EX   | MEM  | WB   |      |     |     |    |
|      | 延迟槽指令 i+1 |    | IF | idle | idle | idle | idle |     |     |    |
|      | 指令 i+2    |    |    | IF   | ID   | EX   | MEM  | WB  |     |    |
|      | 指令 i+3    |    |    |      | IF   | ID   | EX   | MEM | WB  |    |
|      | 指令 i+4    |    |    |      |      | IF   | ID   | EX  | MEM | WB |
| 分支成功 | 分支指令i     | IF | ID | EX   | MEM  | WB   |      |     |     |    |
|      | 延迟槽指令 i+1 |    | IF | ID   | EX   | MEM  | WB   |     |     |    |
|      | 分支目标指令j   |    |    | IF   | ID   | EX   | MEM  | WB  |     |    |
|      | 分支目标指令j+1 |    |    |      | IF   | ID   | EX   | MEM | WB  |    |
|      | 分支目标指令j+2 |    |    |      |      | IF   | ID   | EX  | MEM | WB |

预测分支成功的情况下，分支取消机制的执行情况

## 3.5 流水线RISC-V的实现

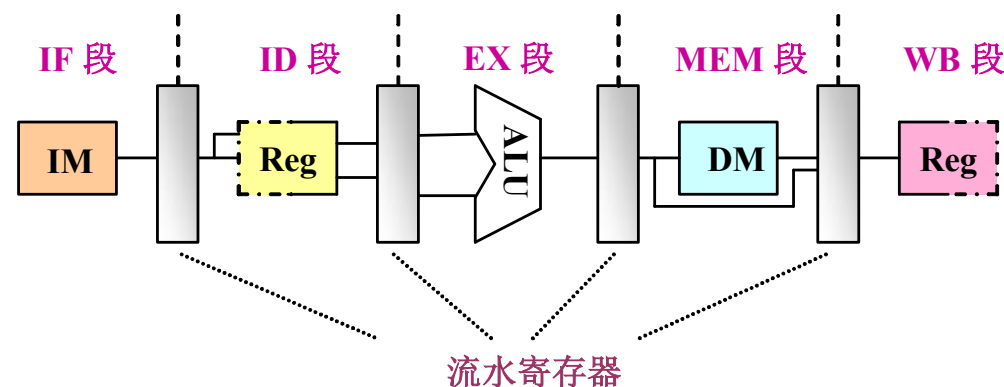
## 3.5.1 流水线的实现

第 118 页

RISC-V是一个基于精简指令集（RISC）原则的**开源**指令集架构（ISA）

| 基本指令集           |                                  | 扩展指令集      |                               |
|-----------------|----------------------------------|------------|-------------------------------|
| RV32I<br>(47条)  | 32 位地址空间与整数指令，支持 32 个通用整数寄存器     | M<br>(8条)  | 整数乘法与除法指令                     |
| RV32E<br>(47条)  | RV32I 的子集，仅支持 16 个通用整数寄存器        | A<br>(11条) | 存储器原子（Atomic）操作指令（load/store） |
| RV64I<br>(59条)  | 64 位地址空间与整数指令及一部分 32 位的整数指令      | F<br>(26条) | 单精度（32 比特）浮点指令                |
| RV128I<br>(71条) | 128 位地址空间与整数指令及一部分 64 位和 32 位的指令 | D<br>(26条) | 双精度（64 比特）浮点指令，必须支持 F 扩展指令    |
|                 |                                  | C<br>(46)  | 压缩指令，指令长度为 16 位               |

通过以上的模块化指令集，能够选择不同的组合来满足不同的应用。例如，追求小面积、低功耗的嵌入式场景可以选择使用 RV32EC 架构；而大型的 64 位架构则可以选择 RV64G。



## 3.5.1 流水线的实现

第 119 页

### 整数计算

add rd, rs1, rs2

$$x[rd] = x[rs1] + x[rs2]$$

加 (add), R类型指令, 属RV32I和RV64指令集

功能: 把寄存器  $x[rs2]$  加到寄存器  $x[rs1]$  上, 结果写入  $x[rd]$ 。

|         |       |       |       |       |         |   |
|---------|-------|-------|-------|-------|---------|---|
| 31      | 25 24 | 20 19 | 15 14 | 12 11 | 7 6     | 0 |
| 0000000 | rs2   | rs1   | 000   | Rd    | 0110011 |   |

### 控制转移

beq rs1, rs2, offset

if ( $rs1 == rs2$ )  $pc += sext(offset)$

相等时分支 (*Branch if Equal*), B类型指令, 属RV32I和RV64I指令集

功能: 若寄存器  $x[rs1]$  和寄存器  $x[rs2]$  的值相等, 把  $pc$  的值设为当前值加上符号位扩展的偏移  $offset$ 。

|                 |       |       |       |                |         |   |
|-----------------|-------|-------|-------|----------------|---------|---|
| 31              | 25 24 | 20 19 | 15 14 | 12 11          | 7 6     | 0 |
| offset[12 10:5] | rs2   | rs1   | 000   | offset[4:1 11] | 1100011 |   |

## 3.5.1 流水线的实现

第 120 页

### 存取操作

lw rd, offset(rs1)  $x[rd] = \text{sext}(M[x[rs1] + \text{sext}(\text{offset})][31:0])$

字加载(Load Word), I类型指令, 属RV32I和RV64I指令集  
功能: 从地址 $x[rs1] + \text{sign-extend}(\text{offset})$ 读取四个字节, 写入 $x[rd]$ 。对于RV64I, 结果要进行符号位扩展。

|              |       |       |       |     |         |
|--------------|-------|-------|-------|-----|---------|
| 31           | 20 19 | 15 14 | 12 11 | 7 6 | 0       |
| offset[11:0] |       | rs1   | 010   | rd  | 0000011 |

### 混杂操作

csrrs rd, csr, rs1  $t = \text{CSRs}[csr]; \text{CSRs}[csr] = t \mid x[rs1]; x[rd] = t$

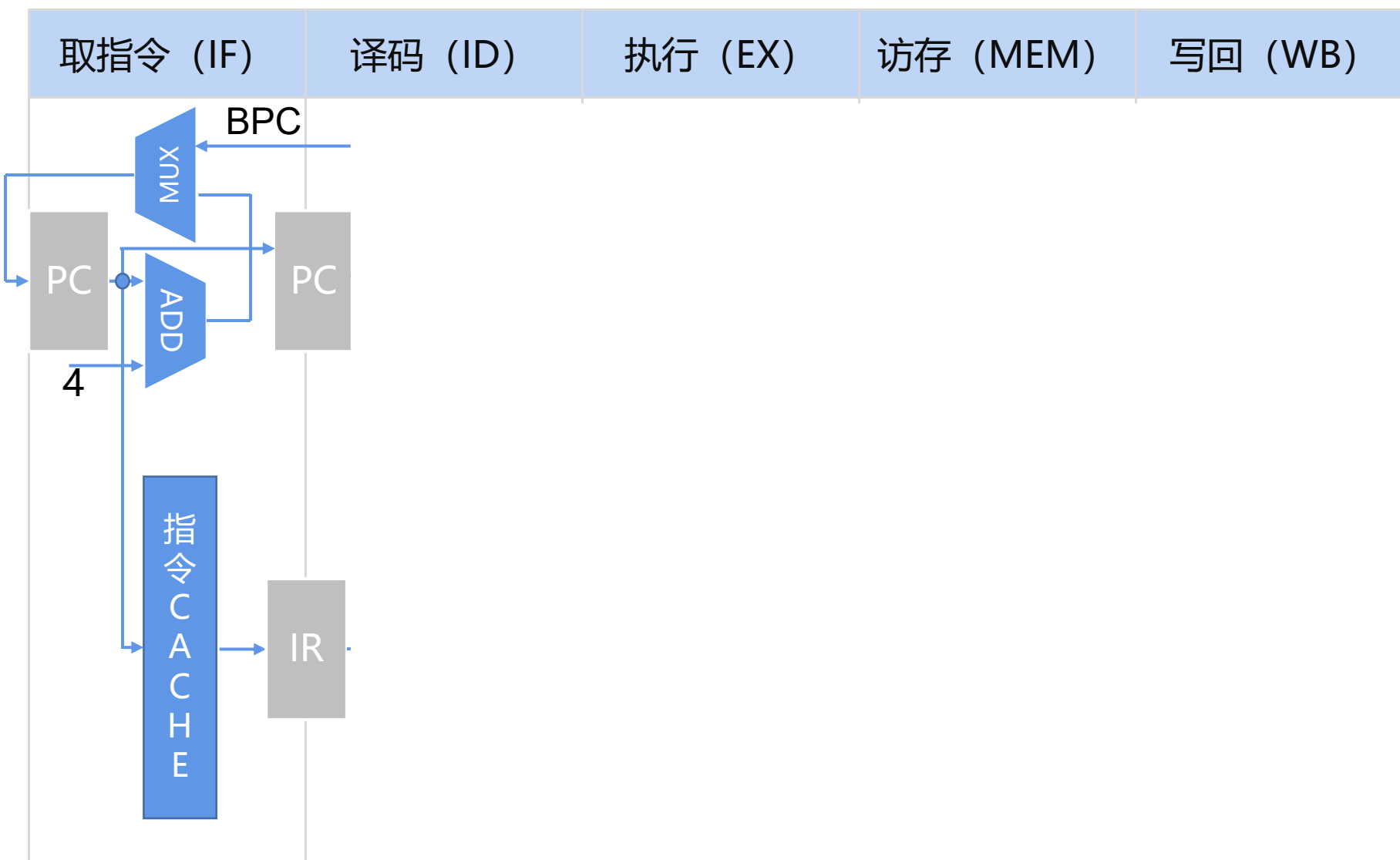
读后置位控制状态寄存器(*Control and Status Register Read and Set*), I类型指令, 属RV32I和RV64I指令集  
功能: 记控制状态寄存器 $csr$ 中的值为 $t$ 。把 $t$ 和寄存器 $x[rs1]$ 按位或的结果写入 $csr$ , 再把 $t$ 写入 $x[rd]$ 。

|     |       |       |       |     |         |
|-----|-------|-------|-------|-----|---------|
| 31  | 20 19 | 15 14 | 12 11 | 7 6 | 0       |
| csr |       | rs1   | 010   | rd  | 1110011 |



## 3.5.1 流水线的实现

第 121 页



取指IF段:

if(branch)  $PC\_IF \leftarrow BPC$

else  $PC\_IF \leftarrow PC\_IF + 4$

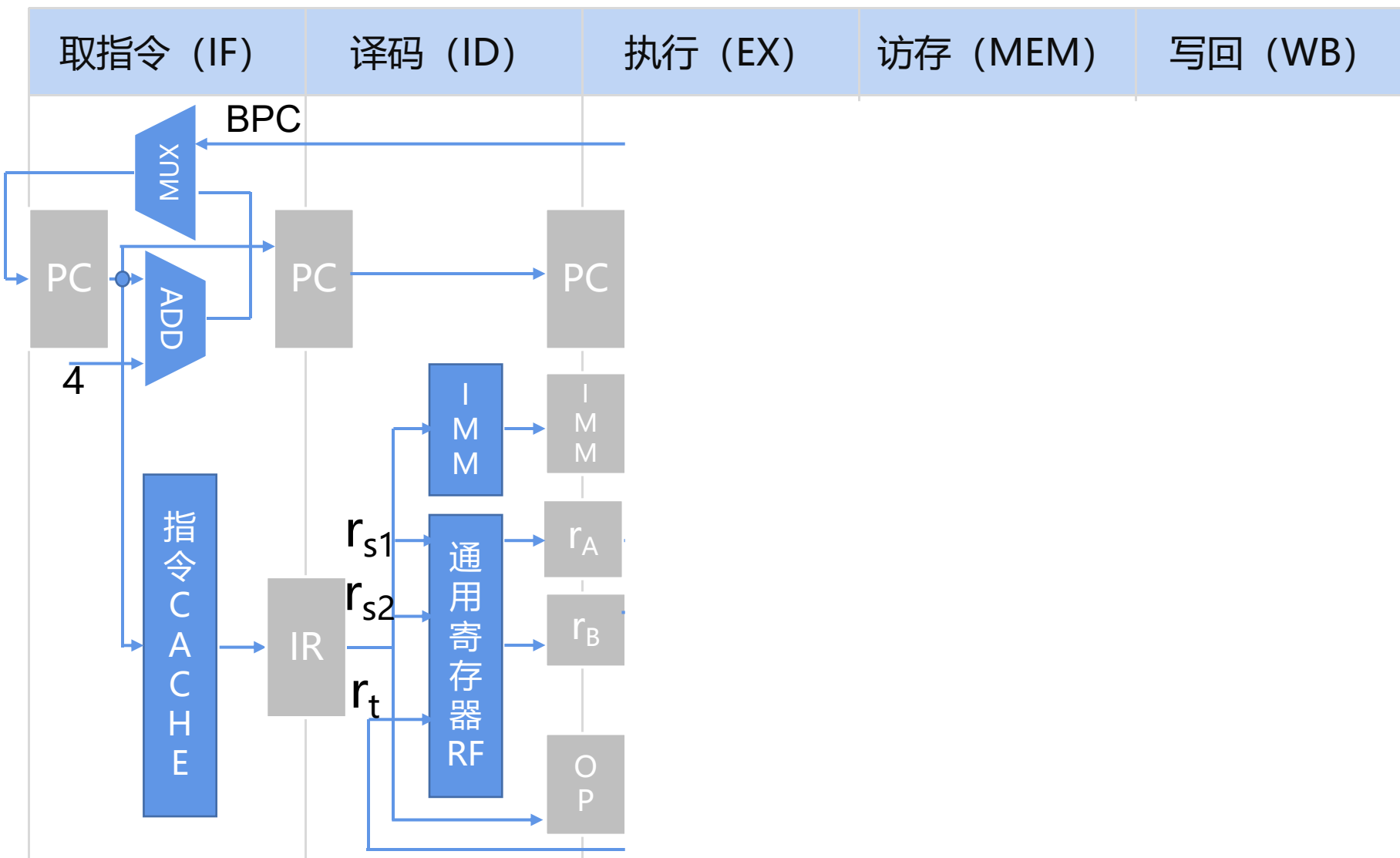
$PC\_ID \leftarrow PC\_IF$

$IR \leftarrow Mem[PC]$

- 取指令：根据PC地址从存储器取指令并装入指令寄存器（IR），IR中保存下一个时钟周期需要的指令
- $PC = PC + 4$ ，在下一个时钟周期更新PC\_IF。

## 3.5.1 流水线的实现

第 122 页



译码ID段:

$A \leftarrow \text{Regs}[rs1]$

$B \leftarrow \text{Regs}[rs2]$

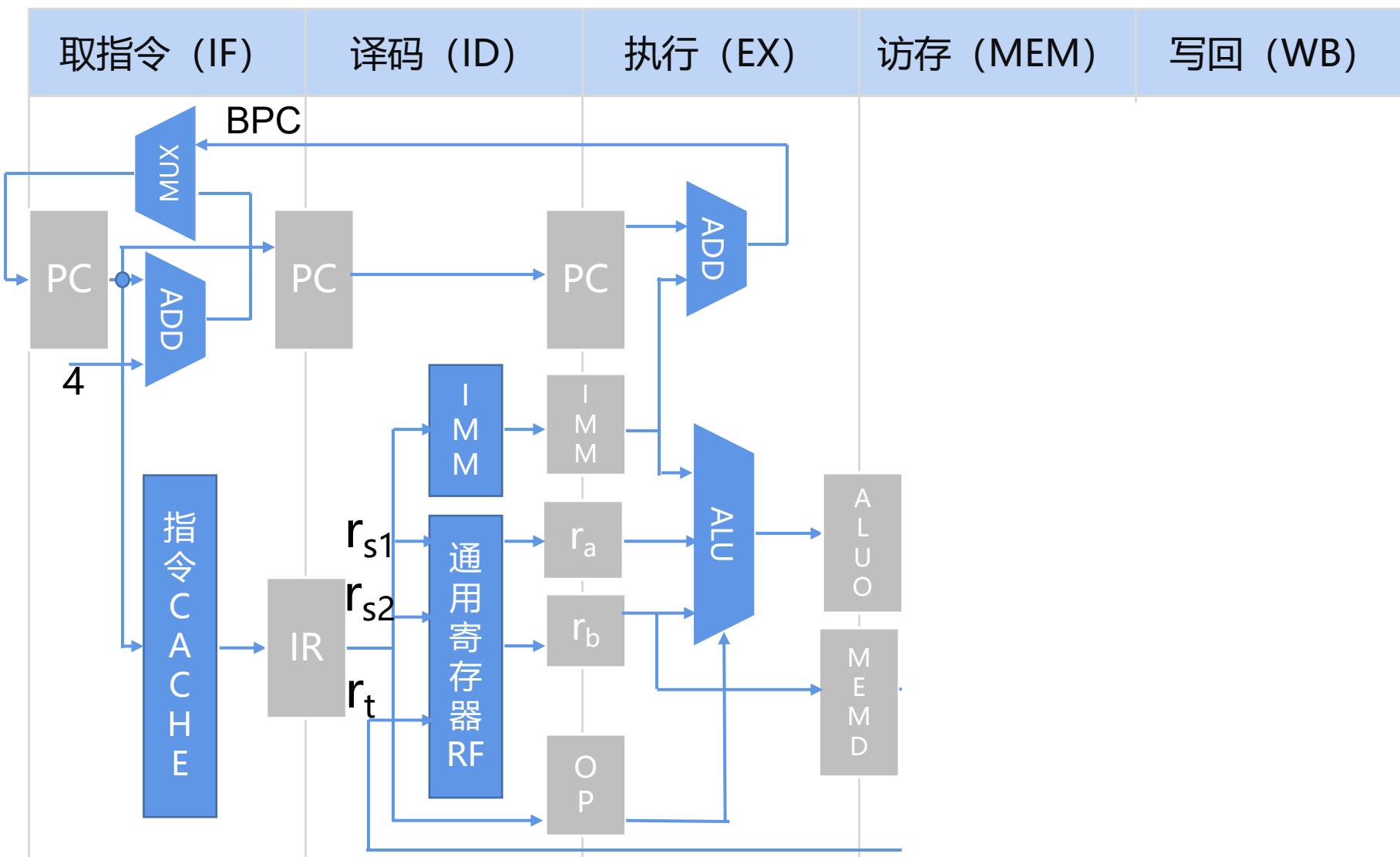
$\text{IMM} \leftarrow \text{立即数扩展}$

$\text{PC\_EX} \leftarrow \text{PC\_ID}$

- 取操作数：分析指令并访问寄存器堆；
- 立即数扩展：根据指令功能需求，实现立即数的逻辑/算术扩展
- 分支指令地址：PC地址传递

## 3.5.1 流水线的实现

第 123 页



执行EX段:

■ 寄存器-寄存器ALU操作

$ALUO \leftarrow A \text{ func } B$

■ 寄存器-立即数ALU操作

$ALUO \leftarrow A \text{ op } Imm$

■ 访问存储器地址计算

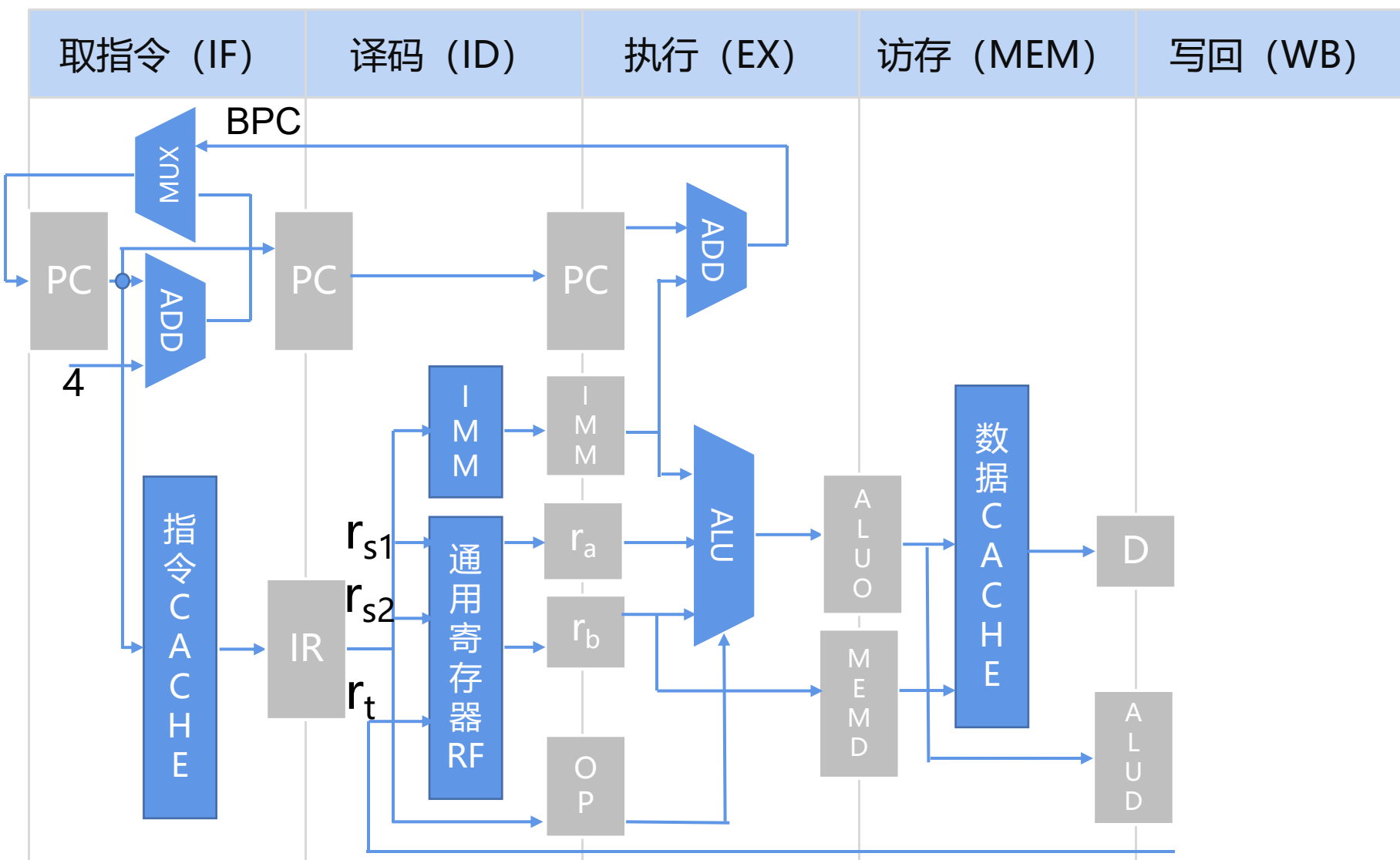
$ALUO \leftarrow A + Imm$

■ 分支地址计算

$BPC \leftarrow PC_{EX} + Imm$

## 3.5.1 流水线的实现

第 124 页



访存MEM段:

$D \leftarrow \text{Mem}[\text{ALUO}]$

$\text{MEM}[\text{ALUO}] \leftarrow \text{MEMD}$

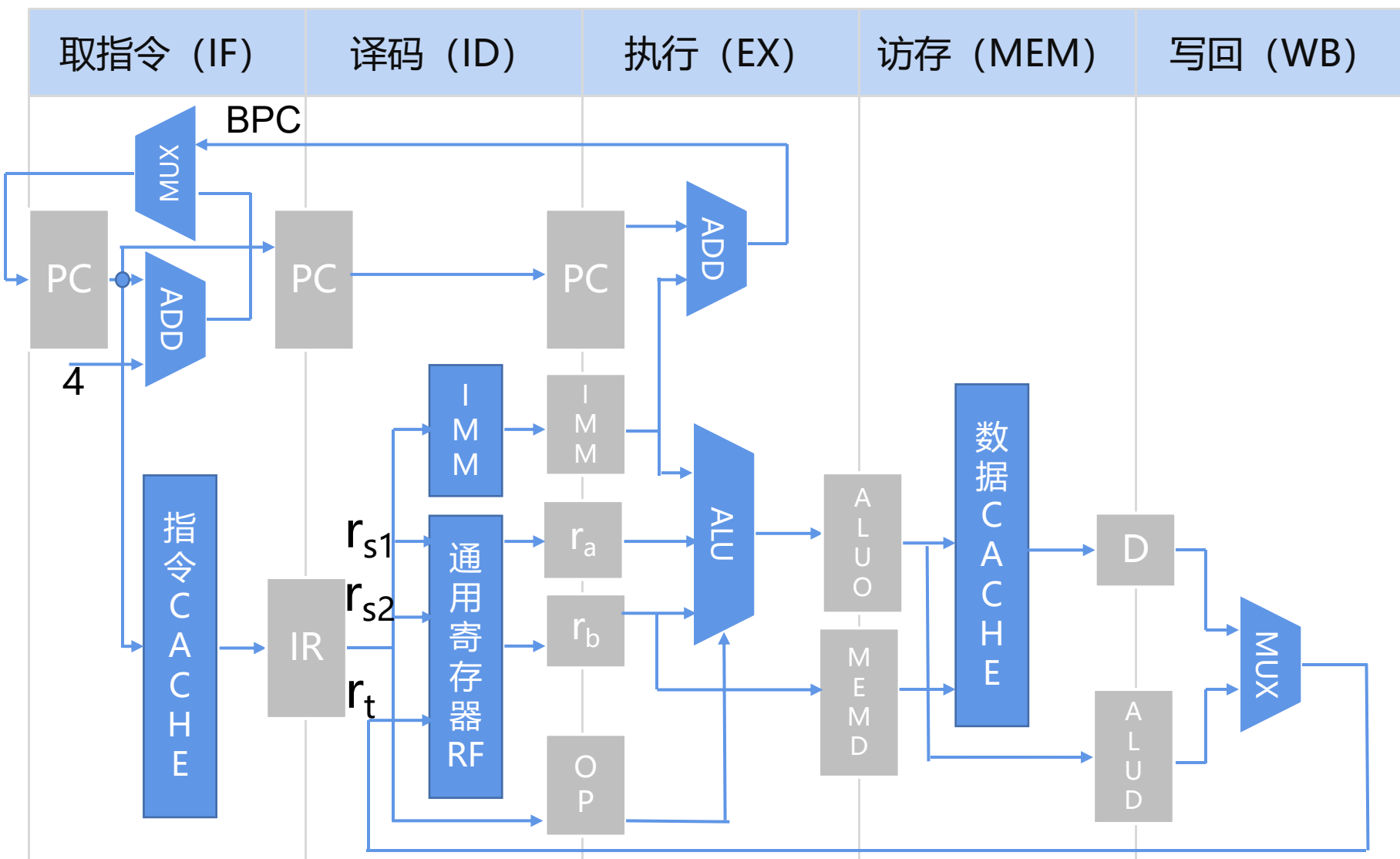
$\text{ALUD} \leftarrow \text{ALUO}$

■ Load指令: 从存储器中  
读出数据并装入LMD  
(装入存储器数据) 寄存器

■ Store指令: 将寄存器B  
中的数据写入存储器

## 3.5.1 流水线的实现

第 125 页



写回WB段:

■ Load指令

$\text{Regs}[r_d] \leftarrow D$

■ 寄存器-寄存器ALU

$\text{Regs}[r_d] \leftarrow \text{ALUD}$

■ 寄存器-立即数ALU

$\text{Regs}[r_d] \leftarrow \text{ALUD}$

## 3.5.2 流水线最佳段数的选择

第 126 页

### ■流水线瓶颈问题

- ◆ 理想情况下，流水线在工作时，其中的任务是同步地每一个时钟周期往前流动一段。
- ◆ 当流水线各段不均匀时，**机器的时钟周期取决于瓶颈段的延迟时间。**
- ◆ 在设计流水线时，要尽可能使各段时间相等。

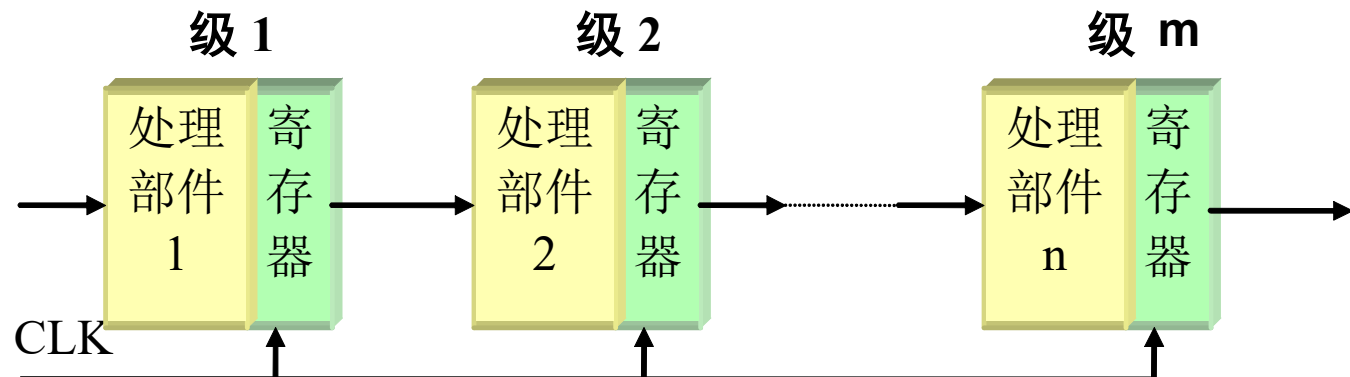
### ■设计需求：流水线各段时间相等

- ◆ 当 $n \gg m$ 时， $S \approx m$ （ $m$ 为流水线的段数或深度）
- ◆ 增加流水线的深度（段数）可以提高流水线的性能。

## 3.5.2 流水线最佳段数的选择

第 127 页

■ 流水线深度问题：理想情况下，流水线深度越大，性能越高。



流水线的总价格为： $C = a + m \cdot b$ 。  
其中， $a$ 为所有流水段的总价格， $b$ 为每个锁存器的价格。

在同等速度的有 $k$ 段（级）流水线的机器上执行一个任务需要的时间为： $t/m + d$ ，其中 $d$ 为寄存器（级间寄存器）的延迟时间）

■ 流水寄存器的延迟时间=建立时间+传输延迟

- **建立时间**：在触发写操作的时钟信号到达之前，寄存器输入必须保持稳定的时间
- **传输延迟**：时钟信号到达后到寄存器输出可用的时间

## 3.5.2 流水线最佳段数的选择

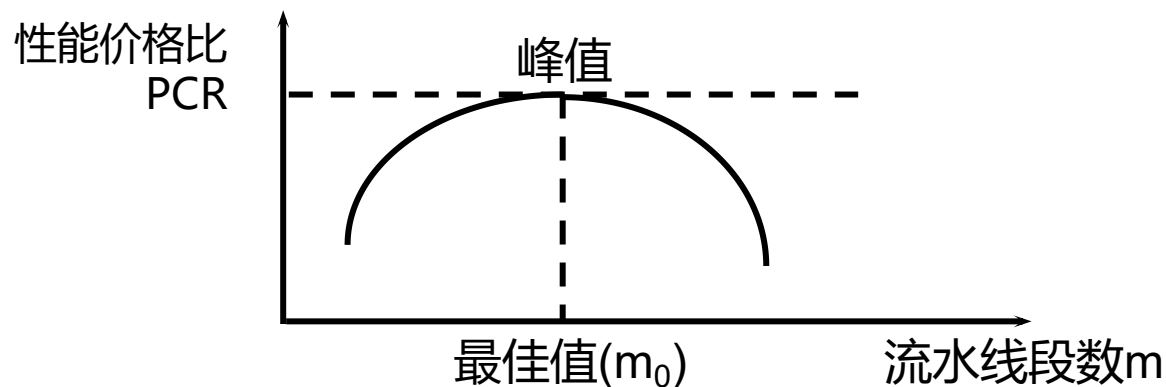
第 128 页

$$\text{性价比 (PCR)} = \frac{\text{最大吞吐量}}{\text{流水线价格}} = \frac{1/(t/m + d)}{a + m \cdot b}$$

- $t$ : 任务总时间
- $d$ : 锁存器时间
- $m$ : 功能段数
- $a$ : 所有功能段价格
- $b$ : 锁存器价格

对自变量 $m$ 求导, 求PCR的最大值, 得到最佳段数为:

$$m_0 = \sqrt{\frac{t \cdot a}{d \cdot b}}$$



$$\text{PCR} = \frac{P_{\max}}{C} = \frac{1}{t/m + d} \cdot \frac{1}{a + mb}$$



## 3.5.2 流水线最佳段数的选择

第 129 页

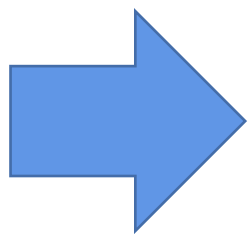
推导过程:

$$\left(\frac{1}{t/m + d} \cdot \frac{1}{a + bm}\right)' = 0 \quad \left(\frac{m}{t + dm} \cdot \frac{1}{a + bm}\right)' = 0$$

$$(t + dm)(a + bm) - dm(a + bm) - bm(t + dm) = 0$$

$$at + adm + btm + bdm^2 - adm - bdm^2 - btm - bdm^2 = 0$$

$$at - bdm^2 = 0$$



$$m = \sqrt{\frac{a \cdot t}{b \cdot d}}$$

一般规律：RISC处理机中的流水线段数大多在2段至10段之间，较少有超过15段的流水线。

**如何进一步提升性能？**

1. 流水线的基本概念：工业流水线，时空图，连接图，部件级、处理器级和系统级流水线，静态与动态流水线，线性与非线性流水线，顺序与乱序流水线，标量与向量流水线
2. 流水线的性能指标：吞吐率、加速比、效率、性能体制
3. 非线性流水线的调度：单功能非线性流水线的调度，多功能非线性流水线的调度
4. 流水线的相关与冲突：数据相关、名相关、控制相关、结构冲突、数据冲突、控制冲突
5. 流水线的实现：RISC-V处理器，最佳段数的选择

# 选择题

第 131 页

1、非线性流水线的特征 ( **B** )

A. 一次运算中使用流水线中的多个段

B. 动态流水线是多功能流水线

C. 流水线中某些功能段在各次运算中的作用不同

D. 流水线的各功能段在不同运算中可以有不同的连接

2、以下是某非线性流水线的调度方案：[(2,7); (2,2,7); (3,4); (4); (3,4,7); (4,7); (4,3); (5); (7)]。其中，平均延迟最小的等间隔调度方案是 ( **A** )

A. (4)                      B. (5)                      C. (3,4)                      D. (4,3)

3、与线性流水线最大吞吐率有关的是 ( **C** )

A. 各个功能段的执行时间

B. 最快的那一段的执行时间

C. 最慢的那一段的执行时间

D. 最后功能段的执行时间

# 应用题

1、在一台单流水线多操作部件的处理机上执行下面的程序，取指令、指令译码各需一个时钟周期，MOVE、ADD和MUL操作各需要2个、3个和4个时钟周期。每个操作都在第一个时钟周期从通用寄存器中读操作数，在最后一个时钟周期把运算结果写到通用寄存器中。

```
K:      MOVE R1, R0      ; R1← (R0)
K+1:    MUL R0, R2, R1    ; R0← (R2)*(R1)
K+2:    ADD R0, R3, R2    ; R0← (R3)+(R2)
```

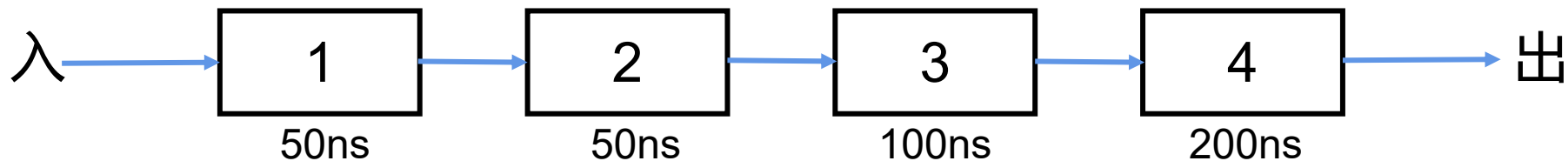
画出指令执行的流水线时空图，并计算执行完3条指令共需要多少个时钟周期。

解

| 指令/时钟 | 1  | 2  | 3  | 4     | 5  | 6  | 7  | 8     | 9  |
|-------|----|----|----|-------|----|----|----|-------|----|
| K     | IF | ID | EX | EX    |    |    |    |       |    |
| K+1   |    | IF | ID | stall | EX | EX | EX | EX    |    |
| K+2   |    |    | IF | stall | ID | EX | EX | stall | EX |

计算执行完  
三条指令共  
使用了9个  
时钟周期

2、有一指令流水线如下图所示。



(1) 求连续输入10条指令，该流水线的实际吞吐率和效率。

解：

$$T_{pipeline} = \sum_{i=1}^m \Delta t_i + (n - 1)\Delta t_{max} = (50 + 50 + 100 + 200) + 9 \times 200 = 2200ns$$

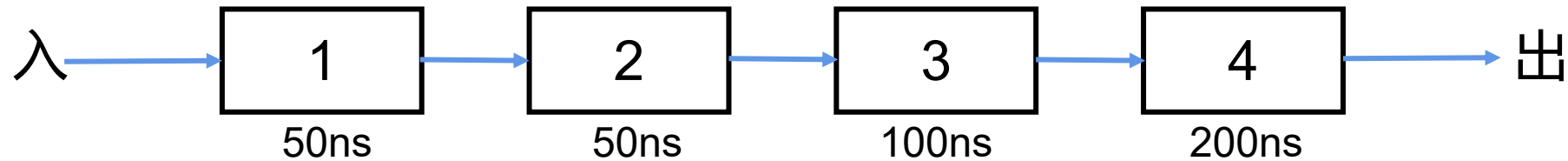
$$TP = \frac{n}{T_{pipeline}} = \frac{10}{2200} = \frac{1}{220} (ns^{-1})$$

$$E = TP \cdot \frac{\sum_{i=1}^m \Delta t_i}{m} = TP \cdot \frac{400}{4} = \frac{5}{11} \approx 45.45\%$$



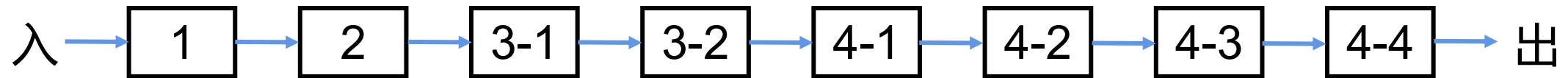
$$E = \frac{n \cdot \sum_{i=1}^m \Delta t_i}{m \cdot [\sum_{i=1}^m \Delta t_i + (n - 1) \cdot \max(\Delta t_1, \Delta t_2, \dots, \Delta t_m)]}$$

2、有一指令流水线如下图所示。



(2) 该流水线的“瓶颈”在哪一段？请采取两种不同的措施消除此“瓶颈”。对于你所给出的两种新的流水线，连续输入10条指令时，其实际吞吐率和效率各是多少？

解：1) 变成8级流水线（细分）如下图所示。



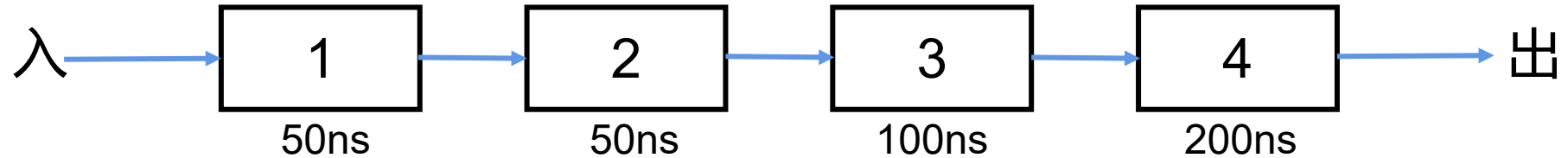
$$T_{pipeline} = \sum_{i=1}^m \Delta t_i + (n - 1)\Delta t_{max} = 50 \times 8 + 9 \times 50 = 850ns$$

$$TP = \frac{n}{T_{pipeline}} = \frac{10}{850} = \frac{1}{85} (ns^{-1})$$

$$E = TP \cdot \frac{\sum_{i=1}^m \Delta t_i}{m} = TP \cdot \frac{400}{8} = \frac{10}{17} \approx 58.82\%$$



2、有一指令流水线如下图所示。



(2) 该流水线的“瓶颈”在哪一段？请采取两种不同的措施消除此“瓶颈”。对于你所给出的两种新的流水线，连续输入10条指令时，其实际吞吐率和效率各是多少？

解：1) 重复设置部件3和4，如下图所示。

$$TP = \frac{n}{T_{pipeline}} = \frac{10}{850} = \frac{1}{85} (ns^{-1}) \quad E = \frac{400 \times 10}{850 \times 8} = \frac{10}{17} \approx 58.82\%$$



3、有一条流水线由4段组成，其中每当流经第3段时，总要在该段循环一次，然后才能流到第4段。如果每段经过一次所需要的时间都是 $\Delta t$ ，问。

(1) 当在流水线的输入端连续地每 $\Delta t$ 时间输入任务时，该流水线会发生什么情况。

|       |       |       |       |       |       |       |       |       |       |       |       |
|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|
| 第1个任务 | $S_1$ | $S_2$ | $S_3$ | $S_3$ | $S_4$ |       |       |       |       |       |       |
| 第2个任务 |       | $S_1$ | $S_2$ | stall | $S_3$ | $S_3$ | $S_4$ |       |       |       |       |
| 第3个任务 |       |       | $S_1$ | stall | $S_2$ | stall | $S_3$ | $S_3$ | $S_4$ |       |       |
| 第4个任务 |       |       |       |       | $S_1$ | stall | $S_2$ | stall | $S_3$ | $S_3$ | $S_4$ |

该流水线会发生流水线阻塞情况。

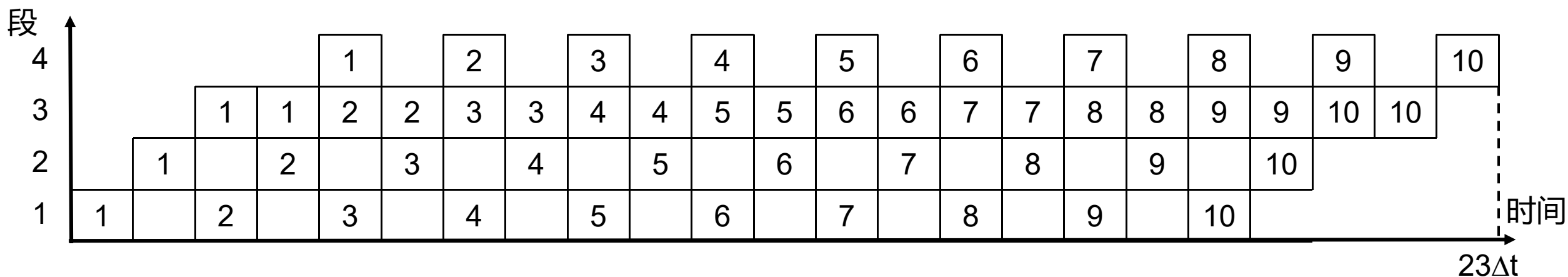
# 应用题

第 138 页

3、有一条流水线由4段组成，其中每当流经第3段时，总要在该段循环一次，然后才能流到第4段。如果每段经过一次所需要的时间都是 $\Delta t$ ，问。

(2) 此流水线的最大吞吐率为多少？如果每 $2\Delta t$ 输入一个任务，连续处理10个任务时的实际吞吐率和效率是多少？

$$\text{最大吞吐率: } TP_{max} = \frac{1}{2\Delta t}$$



$$T_{pipeline} = 23\Delta t$$

$$TP_{max} = \frac{n}{T_{pipeline}} = \frac{10}{23\Delta t}$$

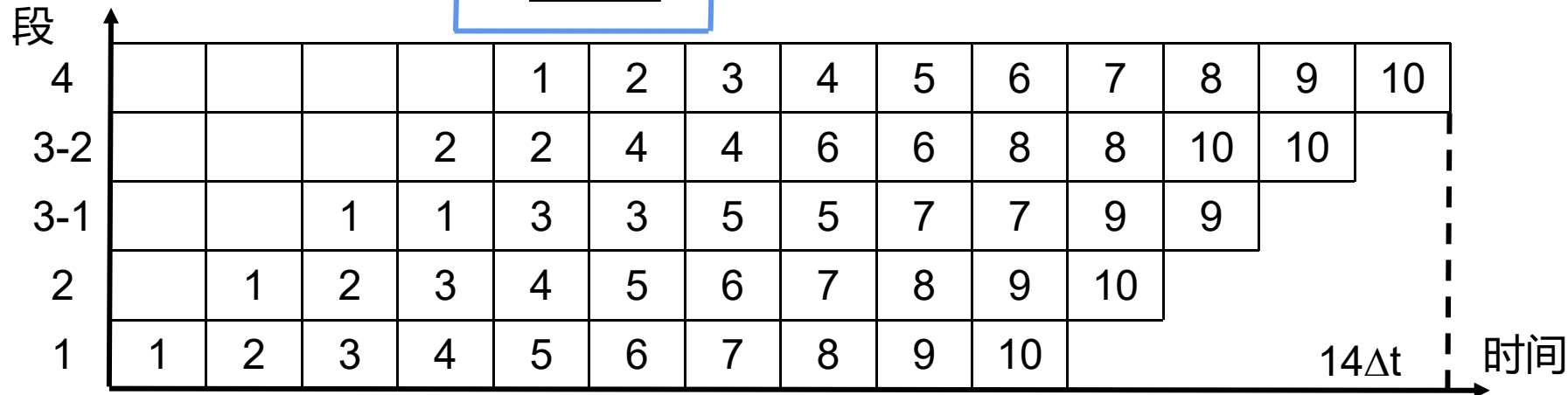
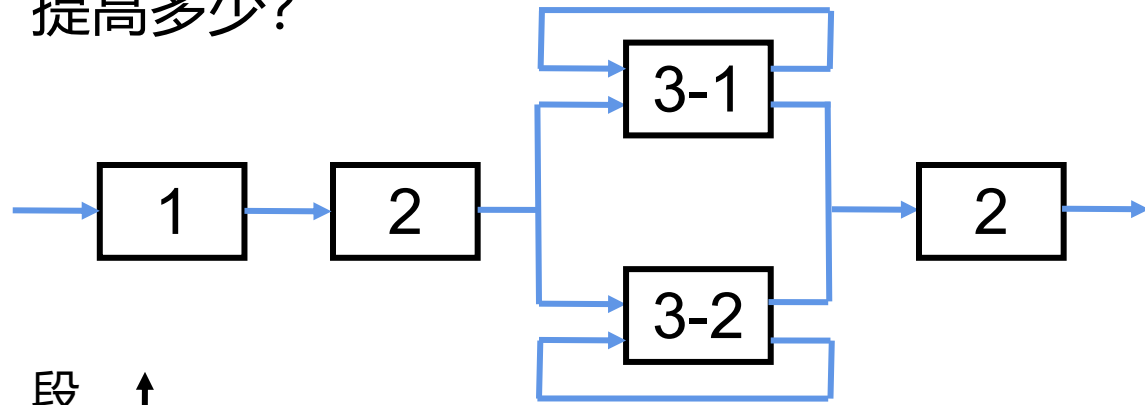
$$E = TP \cdot \frac{5\Delta t}{4} = \frac{50}{92} \approx 54.35\%$$

# 应用题

第 139 页

3、有一条流水线由4段组成，其中每当流经第3段时，总要在该段循环一次，然后才能流到第4段。如果每段经过一次所需要的时间都是 $\Delta t$ ，问。

(3) 当每段时间不变时，如何提高该流水线的吞吐率？仍连续处理10个任务时，其吞吐率提高多少？



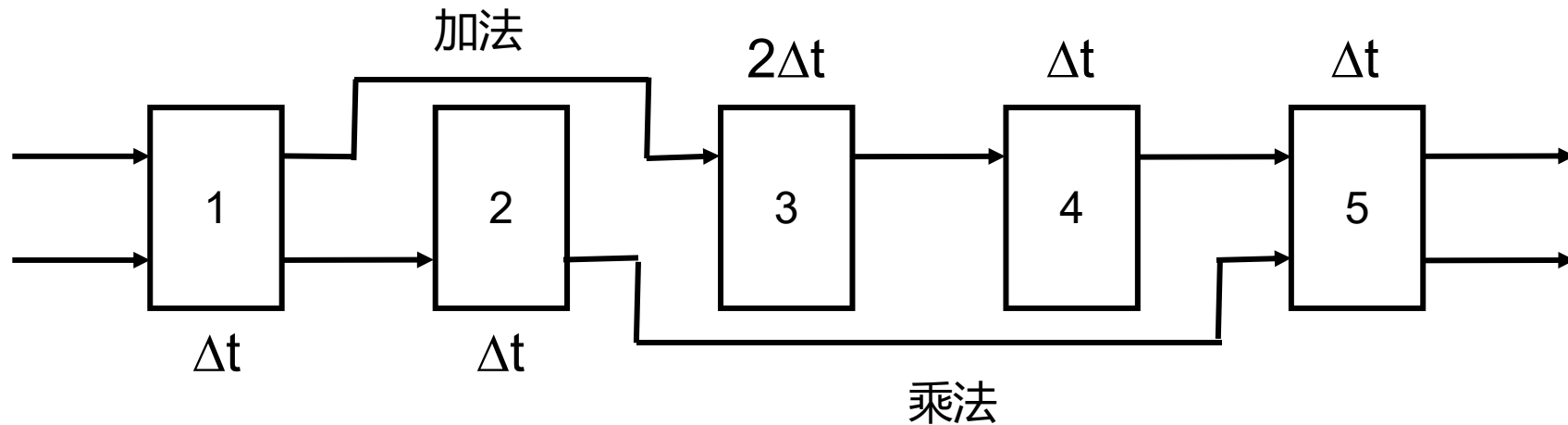
$$TP = \frac{n}{T_{pipeline}} = \frac{10}{14\Delta t} = \frac{5}{7\Delta t}$$

$$E = \frac{\frac{5}{7\Delta t}}{\frac{10}{23\Delta t}} = 1.64$$

## 应用题

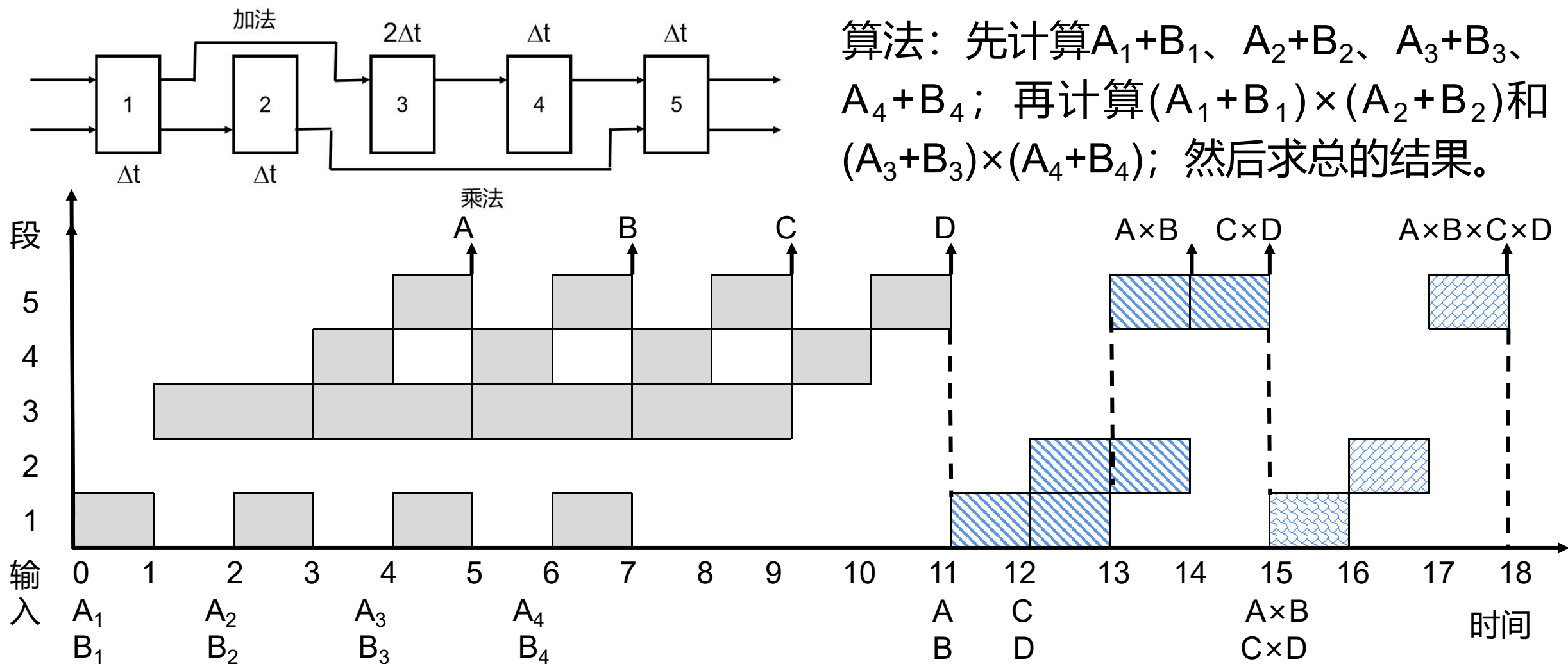
第 140 页

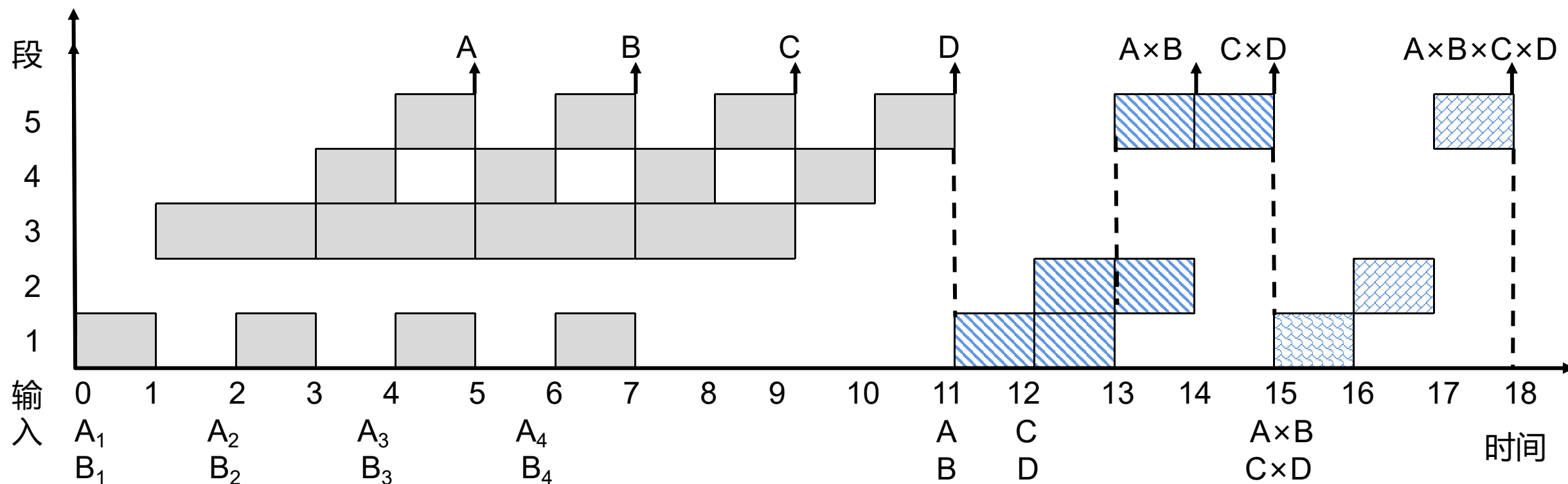
4、有一条静态多功能流水线由5段组成，如下图所示。加法用1、3、4、5段，乘法用1、2、5段，第3段的时间为 $2\Delta t$ ，其余各段的时间均为 $\Delta t$ ，而且流水线的输出可以直接返回输入端或暂存于相应的流水寄存器中。现要在该流水线上计算 $\prod_{i=1}^4 (A_i + B_i)$ ，画出其时空图，并计算其吞吐率、加速比和效率。



# 应用题

第 141 页

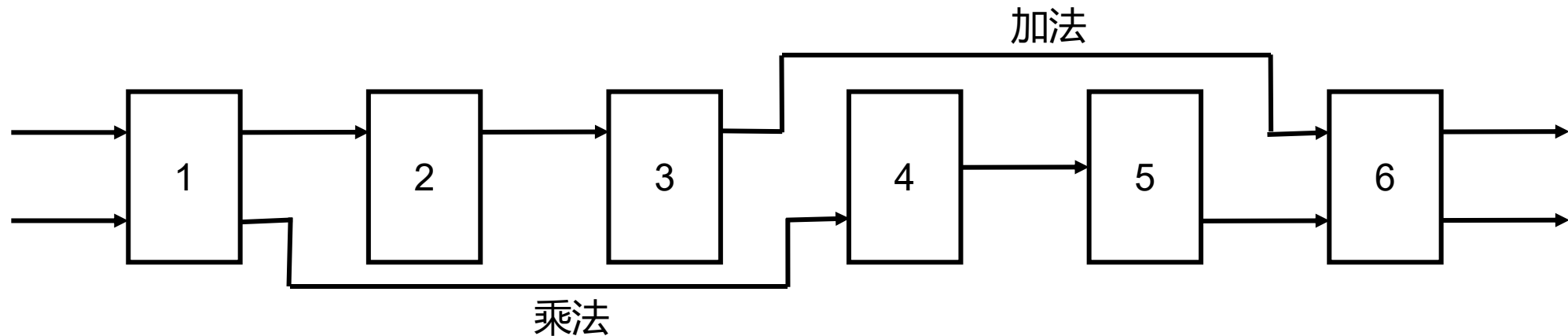




吞吐率:  $TP = \frac{7}{18\Delta t}$       加速比:  $S = \frac{5\Delta t \times 4 + 3\Delta t \times 3}{18\Delta t} = \frac{29\Delta t}{18\Delta t} = 1.61$

效率:  $E = \frac{5\Delta t \times 4 + 3\Delta t \times 3}{18\Delta t \times 5} = \frac{29\Delta t}{90\Delta t} = 0.322$

5、有一条动态多功能流水线由6个功能段组成，如下图所示。



其中 $S_1$ 、 $S_4$ 、 $S_5$ 、 $S_6$ 组成乘法流水线， $S_1$ 、 $S_2$ 、 $S_3$ 、 $S_6$ 组成加法流水线，各个功能段时间均为50ns，假设该流水线的输出结果可以直接返回输入端，而且设置有足够的缓冲寄存器，若以最快的方式用该流水线计算：

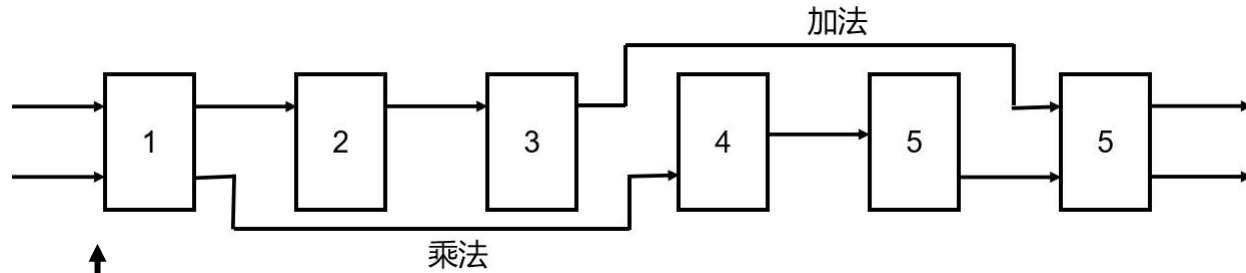
$$\sum_{i=1}^5 x_i y_i z_i$$

(1) 画出时空图； (2) 计算实际的吞吐率、加速比和效率。

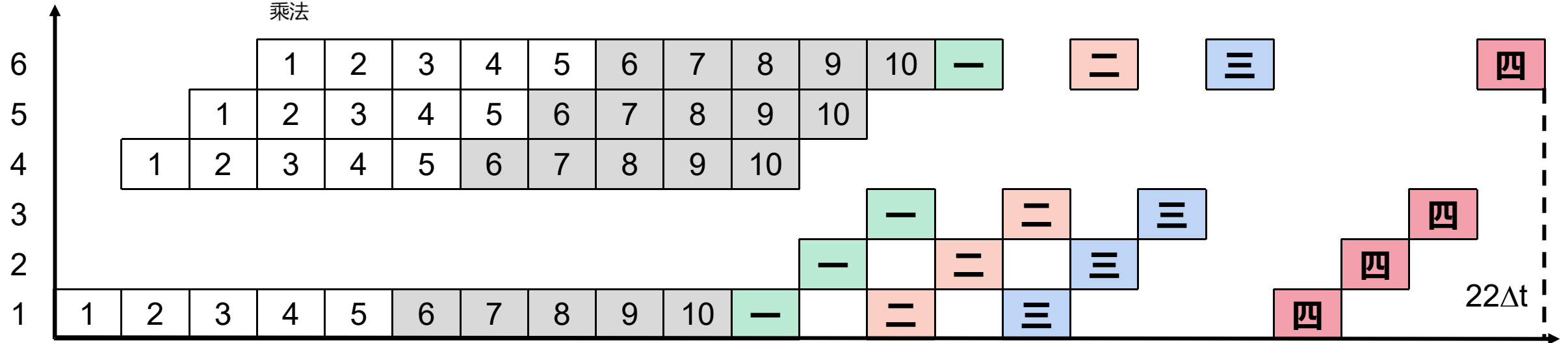
# 应用题

第 144 页

5、有一条动态多功能流水线由6个功能段组成，如下图所示。



算法：先计算 $x_1y_1$ 、...、 $x_5y_5$ ，再计算 $x_1y_1z_1$ 、...、 $x_5y_5z_5$ ，最后相加



吞吐率： $TP = \frac{14}{22\Delta t} = \frac{7}{11\Delta t}$  加速比： $S = \frac{14 \times 4\Delta t}{22\Delta t} = 2.55$  效率： $E = \frac{14 \times 4}{22 \times 6} = 42.42\%$  时间



6、有一个5段流水线，各段执行时间均为 $\Delta t$ ，其预约表如下表所示。

| 功能段 \ 时间 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|----------|---|---|---|---|---|---|---|
| $S_1$    | √ |   |   |   |   |   | √ |
| $S_2$    |   | √ |   |   | √ |   |   |
| $S_3$    |   |   | √ | √ |   |   |   |
| $S_4$    |   |   |   | √ |   |   | √ |
| $S_5$    |   |   |   |   | √ | √ |   |

- (1) 画出流水线任务调度的状态转移图；
- (2) 分别求出允许不等时间间隔调度和等时间间隔调度的两种最优调度策略，以及这两种调度策略的流水线最大吞吐率；
- (3) 若连续输10个任务，求这两种调度策略的流水线实际吞吐率和加速比。

# 应用题

第 146 页

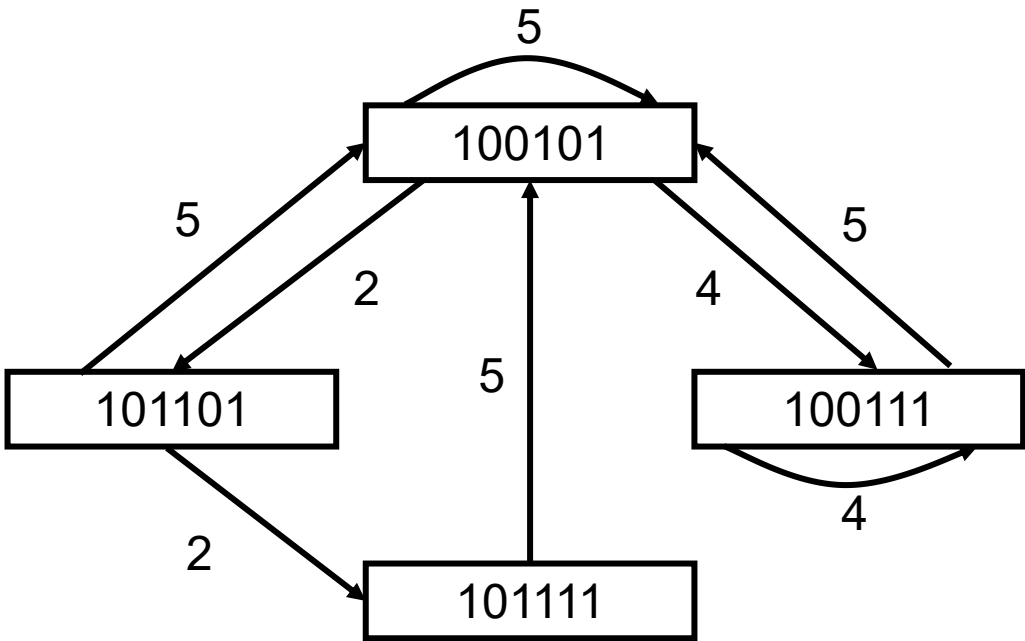
6、有一个5段流水线，各段执行时间均为 $\Delta t$ ，其预约表如下表所示。

| 功能段 \ 时间 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|----------|---|---|---|---|---|---|---|
| $S_1$    | √ |   |   |   |   |   | √ |
| $S_2$    |   | √ |   |   | √ |   |   |
| $S_3$    |   |   | √ | √ |   |   |   |
| $S_4$    |   |   |   | √ |   |   | √ |
| $S_5$    |   |   |   |   | √ | √ |   |

解：（1）由预约表得出禁止表： $F=\{6, 3, 1\}$ ，由禁止表得到冲突向量： $C_0=(100101)$ 。

6、有一个5段流水线，各段执行时间均为 $\Delta t$ ，其预约表如下表所示。

解：（1）由预约表得出禁止表： $F=\{6, 3, 1\}$ ，由禁止表得到冲突向量： $C_0=(100101)$ 。



| 调度策略      | 平均延迟时间        |
|-----------|---------------|
| (2, 2, 5) | $3\Delta t$   |
| (2, 5)    | $3.5\Delta t$ |
| (4)       | $4\Delta t$   |
| (4, 5)    | $4.5\Delta t$ |
| (5)       | $5\Delta t$   |

- 允许不等时间间隔调度的最优调度策略是 (2, 2, 5)，流水线最大吞吐率为 $1/3\Delta t$
- 等时间间隔调度的最优调度策略是 (4)，流水线最大吞吐率为 $1/4\Delta t$

6、有一个5段流水线，各段执行时间均为 $\Delta t$ ，其预约表如下表所示。

解：（2）按最优调度策略是（2, 2, 5），连续输入10个任务的流水线实际吞吐率与加速比分别为：

$$TP_1 = \frac{10}{(2 + 2 + 5 + 2 + 2 + 5 + 2 + 2 + 5 + 7) \Delta t} = \frac{10}{34\Delta t}$$

$$S_1 = \frac{10 \times 7\Delta t}{(2 + 2 + 5 + 2 + 2 + 5 + 2 + 2 + 5 + 7) \Delta t} = 2.06$$

按最优调度策略是（4），连续输入10个任务的流水线实际吞吐率与加速比分别为：

$$TP_2 = \frac{10}{(4 \times 9 + 7) \Delta t} = \frac{10}{43\Delta t}$$

$$S_2 = \frac{10 \times 7\Delta t}{(4 \times 9 + 7) \Delta t} = 1.63$$

## 第4章 向量处理器